Kalkulačka v postfixové notaci na čipu RP2040

KŘESŤANSKÉ GYMNÁZIUM,
KOZINOVA 1000,
102 00 PRAHA 10 – HOSTIVAŘ

ONDŘEJ BĚHAL, 2025/26

Maturitní práce z informatiky

ZADÁNÍ

Školní rok: 2025/26	**Škola:** Křesťanské gymnázium Kozinova 1000 Praha 10 - Hostivař 102 00
Jméno žáka:	Ondřej Běhal
Název maturitní práce:	Kalkulačka v postfixové notaci na čipu RP2040
Termín odevzdání:	9. dubna 2026
Vedoucí maturitní práce:	Mgr. Ivona Spurná
Oponent maturitní práce:	
Délka obhajoby:	15 min
Způsob zpracování maturitní práce:	Žák zpracuje **maturitní práci v elektronické podobě** v programu vhodném ke zpracování – dle charakteru práce. Průběžné výstupy i finální výstup odevzdá elektronicky do vyhrazeného místa určeném vedoucím maturitní práce. Dále žák připraví **prezentaci**, která bude dokumentovat danou práci, její průběh a závěrečný stav. Tato prezentace bude taktéž odevzdána jako soubor do vyhrazeného místa určeném vedoucím maturitní práce.

	Dále žák připraví textovou zprávu – **dokumentaci** – v rozsahu a obsahu definovaném ve *Specifikaci textové části maturitní práce z informatiky*, která je na školních webových stránkách. Tuto dokumentaci odevzdá jak elektronicky v editovatelné podobě (MS Word) a verzi pro tisk (PDF) do vyhrazeného místa určeném vedoucím maturitní práce, tak v tištěné podobě v jednom výtisku. Z hlediska autorství musí být práce provedena výše uvedeným žákem. Uvede použité zdroje v dokumentaci i prezentaci.
Hodnocení:	Z provedení maturitní práce (tj. vlastní praktická práce, soubory prezentace a dokumentace) získá žák na konci školního roku známku odpovídající celkovému bodovému zisku z výstupů. 100–90 bodů – výborný 89–75 bodů – chvalitebný 74–50 bodů – dobrý 49–25 bodů – dostatečný 24–0 bodů – nedostatečný Celkové hodnocení maturitní zkoušky z informatiky: celková známka bude reflektovat z 50 % získané hodnocení z provedení maturitní práce (viz výše) a z 50 % ústní obhajobu této práce u maturitní zkoušky. Žádná z těchto známek nesmí být 5 (nedostatečná), jinak je celkové hodnocení 5 (nedostatečná).
Výstupy:	1. výstup – 1. část maturitní práce – max 15 bodů 2. výstup – 2. část maturitní práce – max 15 bodů 3. výstup – 3. část maturitní práce – max 15 bodů 4. výstup – 4. část maturitní práce – max 15 bodů 5. výstup – soubory prezentace a dokumentace – max 40 bodů
Termíny výstupů:	1. výstup – 13. října 2025 2. výstup – 1. prosince 2025 3. výstup – 12. ledna 2026 4. výstup – 23. února 2026 5. výstup – 9. dubna 2026
Zadání maturitní práce:	Cíl práce:

	Vytvořit jednoduchou kalkulačku využívající postfixovou notaci matematických výrazů (také známá jako Reverzní polská notace – RPN), jež je založena na zásobníku (stack) a nevyžaduje používání závorek ani vyhodnocování precedence operací, neboť pořadí operací je již v zápisu samotném. Kalkulačka není programem běžícím pod operačním systémem, ale je naprogramovaná na „holém železe“ (bare metal) přímo na mikrořadiči RP2040 od britské firmy Raspberry Pi Ltd., konkrétně na vývojové desce Raspberry Pico. Výstup ukazuje uživateli na 128x64bodovém OLED displeji založeném na čipu SSD1306. Vstup obdrží přes klávesnici přes sériové rozhraní UART nebo SWD. Průběžné výstupy: 1. výstup: Žák dokáže využít již existující ekosystém knihoven a nástrojů, aby dokázal napsat kód pro mikrořadič, který posléze umí zkompilovat, nahrát na mikrořadič přes ladící sériové rozhraní SWD a spustit svůj program. 2. výstup: Žák dokáže pomocí vhodných knihoven komunikovat s OLED displejem přes sběrnici I^2C, dokáže na displej vykreslovat jednoduché obrazce i text. 3. výstup: Žák dokáže vytvořit objekt reprezentující výše zmíněný zásobník. Dokáže ho inicializovat, přidat položku na jeho vrchol, odebrat položku z jeho vrcholu, dotázat se na vrchní položku bez jejího odstranění, a dotázat se na prázdnost zásobníku. 4. výstup: Žák dokáže vykreslovat zásobník na displej, i když se na displej nevejde celý; dokáže na něj přidávat nové číselné hodnoty a na nich provádět matematické operace jako sčítání, odčítání, násobení, dělení.

Prohlášení

Prohlašuji, že jsem maturitní práci zpracoval samostatně a že jsem uvedl všechny použité informační zdroje a literaturu v seznamu použitých zdrojů.

Poděkování

Abstrakt

Tato maturitní práce se zabývá vytvořením jednoduché kalkulačky vytvořené v programovacím jazyce Rust. Program kalkulačky je spouštěn na mikrořadiči RP2040 od společnosti Raspberry Pi Ltd., resp. na jeho vývojové desce Raspberry Pi Pico od stejného výrobce, a svůj výstup vykresluje na OLED displeji. Kalkulačka používá tzv. postfixovou notaci matematických výrazů (též známá jako zpětná polská notace – RPN), kdy operátor následuje *po* obou hodnotách, na kterých má být operace vykonána. Cílem práce bylo se naučit pracovat s čipem RP2040, obzvláště při jeho programování v jazyce Rust namísto obvyklejších jazyků jako C nebo C++ podporovaných oficiálním balíčkem pro vývoj softwaru od výrobce čipu.

Klíčová slova

Kalkulačka, Rust, RP2040, Raspberry Pi Pico, nízkoúrovňové programování, postfixová notace, RPN

Abstract (in English)

This graduation thesis focuses on creating a simple calculator created in the programming language Rust. The programme is run on the RP2040 microcontroller made by Raspberry Pi Ltd., or rather its development board Raspberry Pi Pico by the same maker, and it renders its output on an OLED display. The calculator uses the so-called postfix notation of mathematical expressions (also known as reverse Polish notation – RPN), when the operator comes *after* both operands on which the operation shall be carried out. The goal of the project was to learn to work with the RP2040 chip, especially when programming it in the Rust language instead of more common languages like C or C++ supported by the official software development kit made by the chip maker.

Keywords

Calculator, Rust, RP2040, Raspberry Pi Pico, low-level programming, postfix notation, RPN

Obsah

Prohlášení ... 5

Poděkování ... 6

Abstrakt ... 7

Klíčová slova ... 7

Abstract (in English) ... 7

Keywords ... 7

Obsah ... 8

Seznam obrázků ... 11

ÚVOD ... 14

TEORETICKÁ ČÁST ... 15

1 Mikrořadiče ... 15

2 Postfixová notace – RPN ... 16

3 Specifika čipu RP2040 ... 18

4 Protokoly ... 20

4.1 UART ... 20

4.2 I^2C ... 22

4.3 SWD ... 25

5 OLED displej SSD1306 ... 25

6 Programovací jazyk Rust ... 26

6.1 Správa paměti ... 26

6.2 Proměnné ... 29

6.2.1 Rozsahy a vlastnictví ... 29

6.2.2 Stínování a měnitelnost ... 31

6.2.3 Typy proměnných a jejich inference ... 33

6.2.4 Neinicializované proměnné ... 36

6.2.5 Reference ... 37

6.3 Struktury a enumy ... 43
6.3.1 Struktury ... 43
6.3.2 Výčtové typy – enumy ... 46
6.3.3 Použití enumů pro zpracovávání chyb ... 47
6.3.4 Match výrazy, vzorce ... 49
6.4 Generické parametry a traity ... 52
6.4.1 Často používané traity ... 58
6.4.2 Iterátory ... 60
6.5 Closure ... 63
6.5.1 Ukazatele na funkce a Fn* traity ... 66
6.6 Moduly ... 68
PRAKTICKÁ ČÁST ... 69
1. Hardware ... 69
1.1 Napájení ... 70
2 Podpůrné nástroje ... 70
2.1 Cargo ... 70
2.2 probe-rs ... 71
2.3 Linker ... 72
2.4 Ostatní soubory ... 74
3 Knihovny ... 75
3.1 HAL ... 75
3.2 defmt ... 76
3.3 Grafika ... 76
3.4 Ostatní ... 77
4 Aplikační kód ... 77
4.1 main.rs ... 77
4.1.1 Okolní deklarace ... 77
4.1.2 Inicializace ... 78

4.1.3 Hlavní smyčka 79

4.1.4 disp_grave_error(), disp_error() 80

4.1.5 parse_textbox() 81

4.2 command_mode.rs 81

4.3 custom_error.rs 82

4.4 decfix.rs 83

4.5 stack.rs 85

4.6 textbox.rs 86

4.7 Další soubory 87

ZÁVĚR 88

Zdroje 90

Seznam obrázků

Obrázek 1: Příklad UART přenosu jednoho bajtu. Zdroj: vlastní, pomocí logického analyzátoru PulseView. 20
Obrázek 2: Příklad změřeného trvání jednoho bitu z UART přenosu. Zdroj: vlastní, pomocí logického analyzátoru PulseView. 21
Obrázek 3: Vzorek chybného kódu pro demonstraci rozsahů. Zdroj: vlastní. 30
Obrázek 4: Chybová hláška po pokusu o kompilaci kódu z obrázku 3. Zdroj: vlastní. 30
Obrázek 5: Vzorek správného kódu – upraveného kódu z obrázku 3. Zdroj: vlastní. 31
Obrázek 6: Vzorek kódu, který se nechová dle očekávání kvůli stínování proměnných. Zdroj: vlastní. 31
Obrázek 7: Vzorek kódu, kde měníme hodnotu proměnné a který selže. Zdroj: vlastní. 32
Obrázek 8: Chybová hláška pro kód z obrázku 7. Zdroj: vlastní. 32
Obrázek 9: Upravený vzorek kódu s klíčovým slovem mut. Zdroj: vlastní. 33
Obrázek 10: Příklady explicitního i inferovaného určení typu proměnné. Zdroj: vlastní. 33
Obrázek 11: Rust nám zakazuje použít literal větší než maximální hodnota typu proměnné. Zdroj: vlastní. 34
Obrázek 12: Vzorek kódu, který automaticky určí vhodný typ proměnné. Zdroj: vlastní. 34
Obrázek 13: Příklad použití dalších primitivních typů. Zdroj: vlastní. 35
Obrázek 14: Vzorek kódu s neinicializovanými proměnnými. Zdroj: vlastní. 36
Obrázek 15: Vzorek kódu je špatný, protože se pokouší číst hodnotu proměnné před inicializací. Zdroj: vlastní. 36
Obrázek 16: Chybová hláška po pokusu o kompilaci kódu z obrázku 15. Zdroj: vlastní. 37
Obrázek 17: Vzorek kódu ukazující předávání přes hodnotu. Zdroj: vlastní. 37
Obrázek 18: Chybová hláška nás varuje, že předáním přes hodnotu ztrácíme vlastnictví nad proměnnou. Zdroj: vlastní. 38
Obrázek 19: Vzor kódu s předáváním přes referenci. Zdroj: vlastní. 39
Obrázek 20: Vzorek kódu je chybný, protože reference má delší životnost než proměnná samotná. Zdroj: vlastní. 40
Obrázek 21: Chybová hláška pro kód z obrázku 20. Zdroj: vlastní. 40
Obrázek 22: Tento vzorek kódu je neplatný, protože nedeklaruje životnost reference uvnitř struktury. Zdroj: vlastní. 41
Obrázek 23: Kompilátor nám v chybové hlášce radí, jak deklarovat životnost. Zdroj: vlastní. 41
Obrázek 24: Vzorek kódu, kde se snažíme zároveň mít měnitelnou i neměnnou referenci. Zdroj: vlastní. 42
Obrázek 25: Chybová hláška po pokusu o kompilaci kódu z obrázku 24. Zdroj: vlastní. 42

Obrázek 26: Příklad kódu, který využívá RefCell ze standardní knihovny pro vyřešení problému z obrázků 24 a 25. Zdroj: vlastní. ... 43
Obrázek 27: Příklad definice struktury a jejich položek. Zdroj: (1), kapitola 5.1 ... 44
Obrázek 28: Příklad vytvoření struktury z obrázku 27, vyplnění jejích položek hodnotami, a uložení nově vzniklé struktury do proměnné. Zdroj: (1), kapitola 5.1 ... 44
Obrázek 29: Příklad deklarace metody. Zdroj: (1), kapitola 5.3, upraveno ... 45
Obrázek 30: Příklad metody konstruktoru, která nepojímá parametr self, ale navrací instanci Self. Zdroj: (1), kapitola 5.3 ... 45
Obrázek 31: Příklad deklarace jednoduchého enumu. Zdroj: vlastní. ... 46
Obrázek 32: Složitější způsob, jak přenést zároveň možnost enumu i nějaká data. Zdroj: (1), kapitola 6.1 ... 46
Obrázek 33: Příklad enumu s daty ve svých možnostech. Zdroj: vlastní. ... 47
Obrázek 34: Definice enumu Option. Zdroj: (1), kapitola 6.1 ... 48
Obrázek 35: Definice enumu Result. Zdroj: (1), kapitola 9.2 ... 48
Obrázek 36: Příklad použití metod .unwrap() a .expect(). Zdroj: (1), kapitola 9.2, upraveno. Importy vynechány ... 49
Obrázek 37: Tento program zpanikaří, protože se pokusí o přístup prvku v poli mimo rozsah. Zdroj: (Klabnik, a další, 2025), kapitola 9.1 ... 49
Obrázek 38: Chybová hláška poté, co program z obrázku 37 zpanikařil. Zdroj: vlastní ... 49
Obrázek 39: Příklad použití match. Zdroj: (1), kapitola 9.2, upraveno, importy vynechány ... 51
Obrázek 40: Příklad použití operátoru otazníku. Zdroj: (1), kapitola 9.2. Importy vynechány ... 51
Obrázek 41: Příklad výrazů if-let a let-else. Zdroj: vlastní. ... 52
Obrázek 42: Příklad zadání typů v generickém enumu Option. Zdroj: vlastní. ... 53
Obrázek 43: Chybný kód generické funkce, kterému chybí omezení generického parametru. Zdroj: (1), kapitola 10.1, upraveno ... 54
Obrázek 44: Chybová hláška po pokusu o kompilaci kódu z obrázku 43. Zdroj: vlastní ... 54
Obrázek 45: Příklad omezení generického parametru pomocí klíčového slova where. Zdroj: (1), kapitola 10.2 ... 54
Obrázek 46: Příklad omezení metody. Zdroj: (1), kapitola 10.2, upraveno ... 55
Obrázek 47: Opravený kód z obrázku 43. Zdroj: vlastní. ... 55
Obrázek 48: Příklad deklarace a použití asociovaných typů traitu. Zdroj: (2), kapitola 14.8.2 ... 56
Obrázek 49: Příklad deklarace vlastního traitu i jeho implementace na strukturách. Zdroj: (1), kapitola 10.2, upraveno ... 57
Obrázek 50: Příklad pro ilustraci rozdílu mezi Debug a Display. Zdroj: vlastní ... 60
Obrázek 51: Příklad funkcí s návratovými typy určenými pomocí klíčového slova impl. Zdroj: vlastní. 62

Obrázek 52: Definice closure. Zdroj: (1), kapitola 13.1, upraveno 63
Obrázek 53: Nesprávné použití closure. Zdroj: (1), kapitola 13.1 63
Obrázek 54: Chybová hláška po pokusu o kompilaci kódu z obrázku 53. Zdroj: vlastní 64
Obrázek 55: Tento kód se nezkompiluje kvůli pokusu o koexistenci měnitelné a neměnné reference. Zdroj: vlastní 65
Obrázek 56: Chybová hláška po pokusu o kompilaci kódu z obrázku 55. Zdroj: vlastní 65
Obrázek 57: Příklad užití klíčového slova move. Zdroj: vlastní 66
Obrázek 58: Příklad syntaxe traitů Fn* a ukazatelů na funkce. Zdroj: vlastní 67
Obrázek 59: Schéma zapojení hardwaru. Zdroj: vlastní, pomocí schematického editoru balíku KiCad 69
Obrázek 60: Poškození paměti při běžném přetečení zásobníku. Zdroj: (5)..... 73
Obrázek 61: Rozložení linkerem flip-link, které místo toho vyvolá hardwarovou chybu. Zdroj: (5)..... 73
Obrázek 62: Rozložení paměti, se značkami adres popsaných v datasheetu. Zdroj: vlastní 74

ÚVOD

Tato maturitní práce se zaměřuje na vytvoření funkční kalkulačky. Hlavním cílem je vytvořit program, který dokáže přijímat vstup od uživatele, provádět matematické operace a vykreslovat výstup zpět na displej.

Součástí práce je také použití již existujících nástrojů v ekosystému jazyka Rust, jako i mnoha knihoven a projektů, pro jednodušší tvorbu samotného programu a jeho následného nahrání na paměť flash, ze které čip čte své instrukce. Projekt slouží jako praktická ukázka nízkoúrovňového programování v jazyce Rust namísto obvyklého C nebo C++.

Výsledkem je jednoduchá kalkulačka v postfixové notaci, která lze pak dále rozšířit či upravit. Zároveň představuje zkušenost v programování mikrořadičů, což je mnohdy složitější pro začátečníky a pro což není online zdaleka tolik tutoriálů.

TEORETICKÁ ČÁST

Není žádným tajemstvím, že celý náš současný svět je postaven na počítačích. Ale ne každý počítač musí být krabice ležící na stole. Existují i drobné počítače, které zabírají prostor jednotek centimetrů krychlových, které mají omezený výkon a specifický účel. Těmto typům počítačů říkáme mikropočítače. Najdeme je prakticky všude kolem nás: od domácích spotřebičů jako pračka nebo mikrovlnka, přes robotické vysavače a chytré žárovky, až po industriální procesy, kde se čas měří na mikrosekundy a kde to nejdrobnější opoždění může mít vážné následky.

S těmito mikropočítači se zachází zcela jinak než s běžnými počítači, jinak se napájí, jinak s nimi komunikujeme, jinak je programujeme. A i to, jakým způsobem jsou programovány, se může mnohdy lišit od klasického způsobu programování.

1 Mikrořadiče

Konkrétní čipy, které jsou pomyslnými „mozky“ těchto mikropočítačů, nazýváme mikrořadiče (nebo mikrokontroléry, z anglického „microcontroller“). Tyto čipy do sebe integrují procesor, paměť, a obvykle i další periferie usnadňující komunikaci s jinými zařízeními.

Procesor („CPU = central processing unit“) je mozkem samotného mikrořadiče, vykonává strojový kód – instrukce v jazyku symbolických adres („assembly“ / JSA) odpovídající instrukční sadě daného CPU. Ačkoli současné počítače používají až na pár výjimek (jako např. M-čipy v nových Apple počítačích) instrukční sadu x86-64, v kontextu mikrořadičů se používají častěji jednodušší a energeticky efektivnější sady jako ARM, Atmel AVR nebo RISC-V. Procesor pracuje s registry, což je několik míst přímo uvnitř procesoru, které uchovávají data. Obvykle je jich sice velmi málo (několik desítek) a každý z nich dokáže uchovat jen jedno „slovo“ („word“ – jednotka informace závislá na typu procesoru, u 32bitových procesorů je rovna 32 bitům, u 64bitových je to 64 bitů atd.), ale jsou velmi těsně přimknuty k procesoru, a tudíž velmi rychlé, a jen s daty v nich dokáží instrukce pracovat. Ostatní data musí být do nich zkopírována z adresního prostoru (viz níže).

Paměť dělíme na paměť flash, RAM, ROM a OTP. Paměť flash je přepisovatelná, ale poněkud pomalá, takže ji CPU používá ke čtení dat, které se nemění (jako třeba instrukce, které má vykonávat). Její výhodou je schopnost uchovat si data i po odpojení napájení, tedy říkáme, že je „nevolatilní“. Paměť RAM („=random access memory“ – náhodně přistupovaná paměť) si sice data po vypnutí neuchovává, je „volatilní“, ale je možné do ní poměrně rychle přistupovat a přepisovat ji např. pro mezivýsledky operací a jiných dat, se kterými procesor pracuje. Procesor s ní pracuje tak, že ji namapuje do svého adresního prostoru (viz níže). Paměť ROM („=read only memory“ – paměť pouze pro čtení)

je nepřepsatelná a její obsah je určen již během výroby čipu, proto do ní ukládá data výrobce čipu, ať už jsou to informace o čipu samém, nebo kód pro inicializaci čipu a spuštění uživatelského programu. Do OTP („=one-time programmable“ – jednorázově programovatelná) paměti se sice zapisovat dá, ale pouze jednou: po prvním přepsání již nelze změnit. Uchovávají se v ní např. kryptografické klíče nebo jiná data, které si uživatel čipu nastaví obvykle již v továrně při vložení čipu do koncového produktu, a které uživatel koncového produktu nesmí změnit.

Pro přístup k čemukoli jinému, než vlastním registrům potřebuje procesor adresy – číselné hodnoty jednoznačně označující nějaké místo v paměti (nebo jinde, viz později). Rozsah všech těchto číselných adres se nazývá adresní prostor. V případě paměti označuje adresa konkrétní bajt, ke kterému přistupujeme, ale adresní rozsah neobsahuje jen paměť. Komunikaci s periferiemi zvládá totiž CPU tak, že periferie svoje vstupy/výstupy zpřístupňují jako registry, ke kterým CPU přistupuje jako k čtveřicím bajtů v paměti (protože každý registr má 32 bitů – 4 bajty). Každý registr má tudíž přiřazenou adresu ze stejného adresního rozsahu jako paměť. Když nějaké skupině bajtů přidělíme číselný rozsah v adresním prostoru, říkáme, že tuto skupinu namapujeme do adresního rozsahu.

Periferie jsou další jednotky na čipu, které slouží k jiné funkci než vykonávání instrukcí a uchovávání dat a které lze ovládat z programu právě tak, že jsou namapovány do adresního prostoru CPU. Některé z nich jsou zodpovědné za základní funkce nutné ke správnému fungování mikrořadiče, další za různé interní funkce, jiné za komunikaci s okolním světem skrze elektrické signály. Interními periferiemi může být třeba časovač počítající mikrosekundy od startu systému. Periferie komunikující s okolním světem jsou například univerzální vstup-výstup („GPIO = general purpose input/output“), periferie implementující určitý komunikační protokol jako I^2C nebo UART, nebo i generátory signálu PWM („=pulse width modulation“ – signál modulovaný šířkou pulzu) a analogově-digitální převodníky („ADC = analog-digital converter“). GPIO je asi nejjednodušší z nich, dokáže pouze číst digitální elektrický signál zapnuto-vypnuto na pinech nastavených jako vstup, nebo podobný signál vyvolávat na pinech nastavených jako výstup.

2 Postfixová notace – RPN

Běžné kalkulačky operují podobně, jak jsme zvyklí psát matematické výrazy. Od základní školy se děti učí tzv. infixové notaci, kdy „binární operátory“ (symboly označující určitou matematickou operaci, která přijímá dvě vstupní čísla, např. $+, -, \times, \div$) se zapisují mezi jejich „operandy“ (čísla, na kterých má být operace vykonána – sčítance, dělitele a dělenci,…). Příkladem infixového zápisu je například výraz $1 + 1$. Tento zápis sice zná každý od dětství, ale má své nevýhody: pokud se číslo nachází mezi dvěma operátory, je potřeba použít závorek nebo pravidel o precedenci operací (násobení a dělení nad sčítáním a odčítáním). Pokud tedy do „hloupé“ kalkulačky, takové, jaké jsou vhodné pouze

na základní škole, zadá uživatel výraz $1 + 2 \times 3$ klávesu po klávese, výsledek by byl nesprávný, protože kalkulačka provádí operace okamžitě a vyřešila by tudíž vstup jako $(1 + 2) \times 3$. Pokud použijeme pokročilejší středo- nebo vysokoškolskou vědeckou kalkulačku, taková kalkulačka nám umožní zadat celý výraz v infixové notaci a pak stisknout klávesu $=$, která celý výraz vyřeší. Jejich nevýhodou je však komplikovanost programu na nich běžícího, který musí správně zpracovat precedenci a závorky.

Je však další způsob zápisu matematických výrazů, který tato pravidla nevyžaduje, který nevyžaduje ani závorky nebo pravidla o precedenci, jehož zápis je zcela jednoznačný: postfixová notace. V této notaci, také nazývané RPN („=reverse Polish notation“ – zpětná polská notace, polská podle národnosti logika Jana Łukasiewicze) se operandy nezapisují mezi čísla, na kterých má být operace vykonána, ale za obě. Čísla jsou oddělena prostou mezerou. Tento přístup eliminuje nutnost řešit precedenci operací nebo závorky, protože pokud je jeden z operandů ne číslo, ale další výraz, je tento výraz vyřešen vždy jako první.

Výše zmíněný výraz $1 + 2 \times 3$ by tedy šel do RPN přepsat buď jako $1\ 2\ 3 \times +$ při dodržení přednosti násobení před sčítáním, nebo, pokud by byl upraven na $(1 + 2) \times 3$, tak by byl přepsán jako $1\ 2 + 3 \times$. Tyto dva zápisy jsou tedy různé podle toho, jaká operace by měla přednost, a s imaginárními závorkami by byly řešeny jako $1\ (2\ 3 \times) +$ a $(1\ 2 +)\ 3 \times$ – operandy jsou řešeny jednoduše zleva doprava. Pro převod z infixové notace do RPN je používán například tzv. Shunting yard algoritmus vynalezený známým informatikem E. Dijkstrou, ačkoli mnoho uživatelů takovýchto kalkulaček dokáží takovýto převod provést v hlavě, případně pouze pomocí papíry a tužky.

Tento zápis je často modelován právě pomocí datové struktury zásobníku, kdy každé číslo je přidáno na vrchol zásobníku a operace vezmou první dvě čísla svrchu a provedou na nich operaci. Nekomutativní operace (např. odčítání a dělení) jsou tradičně brány tak, že vrchnější položka na zásobníku je druhým operandem, aby šlo na kalkulačce stisknout klávesy `2`, pak `Enter`, pak `1` a nakonec `–`, a výraz byl opravdu řešen jako $2\ 1 -$. Klávesa `Enter` pouze rozděluje dvě čísla, aby nebylo zadání interpretováno jako $21 -$ (= dvacet jedna mínus), přesněji vezme všechny číslice, které byly do té doby zadány, zpracuje je jako číslo a vloží na zásobník. Některé kalkulačky vyžadují stisknout `Enter` mezi zadáním čísla a operátorem, jiné (jako ta naše) jeho akci vykonají implicitně při stisknutí klávesy operátoru, pokud byly mezi posledním stisknutí `Enter` napsány nějaké číslice.

Ačkoli je dnes RPN používaná pouze zřídka, historicky byla velmi populární například v produktové řadě kalkulaček HP. Model HP-12C z roku 1981 je dodnes považován za nejdéle vyráběnou a nejprodávanější kalkulačku, a stal se *de facto* standardem v oblasti financí. Stejně jako první kapesní vědecká kalkulačka na světě, HP-35, používal HP-12C notaci RPN.

3 Specifika čipu RP2040

Čip RP2040 je mikrořadič od britské společnosti Raspberry Pi Ltd., která je známá vyráběním jednodeskových počítačů Raspbery Pi. Poněkud nepřekvapivě, první dvě písmena „RP“ v jeho názvu jsou iniciály společnosti, ale číslo 2040 je poněkud zajímavější. V datasheetu (dokumentu, který výrobce vydává pro svůj čip a ve kterém uvádí všechny detailní informace o jeho používání a fungování) v sekci 1.1 říká výrobce, že první číslice označuje počet jader samotného procesoru (zde 2), druhá číslice udává zhruba typ procesoru (zde ARM Cortex-M0+, postavený na architektuře ARMv6-M), a třetí a čtvrté číslice udávají skrze složitější matematický vzoreček množství RAM a nevolatilní paměti na čipu (zde 4 a 0, protože konkrétní čip má 264 kiB RAM a žádnou nevolatilní paměť na čipu). Je to první pokus tohoto výrobce o tvorbu mikrořadiče, a vzhledem k jeho popularitě, široké adopci naskrz průmyslem a podpoře skrze knihovny i dokumentaci, byl vybrán jako vhodná platforma pro tento projekt. Jeho datasheet je navíc velmi kvalitně napsaný a dobře čitelný.

Avšak tento konkrétní mikrořadič se od své konkurence v mnoha bodech výrazně liší, ať už pozitivně nebo negativně, a je nutno na tyto vlastnosti brát ohled. Mezi ně patří například periferie PIO, dvě procesorová jádra, specifický a poměrně složitý proces zavádění, rozsáhlá podpora USB, otevřený zdrojový kód ROM nebo absence FPU či analogových periferií mimo již zmíněný ADC, který má navíc pověst nízké kvality.

PIO („=programmable input-output“ – programovatelný vstup-výstup) je asi nejvýraznějším, a mezi ostatními, známějšími mikrořadiči unikátním rozdílem. Jedná se o dvojici periferií zahrnujících celkem 8 konečných automatů („state machine“). V podstatě se jedná o jakési jednoduché procesory úzce specializované na vstupně-výstupní operace, se zaměřením na determinismus a přesné časování. Dokáží využívat jakékoli GPIO piny a jsou programovány v jejich vlastním jazyce symbolických adres poskytujícím celkem devět 16bitových instrukcí. Protože každá instrukce trvá jen jeden cyklus hodinového signálu ovládaného děličem frekvence systémových hodin, pokud není nastaveno jinak, umožňuje PIO implementaci mnohých komunikačních protokolů, včetně těch, které neimplementují jiné periferie na čipu. Ačkoli je tato periferie velmi flexibilní, výkonná, a užitečná, je poměrně komplexní a v tomto projektu pro ni nemáme využití.

Mít rovnou dvě procesorová jádra je v této cenové kategorii poměrně vzácnou vlastností, a umožňuje to skutečně paralelní zpracovávání, jako i vyšší výkon dostupný pro složité operace, avšak v zájmu jednoduchosti nevyužíváme v tomto projektu druhé jádro, jen to první. Hned po startu je skrze zavaděč první fáze v ROM uvedeno druhé jádro do režimu spánku, dokud ho uživatelský program z prvního jádra neprobudí.

Proces zavádění (startování) čipu je také poměrně neobvyklý. Celkem má technicky vzato tři fáze. První fáze je hned po startu, kdy začne procesor vykonávat instrukce rovnou z ROM. ROM obsahuje zavaděč první fáze, který načte prvních 256 bajtů z flash paměti a zkontroluje jejich kontrolní součet, aby ověřil, že se tam nachází platná data. Pokud odpovídá, spustí zavaděč druhé fáze, pokud ne, spustí USB zavaděč, který se z pohledu připojeného počítače chová jako standardní USB externí disk, na který lze zkopírovat program ve formátu UF2, který bude nahrán do paměti flash a spuštěn (to je jednodušší způsob, jak tento mikrořadič programovat). Pomocí tlačítka BOOTSEL lze také vynutit přechod rovnou do USB zavaděče. Zavaděč druhé fáze, známý jako boot2, je zprostředkován uživatelem, ačkoli výrobce poskytuje několik ověřených verzí pro různé flash čipy. Jeho jediným úkolem je inicializovat flash čip podle specifik jeho modelu, aby z něj šel kód spouštět co nejvýkonněji. Kód je z něj spouštěn v režimu XIP, kdy je celý rozsah paměti namapován do adresního rozsahu jako paměť pouze pro čtení a z pohledu softwaru je více méně transparentní. Protože však každý flash čip vyžaduje jinou sekvenci příkazů pro svůj nejvýkonnější mód, musí být inicializační kód dodán uživatelem a ne výrobcem v ROM, aby šel měnit. A nakonec, po dokončení zavaděče druhé fáze, se spustí zavaděč třetí fáze, taktéž známý jako „reset handler“, který je specifický pro konkrétní programovací jazyk a který inicializuje běhové prostředí („runtime environment“), které daný programovací jazyk vyžaduje, aby konečně mohl být spuštěn uživatelský program ve specifikovaném vstupním bodě („entry point“), odkud začne uživatelský program běžet.

Podpora USB již byla zmíněna. ROM obsahuje plně funkční zavaděč typu USB MSC („=mass storage class“ – třída velkokapacitního úložiště), který se chová jako externí disk a který umožňuje programovat celý mikrořadič jednoduše přes USB. Ale kromě toho dokáže USB periferie komunikovat jako jakékoli jiné USB zařízení, od třídy MSC (externí disky) přes HID (klávesnice, myši, herní ovladače) až po CDC (síťové karty, sériové porty, modemy); nebo dokonce dokáže spravovat USB sběrnici sama, převzít roli počítače a komunikovat s jinými USB zařízeními.

Fakt, že zdrojový kód celé ROM je otevřený a každý do něj smí nahlížet nebo s ním pracovat, se sice může na první pohled zdát jako nevýznamné, avšak opak je pravdou: umožňuje to větší transparentnost a zjednodušuje to práci s ROM, když programátor ví, co se v ní děje. Navíc pak ROM obsahuje několik vysoce optimalizovaných funkcí např. pro matematické operace na číslech s plovoucí desetinnou čárkou („floating-point numbers“), kterými částečně nahrazuje absenci dedikované jednotky FPU („=floating-point unit“ – jednotka pro plovoucí desetinnou čárku / matematický koprocesor), která by tyto operace prováděla přímo a výkonněji. Subsystém SIO však obsahuje alespoň celočíselný dělič, který během 8 cyklů hodinového signálu vypočítá celočíselný podíl a zbytek po dělení (zároveň). Použitý procesor Cortex-M0+ zároveň obsahuje zabudovaný celočíselný násobič.

Vývojová deska, kterou používáme, Raspberry Pi Pico, obsahuje kromě samotného čipu také podpůrný hardware jako flash čip, napájecí zdroj a USB konektor, který dohromady umožňuje zakoupit hotový produkt a začít vyvíjet pro tento čip.

4 Protokoly

Pro nás projekt využíváme množství protokolů pro komunikaci, zde budou stručně popsány.

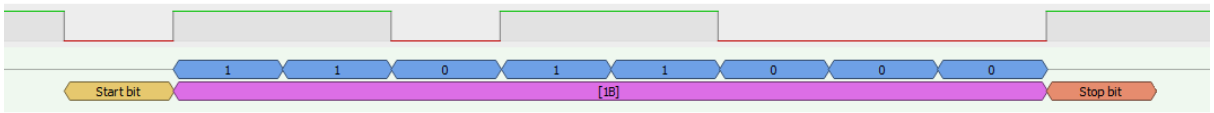
Pro příjem vstupu od uživatele používáme jednoduchý sériový protokol UART, pro komunikaci s displejem používáme sběrnici I^2C a pro nahrávání a ladění programu používáme protokol SWD.

4.1 UART

UART („=universal asynchronous receiver-transmitter“ – univerzální asynchronní přijímač-vysílač) je asi jeden z nejjednodušších způsobů, jak přenášet data mezi dvěma systémy – přenáší holé bajty, bez synchronizace, bez nějakých vyšších protokolů kromě základního způsobu binární elektrické signalizace. Jedná se o asynchronní, sériový, plně duplexní point-to-point protokol; to znamená, že protokol neposílá hodinový signál, bity jsou posílané jeden za druhým namísto několika naráz, je možné přijímat a vysílat data zároveň, a data jsou přenášena mezi dvěma vzájemně rovnocennými uzly.

Pro signalizaci používá protokol UART pouze hodnoty 0 a 1. Pokud se aktuálně žádná data nepřenáší, zůstává linka na hodnotě 1. Pro začátek komunikace je vyslán jeden tzv. start bit o hodnotě 0, aby příjemce věděl, že začíná přenos; poté je vysláno několik bitů přímo tak, jak je máme v úmyslu přenést (bit 1 jako 1, bit 0 jako 0), nejméně významný bit jako první (bit na pozici jedniček v čísle dvojkové soustavy jako první); poté volitelný paritní bit, který však můžeme vynechat; a nakonec jeden nebo více tzv. stop bitů, které jsou vždy o hodnotě 1. Stop bit může trvat nejčastěji dobu jednoho, jednoho a půl nebo dvou normálních bitů.

Na obrázku 1 můžeme vidět příklad UART přenosu bajtu o hodnotě 27 (1B v šestnáctkové soustavě, 00011011 ve dvojkové soustavě). Pod grafem logického stavu (zelená = 1; červená = 0) lze nalézt dekódované bity i značky start a stop bitů. Schéma přenosu je 8N1 – viz níže.

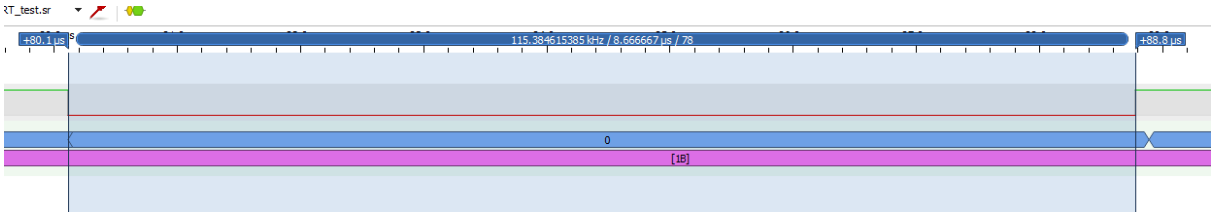


Obrázek 1: Příklad UART přenosu jednoho bajtu. Zdroj: vlastní, pomocí logického analyzátoru PulseView.

Jedním z nejdůležitějších parametrů je rychlost spojení, udávaná v počtech přenesených bitů za sekundu. Jednotka se ale nazývá baud, (potažmo s předponou SI kilobaud / kbaud nebo

megabaud / Mbaud) protože zahrnuje i start, paritní a stop bity, nejen datové bity, a podle této jednotky se rychlost obvykle nazývá „baudrate“. Obvyklými baudrate jsou 110, 300, 9 600 nebo 115 200. Historicky byly podporovány jen určité standardní baudrate, ale v současné době lze díky zlomkovým děličům frekvencí použít téměř jakýkoli baudrate, včetně velmi vysokých rychlostí přes 1 Mbaud. Neboť se jedná o frekvenci, perioda – trvání jednoho bitu v sekundách lze spočítat jednoduše jako $(baudrate)^{-1}$.

Na obrázku 2 logický analyzátor píše, že jeden bit trvá $8.666\,667\ \mu s$, což by odpovídalo frekvenci $115\,384.615\,385\ Hz$, která je velmi blízko použitému baudrate $115\,200\ baud$. Protokol UART naštěstí dokáže tolerovat drobné odchylky od správného baudrate, protože začátkem každého bajtu se opět synchronizuje pomocí start bitu. Zároveň musíme počítat s nepřesností měření.



Obrázek 2: Příklad změřeného trvání jednoho bitu z UART přenosu. Zdroj: vlastní, pomocí logického analyzátoru PulseView.

Paritní bit má 5 možností: „Odd“ („=lichý“), „Even“ („=sudý“), „Mark“ („=značka“), „Space“ („=mezera“) a „None“ („=žádný“). „Odd“ znamená, že paritní bit je 1, pokud je počet bitů hodnoty 1 v přenášených datech sudý, aby byl celkem přenesen lichý počet. „Even“ je opak – nabývá hodnoty 1, pokud je počet 1-bitů lichý, aby byl celkem přenesen sudý počet. Toto je využíváno jako primitivní kontrolní součet, který dokáže někdy detekovat chybu v přenosu, avšak není zcela spolehlivý, protože v případě chyb v sudém počtu bitů nelze chybu detekovat. „Mark“ znamená, že paritní bit je vždy 1, a „Space“ znamená, že je vždy 0, avšak tyto možnosti neposkytují kontrolní součet a jsou tedy poněkud zbytečné – v obvyklých případech je praktičtější paritní bit úplně vynechat (možnost „None“). Vynechání paritního bitu navíc marginálně zrychlí přenos dat, protože pro každý bajt dat je potřeba přenést méně bitů, a tudíž více bitů z každého přenosu je datových.

Celková konfigurace se často zapisuje ve zkrácené formě, například „115200 8N1“, kdy 115200 označuje baudrate, 8 označuje, že jeden bajt je složen z 8 bitů (což historicky nemuselo vždy nutně platit – například původní ASCII tabulka byla jen 7bitová), N, že není použit paritní bit, a 1, že stop bit trvá délku jednoho normálního bitu. Dalším příkladem může být třeba „9600 7E2“ – 9600 baudů, 7 bitů na bajt, paritní bit „Even“, 2 bity dlouhý stop bit.

Detaily jako napěťové úrovně se pak mohou lišit, obvyklými standardy jsou TTL, kdy 0 a 1 jsou přímo napěťové úrovně tranzistorů (obvykle 0 V pro 0 a 1.1 V, 1.8 V, 2.5 V, 3.3 V nebo 5 V pro 1); RS-232

typickým na sériových portech starších počítačů, kdy kladné napětí mezi 3 a 15 V je 0, a stejné záporné napětí je 1; RS-485 obvyklejším v industriálním prostředí, kde přenáší data po sběrnici pomocí diferenciální signalizace – rozdílu mezi dvěma napětími, kdy záporný rozdíl je 1 a kladný rozdíl je 0. Historicky (v době dálnopisů) se také používaly proudové smyčky, kdy přítomnost proudu značí 1 a nepřítomnost 0.

Tato historie se však promítá až do současnosti: důvod, proč nabývá neaktivní linka stálé hodnoty 1, je právě z dob proudových smyček, kdy přerušení smyčky znamenalo, že nešlo přenášet data ani jedním směrem. Když však byla neaktivní linka hodnoty 1, tak po přerušení byla neustále hodnoty 0, což šlo detekovat například tím, že tehdy mechanické dálnopisy začaly dělat velký hluk, protože neustále přijímaly, co se jim zdálo jako nulové bajty. I moderní UART využívají této skutečnosti, aby detekovaly přerušené připojení a vyvolaly signál chyby „framing error“ (když není přijat platný stop bit = stop bit nabývá hodnoty 0) nebo „break condition“ (když linka zůstane neustále na hodnotě 0 po dobu delší než dobu jednoho celého bajtu vč. start, paritních a stop bitů).

Z abstraktního pohledu lze tedy brát UART jako jednoduché tunely pro bajty – dovnitř zapíšu bajt, z druhé strany přečtu (snad) stejný bajt. Tato jednoduchost umožňuje uživateli implementovat vlastní komunikační protokoly, přenášet data nebo text.

Například terminály, které používáme v našem projektu jako vstup pro uživatele, přijímají vstupy z klávesnice a přenášejí je jako text zakódovaný nejčastěji formátem UTF-8 (zpětně kompatibilní nadstavba nad tabulku ASCII). Některé vstupy však nelze zakódovat jako textové znaky. Například klávesové zkratky Ctrl-<písmeno> odešlou bajt, jehož hodnota je rovna pořadí písmena v abecedě (například Ctrl-C tedy odešle bajt hodnoty 03, nebo Ctrl-U odešle hodnotu 21). Pro složitější vstupy jako F-klávesy (F1 až F12) nebo klávesy pro ovládání kurzoru (Home, End, Delete, PgUp, PgDn, šipky) a jejich kombinace s modifikátory (Ctrl, Alt, Shift) jsou použity ANSI escape kódy, které začínají nezobrazitelným ASCII znakem ESC (hodnota 27) a za nimiž následuje několik běžných textových znaků, které mohou reprezentovat určitou klávesu nebo klávesovou zkratku. Například klávesa Delete je reprezentována jako „<ESC>[3~“ a šipka nahoru je „<ESC>[A“. Tyto kódy však dokážou také ovládat terminál, který je přijímá, například vymazat jeho obrazovku, přemístit kurzor nebo nastavit barvu textu. Tyto kódy byly popularizované terminálem DEC VT100 z roku 1978, a staly se *de facto* standardním protokolem terminálů.

4.2 I^2C

Protokol I^2C („=Inter-Integrated Circuit“ – inter-integrovaný obvod; vyslovuje se „eye-squared-see“, neoficiálně také „eye-two-see“ od častého zjednodušeného zápisu I2C) je také velmi hojný, i když trochu složitější protokol, umožňující připojení jednoduchých integrovaných obvodů k mikrořadičům.

Jedná se o synchronní, sériovou, poloduplexní multi-master/multi-slave sběrnici; to znamená, že spolu s daty je přenášen hodinový signál taktující tento přenos, bity jsou opět přenášeny jeden za druhým po jediné datové lince, data jdou přenášet i vysílat, ale pouze v jednom směru naráz, a všechna zařízení jsou propojena ve sběrnicové topologii, kdy všechny stanice připojené ke sběrnici sdílejí jedinou dvojici drátů. Sběrnici ovládá stanice nazývaná „řídící“ („controller“, v dřívější terminologii „master“ – pán), která vysílá hodinový signál a zahajuje komunikaci, a ostatní stanice ovládané řídícím, odpovídající na jeho příkazy, se nazývají stanice „cíl“ („target“, v dřívější terminologii „slave“ – otrok). „Multi-master/multi-slave“ pak jednoduše znamená, že na jediné sběrnici smí existovat více řídících i více cílů naráz.

Celý protokol vyžaduje jen dva dráty – linky SDA a SCL. Aby však mohlo na sběrnici vysílat více stanic naráz, musí být připojeny v konfiguraci s otevřeným kolektorem („open collector“, potažmo „open drain“, protože v současnosti se používají tranzistory MOSFET). To znamená, že ve výchozím stavu je linka tzv. vytažena na napětí logické 1 jen pomocí rezistorů připojených mezi linkou a napájecím napětím (tzv. pull-up resistors – pull-up / vytahovací rezistory) a jednotlivé stanice dokáží linku pouze tzv. stáhnout na logickou nulu tím, že ji skrze svoje tranzistory připojí k elektrické zemi, která má napětí 0 V (protože sama slouží jako reference pro všechna ostatní napětí), a tím vyšlou logickou 0. Po otevření tranzistoru pull-up rezistory zajistí, že se linka vrátí zpět na logickou 1. Čím mají rezistory menší elektrický odpor, tím jsou tzv. silnější, což znamená, že dokáží „vytáhnout“ napětí rychleji, ale zároveň spotřebovávají více energie.

Krom toho však existují i určité vyžadované elektrické vlastnosti sběrnice, které musí sběrnice splňovat pro úspěšnou komunikaci. Konkrétně se jedná o <u>maximální</u> elektrickou kapacitu 400 pF – což nemusí implikovat jen absenci kondenzátorů na sběrnici, ale i limit na parazitní kapacitu, která se projevuje u každého páru vodičů i u každé periferie a která omezuje, jak rychle dokáží pull-up rezistory vytáhnout linku na napětí logické 1. Avšak neméně důležitou roli hraje i elektrický odpor těchto rezistorů, který skrze časovou konstantu $\tau = RC$ také rychlost „vytažení“ linky přímo ovlivňuje (linku zde modelujeme jako RC obvod). Standard však již nespecifikuje doporučený elektrický odpor, tudíž ten je potřeba ve většině případů odhadnout. Například čip RP2040 obsahuje sice integrované pull-up rezistory, ale ty mají nominální elektrický odpor mezi 50 a 80 kΩ, což je pro I^2C naprosto nedostačující. A ačkoli existují určité matematické vzorce určující minimální a maximální doporučený odpor v závislosti na dalších parametrech, pro jednodušší projekty stačí experimentovat s různými odpory v řádech jednotek až nízkých desítek kΩ a zkoušet, co funguje, nebo koupit takové periferie, které již pull-up rezistory obsahují zabudované (jako námi použitý SSD1306).

Drát SDA pak přenáší data, zatímco drát SCL přenáší hodinový signál. Bity jsou přenášeny jako nejvýznamnější bit první (bit na pozici 128 v čísle dvojkové soustavy jako první – obráceně, než u UART). Zároveň je ještě potřeba mít společné referenční napětí 0 voltů – elektrickou zem – vůči kterému měříme napětí. Tento zemnící vodič (označovaný GND) nemusí být nutně připojen k fyzické Zemi jako v zásuvce, ale klidně i jen k zápornému pólu (mínusu) zdroje napětí.

Protože pouze řídící může iniciovat přenos dat, musí vysílat i výzvy k tomu, aby cíl mohl vyslat data. Jako první vyšle tzv. start podmínku („start condition“), kdy SDA přejde z logické 1 do logické 0, zatímco SCL zůstává 1. Pak začne přepínat SCL hodinový signál, aby začal vysílat bity. SDA se smí měnit jen, když je SCL 0, a když je SCL 1, bude přečtena hodnota SDA jako další bit. Po přenesení 8 bitů přenese druhá strana bit ACK, kterým říká, že data úspěšně přijala. Po přenesení jednoho či více bitů může řídící vytvořit buď tzv. stop podmínku („stop condition“), kdy při SCL na hodnotě 1 přejde SDA z 0 do 1, a nechat linku volnou, nebo opět hned start podmínku a nový přenos.

Každý cíl má svou adresu, což je 7bitové číslo, které je potřeba přenést na začátku každého přenosu, aby bylo vybráno, s kterým cílem na sběrnici řídící komunikuje. Jako osmý bit je přenesen tzv. R/W bit, který určuje směr komunikace – pokud je přenesena hodnota 0, řídící bude odesílat data cíli, a pokud je přenesena 1, řídící očekává, že cíl začne odesílat data. Adresa cíle je obvykle pevně daná v jeho hardwaru, obvykle změnitelná jen o několik málo jednotek, pokud cílem není jiný mikrořadič (který je přece jen programovatelný, a tudíž by měl být schopen svou adresu měnit).

Samotný I^2C protokol však také nespecifikuje nic víc než přenos bajtů a výše zmíněnou adresaci, a proto výrobce každé periferie implementuje svůj vlastní protokol. Pokud ho pro zařízení, se kterým pracujeme, neznáme, je většinou k nalezení v datasheetu periferie, avšak stejně většina programátorů ani tento protokol zařízení neimplementuje sama, ale použije „ovladač“ („driver“), což je softwarový program nebo knihovna zjednodušující programátorům práci s periferií tak, aby se nemuseli zabývat specifiky jejího návrhu ani elektrickému inženýrství obecně, ale mohli pracovat s periferií abstraktněji.

Protože je hodinový signál vysílaný řídícím, dokáže řídící určovat rychlost, kterou bude sběrnice komunikovat. Ne každé zařízení však podporuje všechny rychlosti, proto existují určité neoficiální módy, které standardizují maximální rychlosti sběrnice, které musí podporovat každé zařízení na ní (tedy minimální rychlosti, které zařízení musí být schopno podporovat). Nejpomalejší, ale nejpodporovanější je „Standard mode“ o maximální povolené rychlosti 100 kbit/s, o trochu rychlejší je „Fast mode“ o rychlosti až 400 kbit/s a nejrychlejší široce podporovaný mód je „Fast mode plus“ se zvýšenou rychlostí až na 1 Mbit/s. Standard sice podporuje i ještě rychlejší módy s rychlostmi až 5 Mbit/s, avšak v takovém případě už vznikají bariéry na úrovni samotných elektrických charakteristik sběrnice,

ne na úrovni schopností periferií samých, kvůli kterým mají tyto módy přísnější požadavky; a tudíž většina inženýrů použije pro tyto rychlosti jiný protokol než I^2C. Obecně platí, že čím pomalejší rychlost komunikace je, tím tolerantnější je sběrnice vůči svým elektrickým parametrům, a tudíž jeden z nejjednodušších způsobů, jak vyřešit problémy s komunikací po sběrnici, je jednoduše zkusit snížit rychlost.

4.3 SWD

Tento protokol se od předešlých dvou liší tím, že není zamýšlen pro obecný přenos dat, ale jako ladicí rozhraní. To znamená, že ho implementují zejména mikrořadiče a ladicí adaptéry, a umožňuje počítačům naprosto ovládat připojené mikrořadiče a ladit jejich programy. Toho dosahují například tím, že nastaví místo v programu, kde se zastaví a čekají, až jim počítač povolí pokračovat (tzv. breakpoint – „zarážka / bod přerušení“). Když je CPU zastaven na breakpointu, dokáže počítač skrze rozhraní SWD číst i měnit jakékoli místo v paměti, což používá programátor například k ověření, že určitá proměnná nabývá na určitém místě správné hodnoty. Krom toho lze přes něj také naprogramovat paměť flash a tím nahrát náš program do čipu.

Samotný protokol je jen součást celého ekosystému ladění procesorů architektury ARM, slouží k přístupu k přístupovému portu ladění („Debug Access Port“) architektury ARM, který zprostředkovává funkcionalitu ladění. Avšak jak samotný protokol SWD, tak celý ladicí subsystém jsou již poměrně složité věci, které nám abstrahuje použití ladicí software, potažmo vývojové prostředí.

Proto stačí u tohoto protokolu jen vědět, že nám zprostředkovává funkcionality ladění a programování čipu. Protokol využívá stejně jako I^2C jen dvou, potažmo tří vodičů: datovému vodiči SWDIO a hodinovému signálu SWCLK, potažmo zemnícímu vodiči GND.

5 OLED displej SSD1306

Jako displej pro náš projekt používáme displej SSD1306 od výrobce Solomon Systech. Jedná se o relativně malý, černobílý displej o rozlišení 128x64 pixelů založený na technologii OLED.

OLED („=Organic Light Emitting Diode“ – organická svítící dioda) je relativně moderní technologie displejů využívající organické sloučeniny mezi mřížkou elektrod pro výrobu světla podobným způsobem, jako běžné LED, ale díky jeho jednodušší miniaturizaci je vhodnější pro výrobu displejů.

Jenže protože ovládání 8 192 ($= 128 \times 64$) individuálních diod naráz je velmi složité pro univerzální mikrořadiče určené pro I/O operace a práci s daty, používá skoro každý displej na světě čip specializovaný přímo na ovládání displejů, se kterými mikrořadiče interagují skrze komunikační protokoly, ve kterých dokáží mikrořadiče komunikovat jednodušeji. Velké počítačové displeje používají

složité čipy, které dokáží aktualizovat např. 8 294 400 barevných pixelů frekvencí 120 Hz a komunikují přes pokročilé vysokorychlostní protokoly jako HDMI nebo DisplayPort, ale náš drobný displej je násobně méně náročný, tudíž mu stačí komunikovat přes mikrořadičové protokoly. Přesto však se doporučuje používat vyšší než standardní rychlosti přenosu, například výše zmíněný mód Fast Mode Plus u protokolu I^2C.

Čip SSD1306 obsahuje kromě samotné zobrazovací logiky také paměť RAM ukládající stav displeje, dokáže komunikovat přes protokoly I^2C, SPI i paralelní 8bitové sběrnici podobné těm používaným na Motorola 6800 nebo Intel 8080 (oba historické procesory se 70. let) skrze svůj proprietární protokol založený na přenosu příkazů a dat, a obsahuje napěťový převodník schopný vytvořit přibližně 7,5 voltů potřebných k rozsvícení panelu. I tento protokol je však relativně složitý, a tudíž obzvláště pro tento čip je vhodné až nutné používat nějaký ovladač (popsáno výše). Konkrétně v tomto případě obvykle ovladač zajistí, že programátor nemusí dokonale chápat celý postup inicializace a komunikace s displejem, ale pouze zavolá například funkci `display_init()`, která displej inicializuje automaticky pro něj na doporučené hodnoty, a potom bude volat jen funkce jako `display_show(array)`, které zobrazí dané pole bitů na displeji (obvykle hodnota `false` znamená, že pixel na daném umístění je zhasnut, a hodnota `true`, že je rozsvícen).

6 Programovací jazyk Rust

Rust je poměrně moderní programovací jazyk (první stabilní verze vydána v roce 2015) soustředěný na systémové programování, bezpečnost správy paměti a výkon. Avšak aby všech těchto cílů mohl dosáhnout, liší se poměrně výrazně od ostatních programovacích jazyků, a právě tyto odlišnosti a specifika budeme popisovat nejpodrobněji.

Rust je multiparadigmatický, podporuje různé způsoby programování, avšak protože je soustředěn právě na systémové programování, tíhne více k funkcionálnímu a procedurálnímu stylu. Neimplementuje všechny rysy objektově orientovaného programování, ale podle některých definic by ho tak šlo stejně kategorizovat, protože umožňuje nahlížet na své proměnné jako objekty, které mají určitá data a metody – funkce, které s danými daty pracují.

6.1 Správa paměti

Každý program nějak pracuje s pamětí (v tomto kontextu máme na mysli paměť RAM). A proto každý programovací jazyk musí nějak implementovat správu paměti, což lze provádět mnoha různými způsoby.

Asi ten nejprimitivnější, nejjednodušší způsob je dát programátorovi neomezený přístup k paměti a tím přesunout všechnu zodpovědnost na něj. To je sice přístup jednoduchý, ale velmi náchylný na chyby. Pokud pak musí paměť sdílet více programů naráz (včetně operačního systému), stávají se chyby téměř nevyhnutelnými. Jediný v současnosti používaný programovací jazyk, který ještě stále využívá tuto metodu, je jazyk symbolických adres, protože ten je překládán přímo do instrukcí strojového kódu a strojový kód pracuje pouze se „surovou“ pamětí.

Pro ochranu před úmyslnými útoky i chybami programátorů obsahuje téměř každý procesor hardwarovou jednotku MPU („=memory protection unit“ – jednotka pro ochranu paměti), která umožňuje programům specifikovat určité oblasti paměti a pravidla pro přístup k nim. Tyto pravidla jsou obvykle povolení/zakázání čtení a měnění dat v dané oblasti, spuštění daných dat jakožto strojové instrukce, a rozlišování privilegovaného a neprivilegovaného módu běhu programu. Pokud jsou tato pravidla porušena, MPU vyvolá chybu, chyba, která je buď ošetřena, nebo která vyvolá zamrznutí procesoru. Dokonce i náš mikrořadič RP2040 implementuje standardní MPU jednotku architektury ARMv6-M, ale protože v našem projektu na našem řadiči poběží jen náš kód, toto nikterak nevyužíváme a zůstáváme v privilegovaném módu se zakázanou MPU, což je výchozí stav po startu mikrořadiče.

Pokročilejší procesory pro osobní počítače pak používají ještě složitější jednotku MMU („=memory management unit“ – jednotka pro správu paměti), která v sobě kromě jednotky MPU obsahuje i logiku pro implementaci virtuální paměti a detailnější způsoby ochrany. Většina operačních systémů vyžaduje právě jednotku MMU, ale operační systémy reálného času („RTOS = real-time operating system“), které jsou schopné běžet i na mikrořadičích, vyžadují pouze MPU.

Další způsob správy paměti, který je v současné době často implementován na úrovni hardwaru, je tzv. zásobník volání („call stack“ – často jen zkráceno na „zásobník“ nebo „stack“). Zásobník obecně je datová struktura, kterou si může člověk představit jako sloupec papírů naskládaných jeden na druhém na stole – nový papír lze položit pouze na vrch sloupce, a odebrat papír lze také pouze z vrchu, tudíž poslední papír položený na sloupec bude první, který bude ze sloupce odebrán. Zásobník volání je však zvláštní tím, že se do něj ukládají například informace, které mají být předány mezi funkcemi, zejména adresa instrukce, na kterou se má procesor vrátit na skončení funkce. Pro práci se zásobníkem na hardwarové úrovni je používán speciální registr označovaný jako SP („=stack pointer“ – ukazatel zásobníku) a dvě speciální instrukce strojového kódu: `push`, která zkopíruje určitý registr na vrchol zásobníku a sníží hodnotu SP o 1 – položí hodnotu z registru na vrchol zásobníku, a `pop`, která naopak zkopíruje vrchol zásobníku do určitého registru a zvýší SP o 1 – odebere hodnotu ze zásobníku do registru. Na zásobník však lze tímto způsobem ukládat nejen adresy instrukcí, ale jakékoli proměnné.

Zásobník volání využívá i další, trochu pokročilejší metoda správy paměti, kterou využívají programovací jazyky jako C. Proměnné jsou automaticky ukládány na zásobník při jejich vytvoření a automaticky jsou z něj uvolněny při jejich zániku. Ačkoli tyto programovací jazyky stále poskytují přímý přístup k paměti, a tudíž jejich metody jsou označovány jako manuální správa paměti, hojné a automatické využívání zásobníku přece jen zjednodušuje programátorům život, není totiž třeba manuálně spravovat, kde v zásobníku se nachází která proměnná, a jak mají předat parametry funkcím.

Pokud však potřebují programátoři pracovat s pamětí dynamicky, například pro data, jejichž velikost se může měnit, musí z vyhrazeného rozsahu volné paměti (nazývaného „heap“ – halda, hromada) zarezervovat (=alokovat) prostor v paměti pro svá data manuálně pomocí funkcí jako `malloc()` a `free()`, které pracují s ukazateli. Ukazatele jsou proměnné, které obsahují adresy dat, na které tyto ukazatele ukazují. Samotné proměnné jsou ve výchozím stavu ukládané na zásobník, ale právě díky možnosti práce se surovou pamětí mohou být přesunuty do nově alokované paměti, na kterou ukazuje navrácený ukazatel, a pak říkáme, že jsou uloženy na heap.

Protože však jakákoli práce s ukazateli je stále náchylná na chyby, existuje ještě jednodušší způsob práce s pamětí, který implementují jazyky jako Python, Java a mnoho dalších, a který je nazýván automatickou správou paměti. V tomto případě se o jakoukoli práci s pamětí stará samotný programovací jazyk bez potřeby zásahů ze strany programátora, v případě potřeby sám alokuje paměť z heapu a sám ji uvolní zpět. Tento proces se děje zcela transparentně, a programátor vůbec nepracuje s ukazateli, nýbrž pouze s proměnnými. Programovací jazyk sám určuje způsob předávání parametrů funkcím.

Ačkoli používat takovýto programovací jazyk je výrazně jednodušší, má tento přístup mnoho nevýhod. První nepopiratelnou nevýhodou je výkon: většina jazyků implementuje automatickou správu paměti skrze tzv. sběr odpadu („garbage collection“). To znamená, že program paměť za běhu neuvolňuje, pouze alokuje, a program musí být tudíž periodicky přerušován, aby mohl běžet algoritmus zvaný „popelář“ („garbage collector“), který proskenuje celý rozsah paměti a za programátora sám uvolní paměť, kterou program již nebude/nemůže používat. Tato periodická přerušování nejen výrazně poškozují výkon jakéhokoli programu, ale zároveň ztěžují přesné načasování akcí, protože program může být kdykoli přerušen na nepředvídatelně dlouhou dobu, a v neposlední řadě mají výrazně vyšší nároky na celkovou dostupnou paměť, protože s ní pracují výrazně méně efektivně.

Z těchto všech důvodů tedy jazyk Rust nepoužívá metodu sběru odpadu, ale unikátní metodu „kontrolora půjčování“ („borrow checker“). Tento „kontrolor“ určuje <u>během kompilace</u>, ještě před spuštěním programu, očekávanou životnost každé proměnné, a dokáže zkontrolovat naplnění určitých pravidel popsaných níže. Pro správu paměti je však důležitý proto, že pokud je nějaká proměnná ukládaná na heap, tak při jejím vzniku je paměť automaticky alokována, a při jejím zániku, na konci její

životnosti je její paměť opět automaticky uvolněna, stejně jako by byla paměť uložena na zásobníku, ale zároveň ponechávajíce možnost změny velikosti proměnné. Díky schopnosti určit životnost každé proměnné je pak vždy možné určit, kdy životnost končí, a tedy nelze opomenout alokovanou paměť uvolnit. Proto říkáme, že Rust zajišťuje bezpečnost správy paměti.

Když programátor zapomene uvolnit alokovanou paměť, nazývá se to únik paměti, a je to jedna z nejčastějších chyb vyskytujících se u programů napsaných v jazyce s manuální správou paměti, protože u takových programů tato zodpovědnost spadá na programátora, který se chyby dopustí mnohem jednodušeji než zkušená skupina expertů tvořících samotný programovací jazyk.

Za zmínku však rovněž stojí, že v kontextu programování mikrořadičů ukládáme proměnné skoro výhradně na zásobník nebo staticky (kdy vyhradíme prostor pro proměnnou ještě před spuštěním programu mimo zásobník, a kdy je proměnná přístupná po celou dobu běhu a její umístění se nemění – proto staticky). Jakékoli ukládání na heap, i bezpečné, je totiž výrazně méně efektivní, a při programování mikrořadičů s výrazně omezenými zdroji na efektivitě záleží mnohem víc. Kromě toho, algoritmus, který při nové alokaci paměti prohledává heap a hledá volný prostor, který může alokovat, může trvat nepředvídatelně dlouho a tím výrazně poškodit načasování, na kterém u mikrořadičů také záleží mnohem víc. Výše zmíněné RTOS existují právě proto, že na rozdíl od běžných operačních systémů pracují se striktním a vysoce předvídatelným načasováním (proto „operační systémy reálného času“), což je vyžadováno v některých industriálních procesech, kde je potřeba, aby mikrořadič zareagoval v řádech jednotek až zlomků milisekund. RTOS a programy běžící pod nimi tedy obvykle nevyužívají heap vůbec, nebo výrazně jiným způsobem než běžné operační systémy pro osobní počítače.

6.2 Proměnné

6.2.1 Rozsahy a vlastnictví

V práci s proměnnými používá Rust model vlastnictví a půjčování, jehož pravidla vynucuje výše zmíněný borrow checker. Každý blok kódu je svým vlastním rozsahem („scope“), kontextem, ve kterým je spouštěn jiný kód. Rozsahy jsou vždy vnořené, tudíž každý rozsah může uvnitř obsahovat jiné rozsahy (nejčastěji se tak děje uvnitř funkce). Každá proměnná je deklarována (=poprvé uvedena v existenci, zejm. klíčovým slovem `let`) uvnitř určitého bloku kódu, uvnitř určitého rozsahu. Říkáme pak, že rozsah, ve kterým byla deklarována, je „vlastníkem“ proměnné. K proměnné může přistupovat rozsah, jež je vlastníkem proměnné, nebo rozsahy v něm vnořené. Pro vnější nebo zcela jiné rozsahy jsou proměnné definované ve vnořeném rozsahu nepřístupné, takže kód ve vnějším rozsahu buď selže, nebo bude přistupovat k zcela jiné proměnné stejného názvu.

Na obrázku 3 můžeme vidět jednoduchý příklad, kdy se kód ve vnějším rozsahu A pokouší přistupovat k proměnné uvnitř vnořeného rozsahu B.

```
fn main() {                 // ---------+-- A
    let x = 20;             //          |
                            //          |
    {                       // -+-- B   |
        let y = 5;          //  |       |
        println!("x: {x}");// |        |
    }                       // -+       |
                            //          |
    println!("y: {y}");     //          |
}                           // ---------+
```

Obrázek 3: Vzorek chybného kódu pro demonstraci rozsahů. Zdroj: vlastní.

Toto selže, protože nejen že jsou proměnné z B nepřístupné pro A, ale navíc, protože rozsah B skončil ještě dříve, než zavoláme makro `println!()`, tak proměnná Y už dávno došla konce své životnosti a automaticky zanikla; již neexistuje. V rozsahu B můžeme přistupovat k proměnné X z rozsahu A, s tím kompilátor problém nemá, ale v rozsahu A jiná proměnná s názvem Y neexistuje, takže kompilace selže s chybovou hláškou na obrázku 4. Dokonce nám kompilátor radí, že se nachází v jiném rozsahu.

```
error[E0425]: cannot find value `y` in this scope
 --> .\test_code.rs:9:19
  |
9 |     println!("y: {y}");    //          |
  |                   ^
  |
help: the binding `y` is available in a different scope in the same function
 --> .\test_code.rs:5:13
  |
5 |         let y = 5;         //  |       |
  |             ^
```

Obrázek 4: Chybová hláška po pokusu o kompilaci kódu z obrázku 3. Zdroj: vlastní.

Kód na obrázku 5 funguje, přistupujeme pouze k proměnným v našem vlastním rozsahu.

```
fn main() {                 // ---------+-- A
    let x = 20;             //          |
                            //          |
    {                       // -+-- B   |
        let y = 5;          //  |       |
        println!("y: {y}");// |        |
    }                       // -+       |
                            //          |
    println!("x: {x}");     //          |
}                           // ---------+
```

Obrázek 5: Vzorek správného kódu – upraveného kódu z obrázku 3. Zdroj: vlastní.

6.2.2 Stínování a měnitelnost

Pokud již existuje proměnná Y v rozsahu A a pokusíme se deklarovat novou proměnnou v rozsahu B se stejným názvem, není to chyba, ale kód se nebude chovat dle očekávání, protože bude přistupovat k té Y, která existuje v daném rozsahu. To však může být zamýšlené, a nazývá se to „stínování“ („shadowing“). Vzorek kódu je na obrázku 6. V tomto případě se kód zkompiluje, ale nebude se chovat podle očekávání, vypíše `y: 2048`.

```
fn main() {                 // ---------+-- A
    let x = 20;             //          |
    let y = 2048;           //          |
                            //          |
    {                       // -+-- B   |
        let y = 5;          //  |       |
    }                       // -+       |
                            //          |
    println!("x: {x}");     //          |
    println!("y: {y}");     //          |
}                           // ---------+
```

Obrázek 6: Vzorek kódu, který se nechová dle očekávání kvůli stínování proměnných. Zdroj: vlastní.

Pokud však odstraníme klíčové slovo `let`, jako na obrázku 7, nebudeme stínovat, ale jen měníme hodnotu Y.

```
fn main() {                 // ---------+-- A
    let x = 20;             //          |
    let y = 2048;           //          |
                            //          |
    {                       // -+-- B   |
        y = 5;              //  |       |
    }                       // -+       |
                            //          |
    println!("x: {x}");     //          |
    println!("y: {y}");     //          |
}                           // ---------+
```

Obrázek 7: Vzorek kódu, kde měníme hodnotu proměnné a který selže. Zdroj: vlastní.

Ale tento kód nám kompilátor opět odmítne, i když tentokrát s jinou chybovou hláškou na obrázku 8:

```
error[E0384]: cannot assign twice to immutable variable `y`
 ⟶ .\test_code.rs:6:9
  |
3 |     let y = 2048;         //          |
  |         - first assignment to `y`
...
6 |         y = 5;            //  |       |
  |         ^^^^^ cannot assign twice to immutable variable
  |
help: consider making this binding mutable
  |
3 |     let mut y = 2048;         //          |
  |         +++
```

Obrázek 8: Chybová hláška pro kód z obrázku 7. Zdroj: vlastní.

Jazyk Rust totiž všechny proměnné ve výchozím stavu deklaruje jako neměnné („immutable“), a po prvním přiřazení hodnoty (tzv. inicializaci) jejich hodnota již nelze měnit. Ale opět dostáváme nápovědu od kompilátoru, který říká, že přidáním klíčového slova `mut` za `let` lze deklarovat proměnnou jako měnitelnou. Proto se kód na obrázku 9 zkompiluje úspěšně a chová se podle našich očekávání – píše `y: 5`. Změny proměnných provedené ve vnitřním rozsahu se projeví ve vnějším rozsahu, protože nevzniká nová proměnná s novým vlastníkem, ale jen se mění ta stará. Vlastník proměnné se také nemění.

```
fn main() {                 // ---------+-- A
    let x = 20;             //          |
    let mut y = 2048;       //          |
                            //          |
    {                       // -+-- B   |
        y = 5;              //  |       |
    }                       // -+       |
                            //          |
    println!("x: {x}");     //          |
    println!("y: {y}");     //          |
}                           // ---------+
```

Obrázek 9: Upravený vzorek kódu s klíčovým slovem mut. Zdroj: vlastní.

6.2.3 Typy proměnných a jejich inference

V předchozích vzorcích si můžeme povšimnout, že nikde nedeklarujeme typ proměnné. V jazyce Rust můžeme explicitně říct, jaký typ má proměnná mít, jako na obrázku 10, ale v jednodušších případech to není potřeba. Jazyk Rust totiž disponuje schopností automatické inference (odvození) typu proměnné. U proměnných D, E a F si Rust typ proměnné sám doplní.

```
fn main() {
    let a: usize = 98_168_416_450;
    let b: i64 = -2025;
    let c: f32 = 3.14;
    let d = true;
    let e = 20;
    let f = 2.7;
}
```

Obrázek 10: Příklady explicitního i inferovaného určení typu proměnné. Zdroj: vlastní.

Malá odbočka: Rust nám obzvláště pro číselné datové typy poskytuje na výběr mnoho možností. Pro celá čísla jich je 12: `i8`, `i16`, `i32`, `i64`, `i128`, `isize`, `u8`, `u16`, `u32`, `u64`, `u128` a `usize`. První písmeno je I nebo U podle toho, zda dokáže obsáhnout celá čísla (I) nebo jen přirozená čísla a nulu (U). Čísla za tímto písmenem označují délku čísla v paměti v bitech, a tím nepřímo i jeho minimální a maximální hodnotu. Typy `isize` a `usize` označují celá čísla, jejichž délka v paměti odpovídá délce ukazatele na dané platformě – na 32bitových procesorech bude 32 bitů velký, na 64bitových 64 bitů, apod. Pro čísla s plovoucí desetinnou čárkou poskytuje pouze dva typy: `f32` a `f64`, kdy číslo opět označuje délku v bitech a tím nepřímo rozlišení a maximální hodnotu. Pokud inicializujeme proměnnou na konstantní číslo dané ve zdrojovém kódu („literal“), ale nedodáme typ, tak Rust automaticky použije `i32` u celých čísel a `f64` u desetinných čísel.

Pokud se námi dodaný literal nevejde do explicitně nebo i automaticky určeného typu proměnné, Rust nás o tom varuje chybou, jako v obrázku 11, v případě inference typu však nezvýší automaticky velikost typu.

```
error: literal out of range for `u8`
 ⟶ .\test_code.rs:2:17
  |
2 |     let a: u8 = 1000;
  |                 ^^^^
  |
  = note: the literal `1000` does not fit into the type `u8` whose range is `0..=255`
  = note: `#[deny(overflowing_literals)]` on by default
```

Obrázek 11: Rust nám zakazuje použít literal větší než maximální hodnota typu proměnné. Zdroj: vlastní.

Typ literalu můžeme určit nejen explicitním typem proměnné, ale i tak, že název typu napíšeme na konec literalu, jako na obrázku 12. To je vhodné například když dáváme literal jako parametr funkci a z nějakého důvodu si Rust nedokáže inferovat typ literalu sám. Z důvodu čitelnosti je slušností oddělit hodnotu a typ literalu podtržítkem, které je Rustem ignorováno, stejně jako při užití podtržítka jako oddělovač řádů na obrázku 10.

```rust
fn main() {
    let a = -2147483649_i64;
}
```

Obrázek 12: Vzorek kódu, který automaticky určí vhodný typ proměnné. Zdroj: vlastní.

V budoucnosti (níže) uvidíme, že inference typů nám velmi zjednoduší život a zvýší čitelnost našich programů, protože funguje nejen při inicializaci proměnné na nějaký literal, ale i když inicializujeme proměnnou na hodnotu, kterou nám navrací nějaká funkce. Ale uvidíme i, že v některých případech může inference selhat a my tudíž musíme nastavit typ proměnné manuálně.

Dalšími primitivními typy proměnných jsou například `char`, `bool`, `str`, nebo pole.

Typ `char` reprezentující jakýkoli jeden Unicode znak (i znak jako „ř“, „你“ nebo „💜“[1]) a který je tím pádem 32 bitů velký. Jeho literal se značí apostrofy „`'`“.

Typ `bool`, dokáže nabývat hodnoty pouze `true` nebo `false` a používá se pro hodnoty, které potřebují nabývat jen ano / ne.

[1] Ne však např. znak „🦮“, který se skládá z více Unicode znaků – znak pro psa, pro reflexní vestu a znak „zero width joiner“, který obě emoji spojí do jednoho grafému.

Typ `str` reprezentuje textový řetězec, ale protože se jedná jen speciální typ pole, nemá pevně danou velikost, tudíž je používán většinou v neměnné referenci (o tom více níže). Jeho literaly se značí uvozovkami „`"`“ a mají statickou životnost – jejich reference jsou platné po celou dobu běhu programu.

Pole („array“) dokáží obsahovat jakékoli jiné typy (včetně dalších polí) a reprezentují uspořádanou „sadu“ hodnot v určitém pořadí. Při jejich vytváření používáme hranaté závorky „`[]`“, skrze které také přistupujeme k jejich položkám, a musíme vždy udat délku vytvářeného pole, pokud nenecháme typ být inferován automaticky, protože samotné pole nemá definovanou velikost v paměti. Reference na pole však toto omezení nemá, nemusíme udávat (ani znát) její velikost, a často ji nazýváme „slice“ – „plátek“, protože může nahlížet i jen do části nějakého většího pole. Příbuzným typem jsou uspořádané n-tice („tuple“), které mohou obsahovat hodnoty různých typů, které se značí kulatými závorkami „`()`“ a k jejichž položkám přistupujeme pomocí operátoru tečky „`.`“. Pořadí položek počítáme (tzv. indexujeme) od nuly.

Pole lze využít pro tvorbu dvou- i vícerozměrných mřížek jednoduše tak, že vytvoříme pole, které obsahuje další pole.

```
fn main() {
    let a: char = 'a';
    let b: char = '💜';
    let c: bool = true;
    let d: &str = "Hello";
    let e: [u8; 5] = [1, 2, 3, 4, 5];
    let mut f: [bool; 3] = [true, true, true];
    f[1] = false;
    let g: (u16, i64) = (20, -50_000);
    let mut h: (f64, &str, bool) = (
        3.1415926535,
        "Pi",
        true
    );
    h.0 = 3.0;

    let mut i: [[u8; 5]; 5] = [
        [00, 00, 11, 00, 00],
        [00, 11, 11, 11, 00],
        [11, 00, 11, 00, 11],
        [00, 00, 11, 00, 00],
        [00, 00, 11, 00, 00],
    ];
    i[1][3] = 22;
}
```

Obrázek 13: Příklad použití dalších primitivních typů. Zdroj: vlastní.

6.2.4 Neinicializované proměnné

Obvykle, když deklarujeme proměnnou, tak ji rovnou zároveň inicializujeme – přiřadíme jí její první hodnotu. Příklad je na obrázku 14. Pokud proměnnou neinicializujeme, můžeme jí později jednou přiřadit hodnotu bez použití klíčového slova `mut`, protože daná proměnná ještě žádnou hodnotu nemá, a tudíž ji neměníme. Pro přiřazení několikrát však stále `mut` potřebujeme. Neinicializovaná proměnná stále může svůj typ inferovat, protože se Rust dokáže podívat napřed do kódu a určit, jakého typu proměnná nabude, avšak odmítne kód, kde nelze typ jednoznačně určit (například kvůli nepředvídatelným podmínkám – pokud by se podmínka naplnila, byla by proměnná inicializována na jeden typ, pokud ne, byla by inicializována na úplně jiný typ). Neinicializované hodnoty jsou užitečné například v situacích, kdy chceme pracovat s proměnnou uvnitř nějakého bloku, ale chceme, aby měla životnost vně něj, takže ji deklarujeme bez inicializace vně.

```
fn main() {
    let a;
    let b: bool;
    let mut c: u8;

    {
        a = 20;
        b = false;
        c = 15;
    };
    c = c * 2;

    println!("a: {a}");
    println!("b: {b}");
    println!("c: {c}");
}
```

Obrázek 14: Vzorek kódu s neinicializovanými proměnnými. Zdroj: vlastní.

Avšak před inicializací proměnné nelze s proměnnou skoro nijak pracovat, obzvláště nelze číst její hodnotu, protože ještě obsahuje neinicializovaná, „odpadková“ data, která byla v paměti RAM ještě předtím, než jsme proměnnou deklarovali.

```
fn main() {
    let a;
    println!("a: {a}");

    a = 20;
    println!("a: {a}");
}
```

Obrázek 15: Vzorek kódu je špatný, protože se pokouší číst hodnotu proměnné před inicializací. Zdroj: vlastní.

```
 --> .\test_code.rs:3:19
  |
2 |     let a;
  |         - binding declared here but left uninitialized
3 |     println!("a: {a}");
  |                   ^ `a` used here but it is possibly-uninitialized
  |
  = note: this error originates in the macro `$crate::format_args_nl`
```

Obrázek 16: Chybová hláška po pokusu o kompilaci kódu z obrázku 15. Zdroj: vlastní.

Poznámka pod chybovou hláškou nám napovídá, že použité makro `println!()` je jen makro, které je během kompilace rozbaleno do kódu, a ne běžná funkce, ale to nás v tomto případě příliš netrápí. Je to proto, aby mohlo `println!()` přijímat jakýkoli počet parametrů jakýchkoli typů. Důvod, proč běžné funkce nemohou, ale makra ano, je složitý a mimo rozsah tohoto dokumentu. Často je používáme pro formátování textu s i bez proměnných.

6.2.5 Reference

Pokud předáme funkci nějakou proměnnou jako parametr, jsou dva způsoby, jak to provést. První způsob je jednoduchý: prostě předáme volané funkci celou proměnnou i vlastnictví nad ní. Novým vlastníkem proměnné se stává rozsah volané funkce, a v rozsahu volajícím funkci přestane proměnná existovat. To nazýváme „předávání přes hodnotu“ („passing by value“), a o funkcích, kterým parametry předáváme tímto způsobem, říkáme, že „konzumují“ danou proměnnou. Problém nastává ve chvíli, kdy toto není zamýšlený účinek, kdy chceme, aby funkce pouze nějak pracovala s proměnnou a abychom s ní poté mohli dále pracovat ve volajícím rozsahu, například na obrázku 17. Proměnná X je deklarována v rozsahu A, ale jejím předáním funkci `len_by_value()` se jejím novým vlastníkem stane rozsah B, a po skončení funkce proměnná (s novým názvem `param`) zaniká, takže rozsah A už s ní nemůže dále pracovat.

```
fn len_by_value(param: String) -> usize { // ---+-- B
    param.len() //                            |
} // ----------------------------------------+

fn main() { // --------------------------+-- A
    let x = String::from("123"); //      |
//                                       |
    let lenght = len_by_value(x); //      |
    println!("x' lenght: {}", lenght); //|
    println!("x: {}", x); //             |
} // -------------------------------------+
```

Obrázek 17: Vzorek kódu ukazující předávání přes hodnotu. Zdroj: vlastní.

```
error[E0382]: borrow of moved value: `x`
  --> .\test_code.rs:10:23
   |
 6 |     let x = String::from("123"); //      |
   |         - move occurs because `x` has type `String`, which does not implement the `Copy` trait
 7 | //                                       |
 8 |     let lenght = len_by_value(x); //     |
   |                               - value moved here
 9 |     println!("x' lenght: {}", lenght); //|
10 |     println!("x: {}", x); //             |
   |                       ^ value borrowed here after move
   |
note: consider changing this parameter type in function `len_by_value`
to borrow instead if owning the value isn't necessary
  --> .\test_code.rs:1:24
   |
 1 | fn len_by_value(param: String) -> usize { // ---+-- B
   |    ------------        ^^^^^^ this parameter takes ownership of the value
   |    |
   |    in this function
   = note: this error originates in the macro `$crate::format_args_nl`
   which comes from the expansion of the macro `println`
help: consider cloning the value if the performance cost is acceptable
   |
 8 |     let lenght = len_by_value(x.clone()); //     |
   |                                ++++++++
```

Obrázek 18: Chybová hláška nás varuje, že předáním přes hodnotu ztrácíme vlastnictví nad proměnnou. Zdroj: vlastní.

Ale na obrázku 18 nám již kompilátor napovídá, jak to vyřešit. První, ten jednodušší, ale omezenější způsob, je zavolat napovídanou metodu `.clone()`. Tuto metodu implementuje každý typ, který implementuje trait `Clone`. Traity budou dopodrobna rozvedeny níže, ale zde stačí vědět, že trait `Clone` poskytuje možnost udělat klon proměnné o stejné sémantické hodnotě (zde stejný textový obsah). To je však časově i paměťově náročná operace, protože typ `String` je textový řetězec ukládaný na heap, a tudíž jeho klonování vyžaduje další alokaci paměti, proto kompilátor ostatně píše „…if the performance cost is acceptable“ – „…pokud jsou nároky na výkon přijatelné“.

Pokud typ implementuje trait `Copy`, znamená to, že ani klonování není třeba, protože kopii lze udělat jednoduchým zkopírováním bit od bitu, a tím pádem je kopie vytvořena automaticky při každém předávání přes hodnotu. Trait `Copy` implementují například všechny číselné typy nebo booleany, ale ne např. typy ukládané na heap (jako třeba `String`).

Druhý způsob, jak předat funkci nějaký parametr, (který nám kompilátor také napovídá,) je předat funkci pouze odkaz na proměnnou, ne přímo proměnnou samotnou. Tomu se říká „předávání přes referenci“ („passing by reference“). Reference je tedy proměnná reprezentující odkaz na jinou proměnnou, a obsahuje ukazatel na místo, kde se daná proměnná nachází. V analogii ke konceptu vlastnictví, Rust toto nazývá „půjčování“ („borrowing“), protože volaná funkce se nestává novým vlastníkem proměnné, jen si proměnnou „půjčuje“, aby s ní mohla pracovat, ale vlastník zůstává stejný. Ve většině jednodušších případů můžeme referenci používat stejně, jako bychom používali samotnou

proměnnou. Reference se značí znakem „&“. Na obrázku 19 můžeme vidět způsob, jak správně předávat přes referenci.

```
fn len_by_ref(param: &String) -> usize { // --+-- B
    param.len() //                          |
} // ----------------------------------------+

fn mod_by_ref(param: &mut String) { //---+-- C
    param.push('4'); //                  |
} // -----------------------------------+

fn main() { //---------------------------+-- A
    let mut x = String::from("123"); //  |
//                                       |
    mod_by_ref(&mut x); //               |
//                                       |
    let lenght = len_by_ref(&x); //      |
    println!("x' lenght: {}", lenght); //|
    println!("x: {}", x); //             |
} // -----------------------------------+
```

Obrázek 19: Vzor kódu s předáváním přes referenci. Zdroj: vlastní.

Protože Rust ve výchozím stavu deklaruje proměnné jako neměnné, stejně tak deklaruje i reference: abychom mohli používat měnitelnou referenci, musíme opět použít klíčové slovo `mut`.

Jak již ostatně napovídá i jeho název, zde je opět uplatněn borrow checker, jehož zodpovědnost je právě hlavně kontrola *půjčování*. Kontroluje jej právě skrze výše popsané životnosti. Na rozdíl od jiných programovacích jazyků jako C, v jazyce Rust totiž musí reference vždy ukazovat na platná data, ne pouze na nějaké místo v paměti[2]. Tím, že borrow checker sleduje životnost každé proměnné, dokáže jednoznačně ještě během kompilace určit, zda má reference na proměnnou delší životnost než proměnná samotná. Pokud ano, kompilátor kód odmítne. Na obrázku 20 můžeme vidět příklad chybného kódu.

[2] Rust obsahuje i tzv. surové ukazatele („raw pointers“), které mohou skutečně ukazovat na jakékoli místo v paměti a mohou být i nulové. Lze také manuálně specifikovat, na jakou adresu mají ukazovat. Rust však u těchto ukazatelů nedokáže zaručit, že budou ukazovat na platná data, takže je označuje jako „nebezpečné“ a přenáší zodpovědnost za jejich správnost na programátora. Pokud chceme tedy získat hodnotu, na kterou daný ukazatel ukazuje, musíme tak učinit ve zvláštním bloku kódu označeným klíčovým slovem `unsafe`. Tím říkáme kompilátoru „vím, co dělám“ – a je důležité, abychom při tom nelhali.
Surové ukazatele jsou sice opravdu nebezpečné, protože umožňují programátorovi se dopouštět chyb, kterých by se jinak nedopustil, ale přesto jsou velmi důležité například pro nízkoúrovňové programování, kde víme, že určitá periferie leží na určité adrese a potřebujeme na tuto adresu získat nějaký ukazatel, i když Rust na této adrese ještě nevidí žádný objekt, nebo při volání externích funkcí napsaných v jiném programovacím jazyce (tzv. FFI – „foreign function interface“).

```
fn main() {                 // ---------+-- A
    let ref_x;              //          |
                            //          |
    {                       // -+-- B   |
        let x = 5;          //  |       |
        ref_x = &x;         //  |       |
    }                       // -+       |
                            //          |
    println!("r: {ref_x}"); //          |
}                           // ---------+
```

Obrázek 20: Vzorek kódu je chybný, protože reference má delší životnost než proměnná samotná. Zdroj: vlastní.

```
error[E0597]: `x` does not live long enough
 --> .\test_code.rs:6:17
  |
5 |         let x = 5;        //  |       |
  |             - binding `x` declared here
6 |         ref_x = &x;       //  |       |
  |                 ^^ borrowed value does not live long enough
7 |     }                     // -+       |
  |     - `x` dropped here while still borrowed
8 |                           //          |
9 |     println!("r: {ref_x}"); //        |
  |                   ----- borrow later used here
```

Obrázek 21: Chybová hláška pro kód z obrázku 20. Zdroj: vlastní.

Jak nám napovídá chybová hláška na obrázku 21, proměnná X je deklarována v rozsahu B, ale reference na ni je deklarována už v rozsahu A, takže při pokusu o výpis hodnoty pomocí reference již samotná proměnná X zanikla, protože skončil rozsah B. Reference z rozsahu A sice stále existuje, ale nyní ukazuje na neexistující data, a pokus o jejich přečtení by byl vážnou chybou (v případě, že by byla X uložená na heapu, nazývalo by se to „použití po uvolnění“ („use after free“), protože na konci životnosti X by byla paměť rezervovaná na heapu automaticky uvolněna[3]). Toto je přesně jeden z mechanismů, skrze které Rust zajišťuje paměťovou bezpečnost: na rozdíl od ukazatelů v jazycích jako C, zde reference vždy ukazuje na platná data.

Pokud ukládáme nějakou referenci dovnitř struktury, je potřeba deklarovat životnost reference, aby kompilátor věděl, do kdy platí. Nejobvyklejší způsob, jak na to, je jednoduše zdědit životnost dané struktury, tedy dokud má daná struktura životnost, má životnost i reference v ní obsažená. Tyto životnosti jsou obvykle specifikovány skrze generické parametry. Na obrázku 22 se pokoušíme deklarovat strukturu obsahující referenci, ale nedeklarovali jsme její životnost, proto nám na obrázku 23 vyhodí kompilátor chybovou hlášku. U funkcí, které přijímají nebo navracejí reference, je taky

[3] Tento koncept – abstraktní zdroje jako alokovaná paměť jsou navázané na životnost objektu, na začátku životnosti jsou zdroje automaticky alokovány a na konci jsou automaticky uvolněny – se (nevýstižně) nazývá RAII („resource acquisition is initialisation“).

potřeba pracovat s životnostmi, ale u nich si dokáže kompilátor za určitých podmínek tyto deklarace inferovat. To nazývá dokumentace elizí životnosti („lifetime elision“).

```
struct Structure {
    ptr: &u32
}
```

Obrázek 22: Tento vzorek kódu je neplatný, protože nedeklaruje životnost reference uvnitř struktury. Zdroj: vlastní.

```
error[E0106]: missing lifetime specifier
 --> .\test_code.rs:2:10
  |
2 |     ptr: &u32
  |          ^ expected named lifetime parameter
  |
help: consider introducing a named lifetime parameter
  |
1 ~ struct Structure<'a> {
2 ~     ptr: &'a u32
  |
```

Obrázek 23: Kompilátor nám v chybové hlášce radí, jak deklarovat životnost. Zdroj: vlastní.

Další způsob, jakým borrow checker zajišťuje bezpečnost správy paměti, je pravidlo, že v určitém okamžiku (z pohledu životností) může existovat buď pouze jedna měnitelná reference, nebo nekonečno neměnných referencí[4]. Naráz nemůže zároveň existovat měnitelná i neměnná reference, ani více měnitelných referencí. Toto pravidlo zabraňuje určitým typům souběhů (konkrétně tzv. „data race“), což je chyba vyskytující se v paralelních systémech, kdy se kód v jednom místě snaží změnit hodnotu proměnné, zatímco na jiném místě je zrovna v průběhu čtení její hodnoty. Protože Rust je od svých počátků navrhnut s ohledem na paralelizaci (proces, při kterém dokáže počítač dělat více věcí naráz), toto pravidlo bezvýjimečně vynucuje, i když reference nevyužíváme skutečně naráz, protože nepíšeme paralelní kód. Stačí, když pouze existují naráz. Nástroje pro bezpečné sdílení dat, obzvláště pak měnitelný přístup ze dvou míst naráz, poskytuje standardní knihovna.

Na obrázku 24 je příklad kódu, který selže v kompilaci, protože zároveň existuje měnitelná i neměnná reference. Zatímco se pokoušíme předat měnitelnou referenci funkci `inc()`, struktura `PtrCapsule`, jejíž životnost ještě neskončila a kterou budeme ještě používat, obsahuje neměnnou referenci. Struktura tak způsobuje, že by musela zároveň existovat neměnná reference (která stále žije, uložená uvnitř struktury) a měnitelná reference (kterou vytváříme, abychom ji předali funkci, a která po

[4] Ani toto omezení neplatí pro surové ukazatele popsané výše.

skončení funkce automaticky zanikne). Kdybychom již strukturu znovu nepoužívali, byl by konec její životnosti již v okamžiku posledního použití struktury, a problém by nenastal. Na obrázku 25 nám kompilátor naši chybu vysvětluje.

```rust
struct PtrCapsule<'a>{
    ptr: &'a u32
}

impl<'a> PtrCapsule<'a>{
    fn print(&self){
        println!("Value: {}", self.ptr);
    }
}

fn inc(num: &mut u32){
    *num += 1
}

fn main() {
    let mut x = 20;
    let capsule = PtrCapsule{ ptr: &x };

    println!("x: {}", x);

    inc(&mut x);
    capsule.print();
}
```

Obrázek 24: Vzorek kódu, kde se snažíme zároveň mít měnitelnou i neměnnou referenci. Zdroj: vlastní.

```
error[E0502]: cannot borrow `x` as mutable because it is also borrowed as immutable
  ⟶ .\test_code.rs:21:9
   |
17 |     let capsule = PtrCapsule{ ptr: &x };
   |                                     -- immutable borrow occurs here
...
21 |     inc(&mut x);
   |         ^^^^^^ mutable borrow occurs here
22 |     capsule.print();
   |     ------- immutable borrow later used here

error: aborting due to 1 previous error
```

Obrázek 25: Chybová hláška po pokusu o kompilaci kódu z obrázku 24. Zdroj: vlastní.

```rust
use std::cell::RefCell;

struct RefCapsule<'a>{
    cell: &'a RefCell<u32>
}

impl<'a> RefCapsule<'a>{
    fn print(&self){
        println!("Value: {}", self.cell.borrow());
    }
}

fn inc(num: &RefCell<u32>){
    *num.borrow_mut() += 1
}

fn main() {
    let x = RefCell::new(20);
    let capsule = RefCapsule{ cell: &x };

    println!("x: {}", x.borrow());

    inc(&x);
    capsule.print();
}
```

Obrázek 26: Příklad kódu, který využívá RefCell ze standardní knihovny pro vyřešení problému z obrázků 24 a 25. Zdroj: vlastní.

Pro zajímavost, na obrázku 26 je příklad použití objektu `RefCell` ze standardní knihovny k vyřešení tohoto problému.

6.3 Struktury a enumy

Ve většině programovacích stylů si však nevystačíme pouze s primitivními typy, a proto používáme kompozitní/složené typy, které se skládají z více jiných typů. Nejjednoduššími kompozitními typy jsou výše popsaná pole a uspořádané n-tice, ale v reálném programování se častěji používají struktury, a unikátně pro Rust, i enumy.

6.3.1 Struktury

Struktury, rovněž nazývané záznamy nebo uživatelsky definované typy, obsahují obvykle alespoň jednu nebo více položek (také nazývaných prvky nebo členy), které jsou rozlišovány jmény, a ne pořadím, a tuto „skupinu hodnot“ dokáží reprezentovat a přenášet jako jednu hodnotu. Protože struktury dokáží obsahovat data a mohou být pro ně definovány metody (o tom více níže), je možné je nazývat objekty. Vlastníkem dat uvnitř struktury je přímo struktura sama, a skrze to tedy vlastník struktury.

Struktury však nemusí být jen efektivním způsobem přenášení skupiny dat, ale často reprezentují pro náš kód něco abstraktnějšího, jako třeba síťový socket nebo periferii na mikrořadiči. Strukturu můžeme definovat a vytvořit do proměnné například následovně:

```rust
struct User {
    active: bool,
    username: String,
    email: String,
    sign_in_count: u64,
}
```

Obrázek 27: Příklad definice struktury a jejich položek. Zdroj: (1), kapitola 5.1

```rust
let user1 = User {
    active: true,
    username: String::from("someusername123"),
    email: String::from("someone@example.com"),
    sign_in_count: 1,
};
```

Obrázek 28: Příklad vytvoření struktury z obrázku 27, vyplnění jejích položek hodnotami, a uložení nově vzniklé struktury do proměnné. Zdroj: (1), kapitola 5.1

Na příkladu z obrázku 27 definujeme strukturu nazvanou User, která obsahuje položky `active` (typ `bool`), `username`, `email` (typ `String` – textový řetězec) a `sign_in_count` (typ `u64`). Během jejího tvoření a ukládání do proměnné musíme pro každou položku zadat nějakou hodnotu; pro volitelné hodnoty, které mohou nebo nemusí být přítomny, je vhodné použít enum `Option` popsaný níže.

Pro deklaraci metod – funkcí, které pracují s daty uvnitř struktury nebo obecně datového typu – používáme klíčové slovo `impl` a název typu (zde název struktury). Uvnitř tohoto bloku pak deklarujeme funkce, které jako první parametr většinou pojímají buď vlastnictví, nebo nějakou referenci na objekt, pro který jsou deklarované – značené klíčovým slovem `self`, `&self` nebo `&mut self`. Je zcela platná syntaxe, aby měla struktura pro sebe deklarovaných více bloků `impl`. Na obrázku 29 vidíme příklad použití tohoto bloku, stejně jako zavolání metody.

```rust
struct Rectangle {
    width: u32,
    height: u32,
}

impl Rectangle {
    fn area(&self) -> u32 {
        self.width * self.height
    }
}

fn main() {
    let rect1 = Rectangle {
        width: 30,
        height: 50,
    };

    println!(
        "The area of the rectangle is {} square pixels.",
        rect1.area()
    );
}
```

Obrázek 29: Příklad deklarace metody. Zdroj: (1), kapitola 5.3, upraveno

Pro zavolání metody na již existujících objektech používáme operátoru tečky „`.`“, který automaticky vyplní první parametr `self` nebo vhodnou referenci. Pokud voláme metodu, která `self` nepojímá, a tudíž ji nevoláme na již existující objekt (tzv. asociovaná funkce), použijeme operátor dvou dvojteček „`::`“, před který napíšeme jméno struktury, jako na obrázku 28. Asociované funkce jsou obvykle použité jako konstruktory, které vytváří instanci dané struktury a navrací ji jako svou hodnotu. Pro typ objektu, pro který je funkce definována, lze použít alias `Self`. Parametr `&self` je pak jen zkratkovitý zápis `self: &Self`. Příklad je na obrázku 30.

```rust
impl Rectangle {
    fn square(size: u32) -> Self {
        Self {
            width: size,
            height: size,
        }
    }
}
```

Obrázek 30: Příklad metody konstruktoru, která nepojímá parametr self, ale navrací instanci Self. Zdroj: (1), kapitola 5.3

6.3.2 Výčtové typy – enumy

Výčtové typy, označované rovněž jako „enum“ (od anglického „enumerated type – očíslovaný typ), jsou typy proměnných, které mohou nabývat právě jedné hodnoty z přesně definované, omezené množiny hodnot. Obvykle je každá hodnota nějak pojmenována, aby byla pro čtení transparentnější než jen číslice, kterými lze enumy v mnoha jazycích nahradit. Například můžeme definovat enum `Smer`, která bude mít varianty `{Vpred, Vzad, Vlevo, Vpravo}`, pak proměnná typu `Smer` může mít jako hodnotu právě jednu z těchto možností, třeba `Vpred` nebo `Vlevo`, ale nemůže nabývat např. hodnoty `Nahoru`, `3.1415926535` nebo `"Žlutozelená"`. V syntaxi jazyka Rust se to značí jako na obrázku 31:

```
enum Smer {
    Vpred,
    Vzad,
    Vlevo,
    Vpravo
}

fn main() {
    let forwards = Smer::Vpred;
    let left = Smer::Vlevo;
}
```

Obrázek 31: Příklad deklarace jednoduchého enumu. Zdroj: vlastní.

Ale enumy v jazyce Rust toho dokáží ještě více. Pokud chceme kromě možnosti přenést i nějaká další data, například použít tuto možnost jako značku pro druh určitých dat, mohlo by nás napadnout vyřešit problém například jako na obrázku 32:

```
enum IpAddrKind {
    V4,
    V6,
}

struct IpAddr {
    kind: IpAddrKind,
    address: String,
}

let home = IpAddr {
    kind: IpAddrKind::V4,
    address: String::from("127.0.0.1"),
};

let loopback = IpAddr {
    kind: IpAddrKind::V6,
    address: String::from("::1"),
};
```

Obrázek 32: Složitější způsob, jak přenést zároveň možnost enumu i nějaká data. Zdroj: (1), kapitola 6.1

Avšak jazyk Rust umožňuje přenášet další data přímo uvnitř možnosti enumu. Způsob, jakým je toto dosaženo, je již mimo účely tohoto dokumentu, ale jednoduše řečeno je možné deklarovat další datové typy, které musí daná možnost obsahovat a na základě varianty, které enum nabývá, se procesor rozhodne, jako jaké datové typy má připojená data interpretovat. Ne každá možnost musí nutně nějakou obsahovat, možnosti mohou obsahovat i různé typy dat nebo počet polí. Příklad deklarace enumu s daty je na obrázku 33, například v textové hře bychom viděli:

```
enum Akce {
    NedelejNic,
    OtevriInventar,
    Rekni(String),
    JdiNaSouradnice(i32, i32),
    PodivejSeNa { predmet: String, smer: Smer }
}

fn main() {
    let akce1 = Akce::Rekni(String::from("Ahoj!"));
    let akce2 = Akce::JdiNaSouradnice(20, 45);
    let akce3 = Akce::PodivejSeNa {
        predmet: String::from("Meč sv. Jiří"),
        smer: Smer::Vlevo
    };
    let akce4 = Akce::OtevriInventar;
}
```

Obrázek 33: Příklad enumu s daty ve svých možnostech. Zdroj: vlastní.

6.3.3 Použití enumů pro zpracovávání chyb

Téměř každý programovací jazyk musí nějak zpracovávat chyby, které mohou nastat za jeho běhu. Dělení nulou, nenalezený soubor na disku nebo podobné chyby mohou nastat i ve správně napsaném programu, a proto je potřeba s chybami pracovat. Nejjednodušším mechanismem je prostě okamžitě zastavit celý program v momentu, kdy nějaká chyba nastane, ale to není ve většině případů zamýšlený účinek, protože často chceme s chybou nějak pracovat a pokračovat v běhu. Některé jazyky jako C nemají žádný standardní způsob zpracovávání chyb a přístupy jsou fragmentované – někdy je navrácen nulový ukazatel (ukazatel ukazující na adresu 0, která je tradičně považovaná, že na ní neleží žádná data), někdy jsou použité jiné metody. Mnoho jazyků, např. C++, Java nebo Python, používají tzv. výjimky („exceptions“), které přeskočí na zcela jiné místo v programu, kde je dále chyba zpracována. Avšak jazyk Rust používá zcela jiný přístup: kromě tradičního přerušení celého programu, chyby jsou pouze dalšími hodnotami. Používá pro to právě svoje unikátní enumy s daty: `Option` a `Result`.

`Option` je ten jednodušší: komunikuje *volitelnou* hodnotu – buď hodnota existuje (varianta `Some` – „nějaký“), a pak nám tento enum předává daná data, nebo hodnota prostě neexistuje (varianta `None` – „žádný“). Toto je přímý ekvivalent nulového ukazatele, který je nulový právě tehdy, když komunikuje

absenci hodnoty. Jenže nulový ukazatel je nebezpečný, protože když zapomeneme zkontrolovat, zda obdržený ukazatel není nulový (například kvůli chybnému předpokladu o funkci, která nám daný ukazatel poskytla), a pokusíme se ho použít jako platná data, vede to k tzv. nedefinovanému chování, kdy se může stát *naprosto* cokoliv, protože kompilátor předpokládá, že nedefinované chování se nikdy nemůže stát. `Option` nás nutí tuto možnost explicitně označit a později zkontrolovat, než můžeme přistupovat k datům uvnitř.

Většina typů dat může nabývat jakékoli hodnoty včetně nuly, takže kromě dané hodnoty je potřeba zároveň přenášet i dalších několik bajtů označujících, zda se jedná o `Some` nebo `None`. Ale u některých hodnot, zejména u referencí, už kompilátor ví, že *nikdy* nemohou nabývat hodnoty nula (protože reference vždy ukazují na platná data a nemohou být nulovými ukazateli), a tudíž může využít hodnotu nuly (nulového ukazatele) jako variantu `None`. To je nazýváno „optimalizace nulovým ukazatelem“, a díky explicitnosti enumu `Option` je stále bezpečné.

Na obrázku 34 je jeho poměrně jednoduchá definice:

```
enum Option<T> {
    None,
    Some(T),
}
```

Obrázek 34: Definice enumu Option. Zdroj: (1), kapitola 6.1

Syntaxe `<T>` je generická syntaxe popsaná níže, zatím stačí jen vědět, že místo `T` si můžeme nahradit jakýkoli jiný typ nebo referenci na něj, např. `char`, `&mut i32`, `&[u16]` nebo složitější typy.

Pokud potřebujeme komunikovat více informací než jen případnou absenci dat, poskytuje nám standardní knihovna enum `Result`. Ten má také dvě možnosti – `Ok` a `Err` – ale rozdíl spočívá v tom, že i možnost `Err` může obsahovat data, která nemusí (ale mohou) být stejného typu jako data v možnosti `Ok`. Tím dokáže přenést informace třeba o tom, jaká konkrétní chyba se stala. Je zvykem používat enumy namísto číselných chybových kódů, protože enumy jsou čitelnější a nehrozí, že by programátor omylem navrátil buď neexistující kód, nebo kód pro jinou než zamýšlenou chybu.

`Result` definován následovně:

```
enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

Obrázek 35: Definice enumu Result. Zdroj: (1), kapitola 9.2

Opět jsou použity generické parametry, ale protože jsou použita dvě písmena, `<T, E>`, je možné (ale ne nutné) použít pro každou možnost jiný typ.

Pro přístup k hodnotám máme několik možností. Nejprimitivnější je zavolání metody `.unwrap()` na enumu, která pokud je hodnota `Some` nebo `Ok`, tak navrátí vnitřní hodnotu, a pokud je hodnota `None` nebo `Err`, tak takzvaně „zpanikaří“ – okamžitě zastaví běh programu a ukončí ho. Metoda `.expect()` funguje stejně, ale jako parametr předáme metodě textový řetězec, který bude při panice vyhozen jako chybová hláška a může pomoct při hledání původu chyby.

```
fn main() {
    let greeting_file_1 = File::open("hello1.txt").unwrap();
    let greeting_file_2 = File::open("hello2.txt")
        .expect("hello.txt should be included in this project");
}
```

Obrázek 36: Příklad použití metod .unwrap() a .expect(). Zdroj: (1), kapitola 9.2, upraveno. Importy vynechány

Nejčastější způsob, jak program zpanikaří, je zavolání výše zmíněné metody `.unwrap()`, pokus o přístup k prvku v poli mimo rozsah daného pole nebo úmyslné zpanikaření makrem `panic!()`. Příklad na obrázku 37: pole je 3 prvky dlouhé, ale pokoušíme se o přístup ke stému prvku (píšeme 99, protože indexujeme od nuly).

```
fn main() {
    let v = vec![1, 2, 3];

    v[99];
}
```

Obrázek 37: Tento program zpanikaří, protože se pokusí o přístup prvku v poli mimo rozsah. Zdroj: (Klabnik, a další, 2025), kapitola 9.1

```
thread 'main' (13) panicked at src/main.rs:4:6:
index out of bounds: the len is 3 but the index is 99
```

Obrázek 38: Chybová hláška poté, co program z obrázku 37 zpanikařil. Zdroj: vlastní

6.3.4 Match výrazy, vzorce

Rust podporuje speciální syntaxi tzv. vzorců („patterns“). Nejjednoduššími vzorci jsou literaly – pevně zakódované hodnoty, které se musí 100% shodovat – nebo rozsahy, které se shodují s jakoukoli hodnotou spadající do daného (také pevně zakódovaného) rozsahu hodnot. Dále vzorce umožňují rozkládat složené datové typy (pole, uspořádané n-tice, struktury nebo enumy), vytvářet proměnné a další.

Nejčastěji jsou vzorce používány ve výrazech `match` („shodovat, přiřadit, spárovat“) (v jiných jazycích také „switch statement“). Match přijímají na svém začátku nějakou hodnotu a pak jdou jeden řádek po druhém, testujíce danou hodnotu na shodu se vzorcem. U první shody pak match spustí blok kódu přiřazený danému vzorci. Pokud chceme s chybou ve výše zmíněných enumech nějak pracovat, je použit nejčastěji právě match, aby byly spuštěny různé bloky kódu právě podle hodnoty enumu, bez potřeby zdlouhavých a špatně čitelných podmíněných výrazů `if`.

Výběr, který blok má být spuštěn, je proveden tak, že výraz dodaný match je vyřešen – je získána jeho hodnota, a pak jde match od prvního bloku dolů a kontroluje, zda je získaná hodnota rovna hodnotě, kterou jsme označili daný blok, případně zkontroluje, že dodatečné podmínky jsou pravdivé. Match musí být vyčerpávající, musí mít blok pro <u>každou</u> možnou hodnotu výrazu, proto je často na konec match přidáván všezáchytný catch-all vzorec – buď pouze nějaký název proměnné, který zachytí jakoukoli hodnotu a uloží ji do proměnné, nebo zástupný symbol podtržítka `_`, který nejen že zachytí jakoukoli hodnotu, ale explicitně ji zahodí (aby si kompilátor nestěžoval na nevyužitou proměnnou). Enumy označené atributem `#[non_exhaustive]` jsou dále omezeny tak, že vyžadují všezáchytný vzorec vždy, i když byly vyčerpány, aby tak donutily programátora nějak pracovat s možností, že někdy v budoucnosti by mohla být přidána nová varianta, kterou ve svém zdrojovém kódu neočekával.

Pokud používáme match na enumu s daty, do závorek za jméno varianty enumu je možné dát další název nebo názvy, a data uvnitř enumu se stanou novými proměnnými uvnitř daného bloku kódu s dodanými názvy. Před název můžeme napsat `mut`, abychom vytvořili měnitelnou proměnnou. Tuto proměnnou – hodnotu enumu – můžeme navrátit mimo match právě tím, že na konec bloku kódu dáme název proměnné bez středníku. Stejně tak můžeme použít nějaký název jako zástupného symbol namísto podtržítka, a vytvořit proměnnou s hodnotou, na které provádíme match.

Poslední výraz v bloku kódu však nemusí být ukončen středníkem: když není, tak celý match přejímá hodnotu tohoto výrazu. Tento poslední výraz může být například jméno proměnné, jejíž hodnota se stane výsledkem bloku kódu, nebo volání funkce, kdy je použita hodnota, kterou funkce navrátila. Mimochodem, stejný princip platí i u funkcí – poslední výraz ve funkci nemusí používat klíčové slovo `return`, pokud není zakončen středníkem. Pokud však funkce nevrací žádnou hodnotu, nesmí ji navracet ani poslední výraz, jinak je středník nutný, jinak si bude kompilátor stěžovat na pokus o navrácení hodnoty špatného typu (funkce, které nevrací žádnou hodnotu, technicky vzato vrací prázdnou n-tici, zvanou „jednotkový typ“ – „unit type“, značený prostě `()`).

Na obrázku 39 je příklad použití match, jeho syntaxe i způsoby navracení hodnot. Podobně funguje i enum `Option`, akorát nepřidá jméno proměnné do závorky za možnost `None`, protože `None` neobsahuje žádná data. (Pro přístup k variantám nemusíme používat syntaxi `Result::Ok` jako u jiných

enumů, ale jen `Ok`, protože Rust automaticky importuje varianty enumů `Result` i `Option` jako samostatná klíčová slova.)

```
fn main() {
    let file_result: Result<_, _> = File::open("hello.txt");

    let file = match file_result {
        Ok(mut file) => { // Unnecessary mut
            println!("File found, opening.");
            file
        },
        Err(error) => match error.kind() {
            ErrorKind::NotFound => {
                println!("File not found, creating file.");
                File::create("hello.txt")
                    .expect("Failed to create file")
            }
            _ => panic!("Problem opening the file: {error:?}")
        },
    };
}
```

Obrázek 39: Příklad použití match. Zdroj: (1), kapitola 9.2, upraveno, importy vynechány

Avšak psát mnoho match se může rychle stát velmi nečitelným, proto Rust poskytuje operátor otazníku: `?`. Tento operátor lze připojit na konec výrazu, který navrací `Result` nebo `Option`, a funguje vlastně jako `.unwrap()`, ale pokud se jedná o možnost `None`, tak funkce, ve které byl operátor otazníku zavolán, předčasně ukončí průběh a navrátí hodnotu `None`, nebo pokud se jedná o `Err`, tak funkce předčasně navrátí hodnotu `Err` se stejnými daty. Pokud by však funkce navracela jiný typ hodnoty v možnosti `Err`, než ten, který byl dodán operátoru, operátor se automaticky pokusí o převod typu proměnné, pokud je převod možný (pokud typ, do kterého převádíme, implementuje trait `From<T>`, kde `T` je typ, ze kterého převádíme – vše popsáno níže).

Na obrázku 40 vidíme příklad použití operátoru otazníku ve funkci, která navrací `Result`:

```
fn read_username_from_file() -> Result<String, io::Error> {
    let mut username_file = File::open("hello.txt")?;
    let mut username = String::new();
    username_file.read_to_string(&mut username)?;
    Ok(username)
}
```

Obrázek 40: Příklad použití operátoru otazníku. Zdroj: (1), kapitola 9.2. Importy vynechány

Rust poskytuje i jiné způsoby, jak zpracovávat chyby pomocí enumů, které také používají vzorce, ale těmi se v tomto dokumentu nebudeme podrobně zabývat, protože se ve své podstatě jedná jen o zjednodušený, zkrácený zápis, jak udělat to stejné, co pomocí match. Pro účely tohoto projektu stačí poznamenat, že výrazy `if let` rozbalují variantu enumu do proměnné existující v jejich vnitřním bloku, případně s blokem `else` spuštěným, pokud se vzorec nebude shodovat, a výrazy `let … else` vytvoří

proměnnou ve vnějším kontextu, a pokud se vzorec nebude shodovat, spustí (zde povinný) blok `else`, který musí být tzv. divergentní – nesmí dojít do konce, musí končit výrazem `return`, `break`, `panic!()` apod, aby nemohl kód pokračovat dále a pokusit se o práci s neexistující proměnnou. Na obrázku 41 je jejich typické použití a ekvivalentní kód napsaný pomocí match:

```
fn main() {
    let x = Some(0);

    match x {
        Some(value) => println!("The value is: {}", value),
        None => println!("No value found."),
    }

    if let Some(value) = x {
        println!("The value is: {}", value);
    } else {
        println!("No value found.");
    }
    //println!("The value is: {}", value);

    // ----------------------------------------------------

    let y = Some(10);

    let num;
    match y {
        Some(value) => { num = value },
        None => {
            println!("Error occurred.");
            return;
        }
    }
    println!("The value is: {}", num);

    let Some(num) = y else {
        println!("Error occurred.");
        return;
    };
    println!("The value is: {}", num);
}
```

Obrázek 41: Příklad výrazů if-let a let-else. Zdroj: vlastní

6.4 Generické parametry a traity

Jeden z hlavních problémů, kterému se v programování chceme vyhýbat, je zbytečná duplikace kódu. Proto používáme smyčky, proto používáme funkce, proto používáme mnoho různých dalších konstruktů. Další krok je umožnění, aby mohla funkce přijímat parametry (nebo navracet hodnoty) různých typů, případně omezit tyto typy na určitou podmnožinu typů, které mají určité vlastnosti.

V jazyce Rust mohou funkce, struktury nebo enumy mít definovány tzv. generika. Zjednodušeně řečeno, generikum je zástupce za nějaký konkrétní typ, a během kompilace, v procesu zvaném *monomorfizace*, kompilátor zjistí všechny typy, které byly v našem kódu dosazeny do generických parametrů a vygeneruje specifickou kopii funkce (nebo struktury nebo enumu) pracující s daným konkrétním použitým typem, pro každý použitý typ. Z pohledu programátora však vždy voláme tu jednu stejnou

funkci (nebo strukturu nebo enum), se stejným názvem a stejným zdrojovým kódem, nehledě na použitý typ.

U enumů `Option<T>` (obrázek 34) a `Error<T, E>` (obrázek 35) jsme mohli vidět, že generické parametry píšeme do hranatých závorek (které na klávesnici píšeme znaky „menší než“ a „větší než“). Tyto hranaté závorky se používají i při použití nějakého existujícího typu, který obsahuje generické parametry a kdy nepoužijeme inferenci (například v parametrech funkcí nebo když inference z různých důvodů selže), nejen při jeho deklaraci, jako třeba na obrázku 42:

```
fn main() {
    let num: Option<u8> = None;
    let message = Some("Hello!");
    maybe_say(message);
}

fn maybe_say(input: Option<&str>) {
    match input {
        None => return,
        Some(msg) => println!("{}", msg)
    }
}
```

Obrázek 42: Příklad zadání typů v generickém enumu Option. Zdroj: vlastní

Avšak ne vždy dokážeme pracovat opravdu s jakýmkoli typem. U struktur a enumů to je možné, ale funkce, které s generickými parametry pracují, často ne: většina operací prostě u některých typů nedává smysl. Například nelze sčítat textové znaky, nelze se doptávat na 5. prvek v čísle nebo nelze zobrazit hodnotu nějaké abstraktní struktury reprezentující něco mimo náš kód. Proto Rust umožňuje omezit generické parametry pomocí traitů.

Trait (česky asi “rys”, “vlastnost” nebo “příznak”) je konstrukt definující abstraktní chování určitého typu, které může být sdílené s jinými typy. Implementovat daný trait pro určitý typ je pak garance programátora, že typ, který daný trait implementuje, má vlastnosti vyžadované tímto traitem a implementuje určité metody jím vyžadované. Všechny typy implementující daný trait mají dané abstraktní chování sémanticky stejné nebo velmi podobné, i když se může samotná implementace v kódu lišit. Laicky řečeno tedy traity označují, co můžeme s daným typem dělat. Například trait `Clone` nám říká, že pomocí metody `.clone()`, kterou `Clone` vyžaduje, můžeme vytvořit sémantickou kopii nějaké hodnoty (sémantickou kopii = kopii o stejném „logickém významu“, ale ne nutně stejných bitech). Některé traity mohou vyžadovat implementaci jiných traitů, například trait `Eq` vyžaduje implementaci `PartialEq`. `Eq` je pak tzv. „subtrait“, zatímco `PartialEq` je „supertrait“. Na obrázku 43 můžeme vidět generickou funkci, která se nezkompiluje, protože její generický parametr není správně omezen, a tudíž není garance, že by na něj šel aplikovat operátor `>` (větší než):

```rust
fn largest<T>(list: &[T]) -> &T {
    let mut largest = &list[0];

    for item in list {
        if item > largest {
            largest = item;
    }}

    largest
}
```

Obrázek 43: Chybný kód generické funkce, kterému chybí omezení generického parametru. Zdroj: (1), kapitola 10.1, upraveno

```
error[E0369]: binary operation `>` cannot be applied to type
`&T`
 --> .\test_code.rs:5:17
  |
5 |         if item > largest {
  |            ---- ^ ------- &T
  |            |
  |            &T
  |
help: consider restricting type parameter `T` with trait
`PartialOrd`
  |
1 | fn largest<T: std::cmp::PartialOrd>(list: &[T]) -> &T {
  |             +++++++++++++++++++++
```

Obrázek 44: Chybová hláška po pokusu o kompilaci kódu z obrázku 43. Zdroj: vlastní

Ale jak nám již chybová hláška z obrázku 44 napovídá, můžeme omezit generický parametr jen na typy implementující trait `PartialOrd`, který nám umožní funkci zkompilovat. Toto omezení deklarujeme buď tak, že za jméno generického parametru dosadíme dvojtečku `:` a jméno traitu, který parametr musí implementovat (jako na obrázku 44), nebo klíčové slovo `where`, které je vhodnější pro větší počet parametrů a omezení a které dosadíme přímo před samotný kód funkce spolu s čárkami odděleným seznamem omezení, jako na obrázku 45:

```rust
fn some_function<T, U>(t: &T, u: &U) -> i32
where
    T: Display + Clone,
    U: Clone + Debug,
{
```

Obrázek 45: Příklad omezení generického parametru pomocí klíčového slova where. Zdroj: (1), kapitola 10.2

Tyto omezení lze aplikovat kdekoli, kde lze použít generické parametry. Obzvláště praktické jsou v blocích impl deklarujících metody, protože nám umožňují podmíněně deklarovat metody – zajistit, že určité metody půjde použít pouze, když jsou omezení naplněna. Příklad na obrázku 46:

```
struct Pair<T> {
    x: T,
    y: T,
}

impl<T> Pair<T> {
    fn new(x: T, y: T) -> Self {
        Self { x, y }
    }
}

impl<T: PartialOrd> Pair<T> {
    fn cmp_display(&self) -> bool{
        if self.x ≥ self.y { true } else { false }
    }
}
```

Obrázek 46: Příklad omezení metody. Zdroj: (1), kapitola 10.2, upraveno

Na obrázku 47 vidíme opravenou funkci z obrázku 43, která mimochodem také zkontroluje, jestli není dodané pole prázdné (což by v kódu, který s tím nepočítá, vyvolalo paniku), a případně navrátí variantu `None` použitého enumu `Option`.

```
fn largest<T: std::cmp::PartialOrd>(list: &[T]) -> Option<&T> {
    if list.len() == 0 { return None };

    let mut largest = &list[0];
    for item in list {
        if item > largest {
            largest = item;
        }
    }

    Some(largest)
}
```

Obrázek 47: Opravený kód z obrázku 43. Zdroj: vlastní

Zároveň nám obrázek 47 ukazuje další kousek syntaxe, který používáme u generik: tzv. turbofish (doslovně přeloženo „turboryba“ – název vznikl z vizuální podobnosti), značený `::<>`. Mezi hranaté závorky vkládáme opět jméno typu. Turbofish přidáme mezi jméno funkce (nebo struktury nebo enumu) a další syntaxi (jako závorky), a používáme ji, abychom pomohli kompilátoru v případech, kdy si sám nedokáže inferovat, jaký typ má dosadit do generického parametru. Například právě na obrázku 47 kompilátor neví, jaké typy by mělo obsahovat prázdné pole, takže mu musíme poradit, že se má zachovat, jako by to bylo pole hodnot typu `f64`. Pokud bychom však chtěli / potřebovali specifikovat jen jeden z několika generických parametrů, můžeme na místo těch ostatních dosadit podtržítko `_`, aby si kompilátor ty ostatní inferoval. Obecně, pokud tento zástupný symbol použijeme namísto nějakého

typu, říká to kompilátoru „tento typ inferuj“. Další příklad tohoto použití jsme viděli třeba na obrázku 39, kdy jsme typ proměnné určili jako `Result<_, _>` – řekli jsme kompilátoru, aby si samotné typy dat ve variantách inferoval, ale že chceme, aby typ celé té proměnné byl nějaký `Result`.

Svůj vlastní trait můžeme definovat pomocí klíčového slova `trait`, a uvnitř něj můžeme definovat „podpisy funkcí“ („function signatures“) – funkce bez těla (bez kódu, který by spustily), definující pouze, jaké parametry jakých typů musí funkce v implementaci přijímat a jaký typ musí navracet. (Obvykle ale dokumentace ještě dále specifikuje, *co* by měla metoda dělat, pochopitelně.)

Můžeme však ale i dát funkcím tělo: potom se tělo funkce stane výchozí implementací, a implementace daného traitu nemusí (ale mohou) tuto funkci definovat samy. (Pokud tak učiní, přepíšou výchozí implementaci svou konkrétní.) Kód těchto výchozích implementací pak může využívat ostatní funkce vyžadované traitem, i ty bez výchozí implementace. Zatímco deklarace běžných metod využívaly pouze klíčové slovo `impl`, zde je použita dvojice `impl <trait> for` označující jak implementovaný trait, tak implementující objekt.

I traity mohou mít generické parametry, analogicky značené hranatými závorkami `<T>`, pro možnost nahrazení nějakého konkrétního typu generickým parametrem s případnými omezeními. Pomocí klíčového slova `type` pak mohou mít traity ještě tzv. asociovaný typ, který funguje podobně jako generický parametr, ale specifikuje ho implementace traitu namísto jeho uživatele. K názvům těchto asociovaných typů je přistupováno v implementaci traitu pomocí dvou dvojteček `::`, podobně jako u asociovaných funkcí. Příklad asociovaných traitů je na obrázku 48:

```
trait Contains {
    type A;

    fn contains(&self, item: &Self::A) → bool;
    fn first(&self) → i32;
    fn last(&self) → i32;
}
```

Obrázek 48: Příklad deklarace a použití asociovaných typů traitu. Zdroj: (2), kapitola 14.8.2

Příklad definice i implementace vlastního traitu je na obrázku 49 na další straně.

```rust
trait Summary {
    fn summarize_author(&self) -> String;
    fn summarize(&self) -> String {
        format!("(Read more from {}...)", self.summarize_author())
    }
}

struct NewsArticle {
    headline: String,
    author: String,
    content: String,
}

impl Summary for NewsArticle {
    fn summarize(&self) -> String {
        format!("{}, by {}", self.headline, self.author)
    }

    fn summarize_author(&self) -> String {
        format!("By {}", self.author)
    }
}

struct SocialPost {
    username: String,
    content: String,
    repost: bool,
}

impl Summary for SocialPost {
    fn summarize_author(&self) -> String {
        format!("@{}", self.username)
    }
}

fn main() {
    let article = NewsArticle {
        headline: String::from("Penguins win the Stanley Cup!"),
        author: String::from("John Doe"),
        content: String::from("The Pittsburgh Penguins once win."),
    };
    let post = SocialPost {
        username: String::from("horse_ebooks"),
        content: String::from("The best way to learn is to do."),
        repost: false,
    };

    println!("{}", article.summarize());
    println!("{}", post.summarize());
}
```

Obrázek 49: Příklad deklarace vlastního traitu i jeho implementace na strukturách. Zdroj: (1), kapitola 10.2, upraveno

6.4.1 Často používané traity

Standardní knihovna nám poskytuje mnoho traitů pro základní funkce, a pro správné pochopení generického kódu je třeba znát alespoň ty nejpoužívanější. Pro některé z nich poskytuje navíc makro `derive`, které automaticky derivuje implementace traitů, u kterých je to možné. Těmito traity jsou: `Clone`, `Copy`, `Debug`, `Default`, `Eq`, `Hash`, `Ord`, `PartialEq` a `PartialOrd`. Obecně platí, že u struktur je možné derivovat pouze, když všechny typy dat v nich obsažených implementují daný trait, analogicky tomu tak je i u enumů a dat v jejich variantách.

Trait `Clone` je již vysvětlen výše: vyžaduje metodu `.clone()`, pomocí níž můžeme vytvořit sémantický klon nějaké hodnoty. Například u typu `String`, který, jak bylo zmíněno v kapitole 6.2.5, je ukládaný na heap, nestačí ale jen zkopírovat bity, protože pak bychom měli dva objekty ukazující na stejnou paměť na heapu (které si oba myslí, že na ni nic jiného neukazuje), což by mohlo vést k mnohým konfliktům jako třeba používání paměti po jejím uvolnění. Proto implementace `Clone` pro `String` alokuje novou paměť na heapu a zkopíruje pouze samotný textový řetězec. Ukazatel na heap a tedy bity tvořící tuto strukturu v paměti jsou jiné, ačkoli sémantická hodnota (uložený text) je stejná.

Trait `Copy` je pak zobecněním `Clone`: označuje, že data *je* možné naklonovat pouhým zkopírováním bitů bez žádných problémů nebo vedlejších účinků, což platí například pro všechny číselné typy nebo booleany. Protože kopírování bitů není náročná operace, je kopie vytvořena automaticky při každém předávání přes hodnotu, aniž by majiteli zanikalo vlastnictví nad původní předávanou hodnotou. `Clone` nevyžaduje žádné metody, je to tedy jen tzv. marker trait („označovací trait“).

Traity `PartialEq` a `Eq` jsou používané pro operátor rovnosti `==` a jsou implementované pro typy, kde dává smysl posuzovat rovnost dvou hodnot (stejného typu). `PartialEq` říká kromě možnosti porovnávání ještě, že když $a = b$, tak $b = a$, a že když $a = b$ a $b = c$, tak $a = c$. Vyžaduje pouze metodu `eq()`, která navrátí boolean, zda jsou si hodnoty rovny. `Eq` toto ještě rozšiřuje garancí, že $a = a$, což neplatí například pro čísla s plovoucí desetinnou čárkou, kdy hodnota `NaN` („=not a number“ – není číslo) není rovna sama sobě, nebo pro jiné typy s variantou nedefinované hodnoty. Pro implementaci `Eq` je povinné implementovat i `PartialEq`, ale není potřeba implementovat žádné metody, je to pouze garancí, při jejímž porušení nastávají logické (ne nutně počítačové) chyby. Když jsou tyto dva traity derivovány pro strukturu, musí být všechny její pole stejné, aby byly struktury stejné, při derivaci pro enum musí být stejná varianta a její data musí být rovné.

Traity `PartialOrd` a `Ord` poskytují možnost porovnávání hodnot pomocí operátorů `<`, `<=`, `>`, a `>=` a analogicky jsou implementovány pro typy, kde dává smysl porovnávat nebo dokonce seřazovat hodnoty. `PartialOrd` vyžaduje implementaci metody `partial_cmp()`, která přijme dvě hodnoty stejné metody a navrátí enum v `Option`, zda je první hodnota větší, rovna nebo menší než druhá

hodnota, nebo navrátí None, je-li porovnání nemožné (například opět u hodnoty NaN), což zapříčiní, že každý operátor porovnání je nepravdivý. Vyžaduje implementaci PartialEq. Trait Ord je pak rozšířením garantujícím, že jakákoli dvojice hodnot jde nějakým způsobem porovnat. Vyžaduje implementaci traitů Eq i PartialOrd, a jím vyžadovaná metoda cmp() navrací pouze použitý enum, bez Option. Když tyto dva traity derivujeme pro enumy, každá varianta je vzestupně očíslovaná (pokud neočíslujeme varianty manuálně) v pořadí, ve kterém se objevuje ve zdrojovém kódu, a tyto čísla jsou porovnávány. Pokud se jedná o stejnou variantu, jsou dále porovnávána data ve variantě, jsou-li nějaká. Při derivaci traitů jsou struktury porovnávány podle prvních polí s nestejnou hodnotou, enumy podle variant a v případě shody varianty podle dat uvnitř. (Pole i varianty jsou uspořádány shora dolů podle jejich deklarace ve zdrojovém kódu).

Traity From<T> a Into<T> jsou používané pro převádění mezi typy. Jsou to generické traity – přijímají generický parametr, který musí programátor během implementace specifikovat, a může tento trait implementovat vícekrát s různými generickými parametry. Kdykoli to je možné, je doporučeno implementovat From namísto Into, protože Into je implementován automaticky pro každý typ implementující From, ale při omezování generických parametrů je naopak doporučeno omezovat pomocí Into kvůli případům, kdy je možné implementovat pouze Into. Převody nesmí selhat, nesmí ztrácet data a musí zachovat logický význam (např. nelze převést hodnotu -1 typu i8 na typ u8, který nedokáže reprezentovat záporná čísla). Pro převody, které mohou selhat, existují traity TryFrom<T> a TryInto<T>.

Pokud aplikujeme výše zmíněný operátor otazníku ? na variantu Err, automaticky zavolá metodu from() poskytovanou implementací typu varianty Err, který je specifikován v podpisu funkce (= který se naše funkce pokouší navracet), takže je doporučené implementovat From pro naše vlastní enumy reprezentující chyby, abychom mohli použít otazník i na funkce navracející chyby jiného typu, a pak převést jejich navrácenou chybu na odpovídající variantu našeho enumu.

Trait Default vyžaduje metodu default(), která jednoduše navrátí výchozí hodnotu určitého typu. Například pro číselné typy, výchozí hodnota je nula. Pokud ho derivujeme pro struktury, každé pole v nich musí implementovat Default. Pokud ho derivujeme pro enumy, musíme označit výchozí variantu atributem #[default], tato varianta pak nesmí obsahovat žádná data, jinak musíme implementovat Default manuálně.

Dále jsou dva velmi podobné traity: Debug a Display. Oba jsou použity při formátování nějaké hodnoty na text, s tím rozdílem, že Debug navrací formátování pro programátora pro ladění, často surové hodnoty, a Display navrací formátování, které má být zobrazeno uživateli. Debug lze derivovat pomocí výše zmíněného makra, ale Display ne, protože zobrazení struktury nebo enumu uživateli

často vyžaduje abstraktnější myšlení o logickém významu hodnoty, nejen zobrazení, z jakých dat se struktura skládá nebo jaké varianty enum právě nabývá. Pro formátování traitem `Display` používáme (mimo jiné) formátovací značku `{}`, pro `Debug` používáme `{:?}`. Po spuštění příkladu na obrázku 50 je nejdříve vypsáno `201 kilohertz` (formátování `Display`), poté prostě `kHz(201)` (formátování `Debug`):

```
#[derive(Debug)]
enum Freq {
    Hz(u32),
    kHz(u32),
    MHz(u32),
    GHz(u32)
}

impl std::fmt::Display for Freq {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) ->
std::fmt::Result {
        match self {
            Freq::Hz(val) => write!(f, "{} hertz", val),
            Freq::kHz(val) => write!(f, "{} kilohertz", val),
            Freq::MHz(val) => write!(f, "{} megahertz", val),
            Freq::GHz(val) => write!(f, "{} gigahertz", val),
        }
    }
}

fn main() {
    let freq = Freq::kHz(201);
    println!("{}", freq);
    println!("{:?}", freq);
}
```

Obrázek 50: Příklad pro ilustraci rozdílu mezi Debug a Display. Zdroj: vlastní

Existují i další formátovací traity, jako `Binary`, `LowerExp`, `UpperExp`, `LowerHex`, `UpperHex` nebo `Octal`, většina z nichž jsou implementovány pro číselné typy pro jejich formátování v jiných číselných soustavách nebo zápisech.

6.4.2 Iterátory

Trait `Iterator` je poněkud složitější: označuje obecně iterátory. Iterátor je nějaký objekt, přes který lze iterovat, ale to je cyklická definice. Iterace je obecně proces opakovaného spouštění určitého bloku kódu pro každý prvek v nějaké kolekci prvků. Jedním ze známých příkladů iterace jsou for-smyčky. Často používaným iterátorem je rozsah, který je ve své podstatě jednoduchou kolekcí obsahující všechny celá čísla mezi zadanými okraji rozsahu (v některých případech včetně), používaný často pro opakované spuštění nějakého bloku kódu n-krát. Většinou může daný blok zjistit, u jakého číslo v rozsahu právě je – po kolikáté daný blok běží.

Iterátor může mít mnoho implementací a funkcionalit, ale v kontextu jazyka Rust se obecně projevuje tak, že daný iterátor jako by v sobě obsahoval seznam hodnot – kolekci, a pokaždé, když zavoláme jeho

jedinou požadovanou metodu bez výchozí implementace, `.next()` („další“), odstraní jednu hodnotu z tohoto imaginárního seznamu a navrátí nám ji zabalenou přes `Option`, nebo navrátí `None`, pokud je jeho seznam prázdný. Trait `Iterator` má také asociovaný typ `Item`, což je prostě typ dat, které bude iterátor vydávat, a implementace traitu ho musí (povětšinou generickým parametrem) specifikovat.

Všechny další metody iterátorů kromě `.next()` mají výchozí implementace, které (skrze několik vrstev) konečně závisí pouze na metodě `.next()` (i když výchozí implementace nemusí být vždy nejoptimálnější – proto mnoho iterátorů nahrazuje některé z nich vlastními implementacemi, které mohou pracovat se specifickými vlastnostmi dané konkrétní struktury), nebo zabalují iterátor do další struktury, která je sama iterátorem a která nějakým způsobem mění chování vnitřního iterátoru (např. metoda `.map()`[5]), když je sama iterována. Takovéto struktury jsou pak nazývané „adaptéry“ nebo „nástavce“ („adapters“).

Protože prvky z imaginárního seznamu uvnitř iterátoru odstraňujeme, říkáme, že prvky *konzumujeme*, a když zkonzumujeme všechny, tak jsme zkonzumovali celý iterátor. U určitých iterátorů je možné, aby se po vyprázdnění iterátor znovu naplnil a znovu začal navracet data ve variantách `Some`[6]. Iterátory jsou však tzv. líné, což znamená, že dokud opravdu na nich nezavoláme nějakou metodu, nestane se vůbec nic.

V závislosti na implementaci je možné (ale ne nutné), aby samotný akt konzumace prvku nějak měnil data, ze kterých byl iterátor sestaven, nebo aby samotné prvky iterátoru byly měnitelnými referencemi na původní data.

Iterátory můžeme vytvářet například z rozsahů, z polí nebo textových řetězců zavoláním nějaké metody zprostředkované daným typem proměnné. Protože jsou iterátory velmi specifické pro daný typ, a protože je v některých případech i několik způsobů, jak sestrojit z nějakého typu iterátor[7], jsou jména metod různá a je potřeba si přečíst dokumentaci daného typu. Existuje však také trait `IntoIterator`, který umožní automatické vytvoření iterátoru například pro použití ve smyčce `for`, která jednoduše převede vstup do iterátoru a stále volá metodu `.next()`, dokud není navrácen `None`. Jeho metoda pro

---

[5] Metoda `.map()` přijímá jako parametr ještě funkci a zabalí iterátor do struktury `Map`. Po zavolání `.next()` na této struktuře zkonzumuje jeden prvek z původního iterátoru, předá ten prvek jako parametr dané funkci (pokud je navrácena možnost `Some`) a navrátí výsledek.

[6] Iterátory implementující trait `FusedIterator` však garantují opak: poté, co jednou navrátí `None`, budou nadále vždy navracet `None`. Pomocí metody `.fuse()` lze jakýkoli iterátor k tomuto chování donutit zabalením do struktury `Fuse` („pojistka“), která po prvním výskytu `None` původní iterátor zničí a dále navrací pouze `None`. Název pochází nejspíše z toho, že se po prvním navrácení `None` se přepálí pomyslná „pojistka“ a iterátor přestane fungovat, analogicky k elektrické pojistce.

[7] Například textový řetězec poskytuje `.chars()` iterující přes jednotlivé Unicode znaky, `.bytes()` iterující přes jednotlivé bajty kódující znaky kódováním UTF-8, `.split()` rozdělující řetězec podle určitého vzorce a iterující po jednotlivých dílcích, a další.

převod, `.into_iter()`, ale konzumuje původní variantu, proto není vždy optimální ji použít, pokud je potřeba pracovat pouze s referencemi, v takovém případě používáme metody dodané konkrétním typem. Každý iterátor tento trait automaticky implementuje jednoduše tak, že navrátí sám sebe.

Protože však existuje mnoho druhů iterátorů a protože náš často ani nezajímá, jaký to iterátor vlastně dostaneme, hlavně když to bude iterátor, tak by se ve funkcích mnoha případech nabízela možnost používat generické parametry omezené na trait `Iterator`. Ale Rust nám umožňuje ještě elegantnější řešení: když omezujeme na pouze jeden trait, nemusíme používat syntaxi `<T: Jmeno_traitu>` jako na obrázku 45, ale můžeme použít klíčové slovo `impl` a název traitu pro označení, že zde lze dosadit jakýkoli typ implementující daný trait, a že nás nezajímá, jaký typ to bude. Můžeme dokonce specifikovat, jaký typ má iterátor navracet. Tuto syntaxi však nelze použít všude, jen u typů parametrů a návratových hodnot funkcí. Protože se však nejedná o skutečný generický parametr, nelze používat například syntaxi turbofish a podobně. Pokud funkce používá `impl` ve svém návratovém typu, může navrátit jakýkoli typ implementující daný trait, ale v rámci ní musí navrátit vždy ten stejný. Příklad je na obrázku 50 – první dvě funkce se zkompilují, třetí selže, protože ačkoli metoda `.chars()` na textovém řetězci i metoda `.iter()` na referenci na pole (na „plátku“) navracejí iterátor, navracejí jiný typ iterátoru – navracejí jiné struktury, přestože obě z nich implementují trait `Iterator`:

```
fn compiles<'a>() -> impl Iterator<Item = &'a u16>
{
    let arr: &[u16] = &[1, 1, 2, 3, 5, 8, 13];
    arr.iter()
}

fn compiles_too(maybe: bool) -> impl Iterator {
    let arr: &[i64];
    if maybe {
        arr = &[1, 1, 2, 3, 5, 8, 13];
    } else {
        arr = &[42];
    };
    arr.iter()
}

fn not_compiles(maybe: bool) -> impl Iterator {
    if maybe { "".chars() } else { [].iter() }
}
```

Obrázek 51: Příklad funkcí s návratovými typy určenými pomocí klíčového slova impl. Zdroj: vlastní

Pokud specifikujeme, že má iterátor navracet reference, nefunguje zde již elize životností jako u klasických parametrů a musíme použít generický argument životnosti.

Rust sice podporuje navracení různých hodnot na základě traitů (nazývá to „dynamic dispatch“ – „dynamický výběr“) a umožňuje tak tvořit tzv. traitové objekty, jejichž typ není během kompilace znám,

jen to, že implementují určitý trait, a u kterých je určováno až za běhu programu, jaký kód má být použit. Traitové objekty však z důvodu složitosti a irrelevance pro projekt nebudou více rozebírány.

6.5 Closure

Rust umožňuje, aby funkce přijímaly za parametry jiné funkce a volaly je ve svém kódu, takové funkce jsou pak nazývány „funkce vyššího řádu“ („higher-order functions“). Protože však není vždy jednoduše čitelné ani praktické deklarovat pro každou drobnou operaci novou funkci a používat ukazatele/reference na funkce, umožňuje nám Rust používat tzv. „uzávěry“ („closures“), v jiných programovacích jazycích nazývaných také např. „lambdy“, které zjednodušeně řečeno fungují jako zkráceně zapsané, anonymní funkce (kromě případů popsaných níže). Stejně jako funkce je lze ukládat do proměnných a předávat jako parametry jiným funkcím. Na obrázku 52 vidíme syntaxi definice closure:

```
fn main() {
    fn  add_one_v1   (x: u32) → u32 { x + 1 }
    let add_one_v2 = |x: u32| → u32 { x + 1 };
    let add_one_v3 = |x|             { x + 1 };
    let add_one_v4 = |x|               x + 1  ;

    // Works fine without
    /*add_one_v1(5_u32);
    add_one_v2(5_u32);*/

    // Needed for the closures to infer their types
    add_one_v3(5_u32);
    add_one_v4(5_u32);
}
```

Obrázek 52: Definice closure. Zdroj: (1), kapitola 13.1, upraveno

Na rozdíl od funkcí, pokud obsahuje closure pouze jeden výraz, mohou být vynechány složené závorky obvykle označující bloky kódu. Stejně tak návratové typy i typy parametrů mohou být inferované, avšak po provedení inference jsou již typy pevně dané, nelze tedy použít absenci typů jako pseudogenerický parametr. Kód na obrázku 53 tudíž selže v kompilaci:

```
fn main() {
    let example_closure = |x| x;

    let s = example_closure(String::from("hello"));
    let n = example_closure(5);
}
```

Obrázek 53: Nesprávné použití closure. Zdroj: (1), kapitola 13.1

```
error[E0308]: mismatched types
 ⟶ .\test_code.rs:5:29
  |
5 |     let n = example_closure(5);
  |             --------------- ^ expected `String`, found integer
  |             |
  |             arguments to this function are incorrect
  |
note: expected because the closure was earlier called with an argument of type `String`
 ⟶ .\test_code.rs:4:29
  |
4 |     let s = example_closure(String::from("hello"));
  |             --------------- ^^^^^^^^^^^^^^^^^^^^^ expected because this argument is of type `String`
  |             |
  |             in this closure call
note: closure parameter defined here
 ⟶ .\test_code.rs:2:28
  |
2 |     let example_closure = |x| x;
  |                           ^
help: try using a conversion method
  |
5 |     let n = example_closure(5.to_string());
  |                              ++++++++++++
```

Obrázek 54: Chybová hláška po pokusu o kompilaci kódu z obrázku 53. Zdroj: vlastní

Pokud closure nepřijímá žádné parametry, značí se prázdnými svislými čarami `||`, aby nedošlo k záměně s bloky kódu, které by byly vyhodnoceny a jejich návratová hodnota pak přiřazena proměnné nebo předána jako parametr. Svislé čáry v jejich syntaxi fungují podobně jako kulaté závorky v syntaxi funkcí. Pokud closure musí přijat parametr např. kvůli traitovému omezení parametrů popsaného níže, ale má ho v úmyslu prostě zahodit a nepoužít ho, napíšeme místo jména parametru podtržítko `_`.

Closure se však od prostých funkcí v jednom ohledu (kromě jejich anonymity) liší: dokáží tzv. „zachytit“ prostředí, ve kterém byly deklarovány, a přistupovat k proměnným z něj, nemusí být jen izolovanými bloky kódu s parametry a návratovými hodnotami. Tudíž při každé deklaraci closure vznikne reference na každou proměnnou z vnějšího prostředí, kterou by closure *mohl* za běhu použít, i kdyby ji ve skutečnosti nepoužil[8], a i pro tyto implicitní reference platí pochopitelně pravidla o koexistenci referencí popsaných v kapitole 6.2.5. Pokud je potřeba vytvořit měnitelnou referenci, je potřeba deklarovat i proměnou uchovávající closure měnitelnou (při přímém předávání closure jako parametru pro funkci není třeba žádného zvláštního značení). Kód na obrázku 55 se nezkompiluje právě kvůli těmto pravidlům, protože proměnná `no_params` obsahuje měnitelnou referenci podobně, jako by ji obsahovala struktura:

[8] Pochopitelně však nevznikají reference na proměnné, se kterými closure vůbec nepracuje. Zde jsou myšleny spíše případy jako „pokud je určitá podmínka pravdivá, pracuj s touto proměnnou, jinak nedělej nic“.

```rust
fn main() {
    let mut x = String::from("abc");
    let mut no_params = || {
        x.make_ascii_uppercase();
    };

    println!("{}", x);
    no_params();
}
```

Obrázek 55: Tento kód se nezkompiluje kvůli pokusu o koexistenci měnitelné a neměnné reference. Zdroj: vlastní

```
error[E0502]: cannot borrow `x` as immutable because it is also borrowed as mutable
 --> .\test_code.rs:7:20
  |
3 |     let mut no_params = || {
  |                         -- mutable borrow occurs here
4 |         x.make_ascii_uppercase();
  |         - first borrow occurs due to use of `x` in closure
...
7 |     println!("{}", x);
  |                    ^ immutable borrow occurs here
8 |     no_params();
  |     --------- mutable borrow later used here
  |
  = note: this error originates in the macro `$crate::format_args_nl` which comes
from the expansion of the macro `println`
```

Obrázek 56: Chybová hláška po pokusu o kompilaci kódu z obrázku 55. Zdroj: vlastní

Pokud chceme místo toho předat closure potřebné proměnné z prostředí přes hodnotu, a tím předat i vlastnictví nad nimi, přidáme před svislé čáry klíčové slovo `move`. Definice tohoto klíčového slova v dokumentaci je (přeloženo do češtiny): *„`move` převede jakékoli proměnné zachycené přes referenci nebo měnitelnou referenci na proměnné zachycené přes hodnotu“*. (3) Proměnné, které closure sám předává přes hodnotu, a tím je nechává zaniknout (tzv. je „konzumuje“), jsou automaticky zachycené přes hodnotu (pokud neimplementují `Copy`) a jejich vlastnictví je předáno closure, ale v určitých případech, např. kdyby mohl mít closure delší životnost než rozsah, ve kterém byl deklarován, je potřeba mu předat vlastnictví i nad proměnnými, které nezkonzumuje, a to je provedeno právě klíčovým slovem `move`.

Hodnoty a vlastnictví nad nimi jsou zachovány mezi voláními, tudíž lze zavolat closure vícekrát, a při druhém spuštění bude mít closure k dispozici proměnné z prvního spuštění, pokud je nezkonzumoval. Totéž ostatně platí pochopitelně i bez použití `move`, protože reference budou ukazovat stále na ty stejné proměnné.

Closure z obrázku 57 můžeme zavolat několikrát, a výstupem celého programu bude `ABC abc ABC`. Pokud bychom nepoužili `move`, vlastníkem proměnné `x` by stále byl rozsah funkce `main()` a tudíž

bychom mohli zobrazit hodnotu `x` na posledním řádku[9]. (V tomto případě není `move` potřeba, přidáváme ho jen pro ilustraci syntaxe a následků jeho použití.)

```
fn main() {
    let mut x = String::from("abc");
    let mut up = false;
    let mut no_params = move || {
        if up {
            x.make_ascii_lowercase();
            up = false;
        } else {
            x.make_ascii_uppercase();
            up = true;
        };
        print!("{} ", x);
    };

    no_params();
    no_params();
    no_params();
    // Would throw "borrow of moved value" error
    //println!("{}", x);
}
```

Obrázek 57: Příklad užití klíčového slova move. Zdroj: vlastní

6.5.1 Ukazatele na funkce a Fn traity*

Pokud potřebujeme přijmout funkci nebo closure jako parametr další funkce, je pochopitelně potřeba zadat typ parametru. Funkce i closure však mají svůj unikátní, nepojmenovatelný typ, který je jednoznačně identifikuje v celém zdrojovém kódu, a proto nelze použít jejich typ přímo. Rust nám však nabízí dva způsoby, jak můžeme přijímat funkce jako parametry: ukazatele na funkce, nebo traity. Oba způsoby pevně určují počet a typy parametrů a návratový typ (příp. jeho absenci).

Ukazatele na funkce jsou samostatné typy, které v sobě obsahuje počet a typy parametrů a návratový typ funkce. Mohou být vytvořeny (většinou implicitním) převodem typu z opravdových funkcí nebo z closure, které nezachytávají *žádné* proměnné ze svých prostředí, pouze přijímají parametry a navracejí hodnoty – tedy které se chovají stejně jako funkce. Protože jsou skutečnými, samostatnými typy, ukazatele na dvě různé funkce se stejnými podpisy (se stejnými počty a typy parametrů a typem návratových hodnot) jsou stejného typu, a tudíž je jednoduché je uchovávat ve strukturách nebo jiných objektech bez potřeby generických parametrů nebo traitových objektů. Právě nepotřebnost generických parametrů je také důvod, proč jsou jediným vhodným způsobem, jak předávat funkce přes

[9] V tomto případě je tento poslední řádek tzv. „vykomentován“ – bylo před něj napsáno `//`, což ho přeměnilo na komentář, na který není během kompilace brán zřetel, za účelem dočasného vyřazení z činnosti – a pokus o zobrazení `x` nenastane, takže se kód úspěšně zkompiluje. Komentáře obecně jsou drobné poznámky ve zdrojovém kódu sloužící pro jeho dovysvětlení pro programátory, kteří ho budou někdy v budoucnosti číst (včetně původního autora samotného), nebo pro dokumentaci funkcí.

FFI („foreign function interface“ – rozhraní pro volání externích funkcí napsaných v jiném programovacím jazyce), protože rozhraní FFI ze své podstaty nemůže být generické, a tudíž nelze použít traity. Kvůli omezením, které však kladou na closure, nejsou ukazatele na funkce příliš často používané jako parametry pro funkce napsané v Rustu.

Traity pro funkce a closure poskytuje Rust celkem tři: `FnOnce`, `FnMut` a `Fn`, dohromady (neoficiálně) označovány jako `Fn*`. Používají se podobně jako jiné traity pro omezování generických parametrů. Ten nejobecnější je `FnOnce`, a označuje closure, u kterých je možné (i když ne nutné), že je lze spustit pouze jednou, protože by nějakým způsobem zkonzumují proměnné, které zachytí, a při dalším spuštění by už proměnné neexistovaly. `FnMut` je již omezenější, protože označuje closure, které je možné spustit vícekrát a které mohou nějak měnit stav = které obsahují měnitelné reference na zachycené proměnné, což s sebou pochopitelně nese omezení ohledně koexistence referencí. A nakonec nejomezenější `Fn` označuje closure, které je možné spustit vícekrát a které obsahují pouze neměnné reference na zachycené proměnné, nebo žádné vůbec nezachytávají. Protože `Fn` vyžaduje implementaci `FnMut`, a `FnMut` vyžaduje implementaci `FnOnce`, každý closure automaticky implementuje všechny aplikovatelné traity (vždy alespoň `FnOnce`). Proto je vhodné při omezování parametrů traity omezit na ten nejobecnější, nejméně restriktivní použitelný trait. Ukazatele na funkci vždy automaticky implementují všechny tři traity, protože nedokáží zachytávat žádné proměnné ze svého prostředí (což odpovídá traitu `Fn`).

Na obrázku 58 lze vidět příklad syntaxe `Fn*` traitů i ukazatelů na funkce jako parametrů běžné funkce (syntaxe `impl <trait>` v podpisech funkcí funguje podobně jako anonymní generický parametr omezený na `<trait>`):

```
fn test(f: impl Fn(u8) → i32) {
    f(20);
}

fn test2(f: fn (u8) → i32) {
    f(55);
}

fn main() {
    let clos = |_| { println!("Hello!"); 0 };

    test(clos);
    test2(clos);
}
```

Obrázek 58: Příklad syntaxe traitů Fn a ukazatelů na funkce. Zdroj: vlastní*

6.6 Moduly

Kdykoli začne nějaký programovací projekt růst ve velikosti a komplexitě, je vhodné rozdělit zdrojový kód do více oddílů, které je jednodušší spravovat a číst. Proto i Rust podporuje dělení zdrojového kódu do crate a do modulů.

Každá knihovna nebo spustitelný program se nazývá „crate“ (dosl. přeloženo „bedna“). Obecně platí, že 1 crate = 1 projekt. Crate se dělí na binární, jejichž výsledek je spustitelný soubor, a knihovny, které jsou použity jako závislosti pro jiné crate. Každá crate se může sestávat z jednoho či více modulů. Jeden modul může obsahovat funkce, struktury a enumy, traity, bloky `impl` a dokonce i jiné moduly. Ve výchozím stavu je celý zdrojový kód součástí jednoho hlavního, implicitního modulu („kořenového“ modulu).

K modulu, nebo nějakému předmětu z něj (funkce, struktura, enum, trait, atd.) lze přistupovat buď skrze jeho plnou cestu, nebo skrze kontext. Ve výchozím stavu máme kontextuální přístup ke všem předmětům ze stejného kontextu, ve kterém píšeme náš kód. Pro přístup k předmětům z modulů obsažených v aktuálním kontextu („dceřiným“ modulům) použijeme jméno modulu, dvě dvojtečky `::` a jméno předmětu. Pro hlubší vnoření je možné za sebe připojit více jmen modulů oddělených dvojtečkami. Pro vyjasnění v nejasných případech (například konfliktní importy) je možné použít klíčové slovo `self` označující modul, ve kterém aktuálně jsme, `super` označující modul o jednu úroveň výše („rodičovský“ nebo „mateřský“ modul) nebo `crate` označující výše zmíněný kořenový modul naší crate.

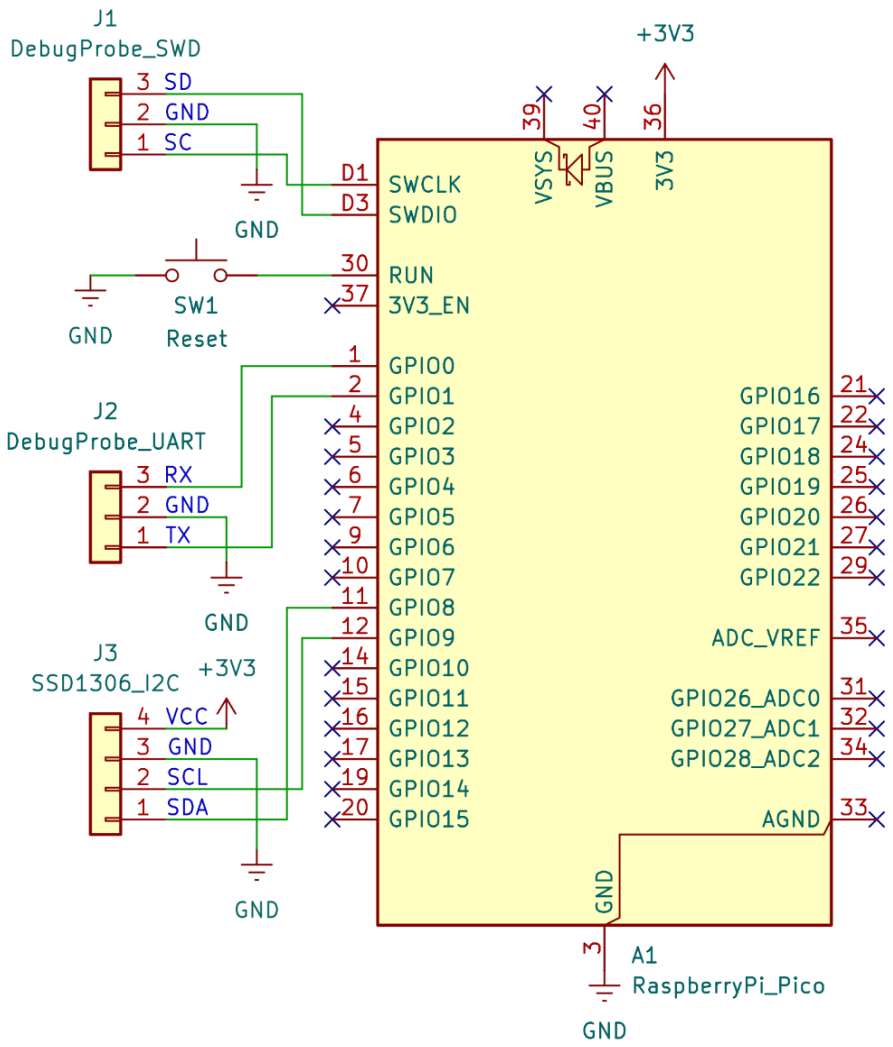
Při přístupu skrze plnou cestu je potřeba použít buď klíčové slovo `crate`, nebo místo něj napsat jméno jiné crate z našich závislostí pro použití objektů z dané externí crate.

PRAKTICKÁ ČÁST

1. Hardware

Jako hardware byla použita výše zmíněná vývojová deska Raspberry Pi Pico a displej SSD1306, spolu s ladicí sondou přímo od výrobce používaného čipu, nazvanou Raspberry Pi Debug Probe. Tato sonda obsahuje výše zmíněná sériová rozhraní SWD a UART. K počítači se připojuje přes USB. Rozhraní UART zprostředkovává počítači jako standardní sériové zařízení USB CDC, které se zobrazí jako běžný sériový port, a rozhraní SWD zprostředkovává pomocí protokolu CMSIS-DAP, který standardizuje ladění mikrořadičů založených na jádrech ARM Cortex (na kterých je založen i RP2040, který obsahuje dvě Cortex-M0+ jádra).

Na obrázku 59 je schéma zapojení, připojení k ladicí sondě a k displeji je zakresleno jako konektory vlevo:



Obrázek 59: Schéma zapojení hardwaru. Zdroj: vlastní, pomocí schematického editoru balíku KiCad

Ve skutečnosti jsou všechny součásti propojeny na nepájivém poli podle výše zakresleného schématu.

1.1 Napájení

Deska Pi Pico je zde napájena skrze svůj USB konektor, ale kromě něj obsahuje i dva napájecí piny: VBUS a VSYS. VBUS je připojen přímo k USB napájení, nominálních 5 V, a skrze Schottkyho diodu (která má menší úbytek napětí v propustném směru než tradiční polovodičové diody) vede k VSYS, který je připojen k vstupním pinům stejnosměrného měniče napětí, který převede vstupní napětí na nominálních 3.3 V pro napájení. Jediný rozdíl mezi těmito dvěmi piny je, na které straně diody jsou připojeny.

Kromě napájení přes USB lze také desku napájet přibližně 1.8 až 5.5 V na pinu VSYS, ale pokud by mohlo zároveň být připojeno napájení USB, je doporučeno toto sekundární napájení připojit také skrze Schottkyho diodu nebo unipolární tranzistor P-MOSFET (který by měl být připojený na hradle k VBUS, aby se zapnul, pokud napětí VBUS dostatečně klesne, a jinak byl vypnutý). Důvod, proč připojujeme k VSYS a ne VBUS je, aby nás případně dioda ochránila proti neúmyslnému toku proudu zpět ze sekundárního zdroje do připojeného USB zdroje, což by ho mohlo poškodit. (Tento jev se nazývá „backfeeding“ – zpětné napájení.) Pokud je zpětné napájení USB konektoru úmyslné, například při používání USB periferie čipu v módu Host (kdy se sám čip chová jako řídící sběrnice USB a může komunikovat s periferiemi), je ale naopak vhodné připojit zdroj napětí k VBUS, ale v takovém případě by měl mít napětí nominálních 5 V, které USB standard vyžaduje.

2 Podpůrné nástroje

Protože nestačí pouze napsat program, ale je potřeba ho i zkompilovat, nahrát na čip a ladit, je potřeba používat i další nástroje, které toto umožňují.

Pro start projektu byla použita oficiální šablona (4) pro námi použitý HAL (popsaný níže), tudíž mnoho konfiguračních souborů pochází z ní, s pouze drobnými úpravami.

2.1 Cargo

Ekosystém jazyka Rust obsahuje i správce balíčků Cargo, který stahuje závislosti našeho projektu z centrálního registru balíčků a zkompiluje náš projekt, a případně dokáže i zabalit náš projekt a vytvořit nový distribuovatelný balíček (což je v době psaní tohoto dokumentu pouze zkomprimovaný zdrojový kód), který pak může nahrát zpět do centrálního registru. Výchozím registrem je crates.io, ačkoli je možné používat i jiné registry, například intranetové.

Kompilace projektu je pomocí tohoto nástroje velmi jednoduchá: stačí pouze nainstalovat všechny nástroje (pro což je poskytován instalační nástroj zvaný `rustup`), sepsat platné konfigurační soubory, a

v příkazovém řádku spustit příkaz `cargo build`. Cargo automaticky stáhne všechny deklarované závislosti a jejich závislosti, zkompiluje je i náš projekt, a v procesu zvaném „linkování“ je všechny spojí do jednoho spustitelného souboru, pokud se jedná o binární crate, aniž by musel programátor vytvářet komplexní konfigurační soubory nebo jejich generátory. Cargo sám ale pouze přečte konfigurační soubory a invokuje kompilátor jazyka Rust, `rustc`, se všemi potřebnými parametry, který pak provede samotnou kompilaci.

Soubor `Cargo.toml` v kořenovém adresáři popisuje samotný projekt, jeho detaily, závislosti, nastavení atd. Konkrétně, oddíl `[package]` popisuje metadata projektu (jeho název, verzi, edici jazyka[10] a licenci), oddíl `[dependencies]` vypisuje všechny závislosti projektu, jejich verze a volitelné funkce (další kód, který je během kompilace přidán pouze, pokud je daná funkce povolená), a další oddíly popisují nastavení samotného kompilátoru v různých „plánech“ („targets“), což jsou předdefinované „balíčky“ nastavení, abychom nemuseli vypisovat každou možnost zvlášť. V našem projektu používáme plán `dev`, který kompiluje bez optimalizací a obsahuje ladicí informace, a `release`, který maximálně optimalizuje výkon i velikost[11] za cenu ztížené laditelnosti a pomalejší kompilace. Celý soubor je strukturován ve formátu TOML, jak je v ekosystému jazyka Rust běžné.

Soubor `config.toml` v adresáři `.cargo/` konfiguruje samotný Cargo. Zde specifikuje, jaký program spustit po dokončení kompilace při příkazu `cargo run`, abychom program automaticky nahráli a spustili na čipu, jakou úroveň logování máme zájem zobrazit a další nastavení, které má předat přímo kompilátoru, například cílovou architekturu `thumbv6m-none-eabi` (která vlastně říká pouze, že kompilujeme pro instrukční sadu ARMv6-M Thumb, kterou používají jádra Cortex-M0+ použitá v RP2040 bez operačního systému za použití ABI[12], které nevyžaduje matematický koprocesor, který na RP2040 chybí).

2.2 probe-rs

Pro vše, co se děje po dokončení kompilace, tedy nahrávání programu na čip, jeho spouštění a ladění, je použita sada nástrojů nazvaná probe-rs („probe“ znamená anglicky „sonda“, a koncovka „rs“

[10] Edice je „verze" jazyka Rust, která provádí nekompatibilní změny v samotném jazyce, podobně jako například standardy ANSI C, C99 nebo C23 pro jazyk C. Edice jsou pojmenovávány podle roku vydání, a v tomto projektu používáme nejnovější edici 2024.

[11] Naštěstí, Pi Pico obsahuje velmi štědré 2 MiB (=2048 KiB) paměti flash, v porovnání s jinými deskami, například BBC micro:bit v2, který má 512 KiB, nebo Arduino Uno Rev3, obsahující pouze 32 KiB. Samotný RP2040 podporuje až 16 MiB.

[12] ABI („=application binary interface“ – aplikační binární rozhraní) je standard specifikující, jak si funkce mezi sebou předávají parametry. Pokud neprogramujeme v jazyce symbolických adres nebo nevytváříme kompilátory, není třeba nad ním příliš přemýšlet.

neoficiálně označuje projekty napsané v jazyce Rust nebo určené pro jeho ekosystém). Tato sada nástrojů využívá právě výše zmíněné ladicí sondy, skrze kterou čip ovládá a komunikuje s ním.

Jak bylo k vidění v souboru `.cargo/config.toml`, abychom spustili náš program, je automaticky spuštěn příkaz `probe-rs run --chip RP2040 --protocol swd --preverify --verify`, k jehož konci je ještě automaticky připojena cesta k umístění spustitelného souboru. Tento příkaz říká, aby se probe-rs připojil k první kompatibilní sondě, kterou najde, že je sonda připojena k čipu RP2040 protokolem SWD, aby před nahráváním zkontroloval, jestli již čip neobsahuje shodnou verzi programu (což šetří omezenou životnost flash čipu, aby nebyl přepisován, když dvakrát za sebou spustíme program, aniž bychom ho upravili) a po nahrání opět zkontrolovat, že se program nahrál bezchybně.

Soubor `Embed.toml` v kořenovém adresáři konfiguruje příbuzný nástroj cargo-embed, taky součástí výše zmíněné sady nástrojů, ačkoli ten nepoužíváme.

Pro ladění používáme zásuvný modul pro náš použitý editor zdrojového kódu (Visual Studio Code, zkráceně také jako VSCode), nazvaný prostě „Debugger for probe-rs“. Jeho konfigurace se nachází v souboru `launch.json` v adresáři `.vscode`, kde se obecně nachází všechna konfigurace našeho editoru. Tento soubor obsahuje dvě téměř stejné konfigurace pro ladění, lišící se pouze v tom, zda ladí výstup z kompilačního plánu `dev`, nebo `release` (popsáno výše). Samotný proces ladění je integrován do editoru, a obsahuje mimo jiné například možnost číst logovací hlášky (stejně jako při běžném spuštění), nastavovat breakpointy (popsány v kapitole 4.3 v teoretické části) nebo číst hodnoty proměnných.

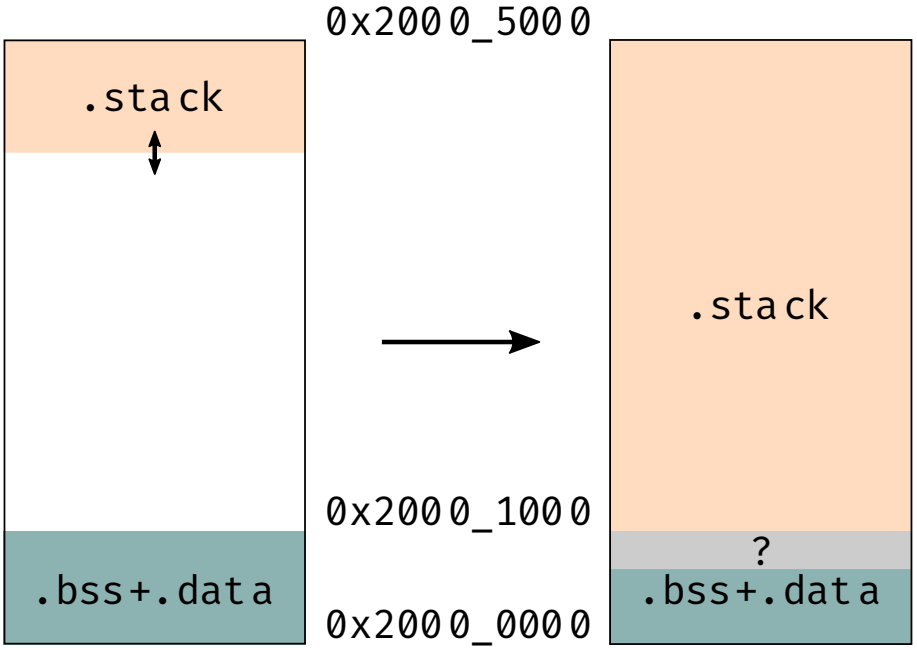
2.3 Linker

Jako linker máme v `.cargo/config.toml` nakonfigurovaný `flip-link`, který je potřeba nainstalovat jako samostatnou utilitu příkazem `cargo install flip-link` a který chrání před poškozením paměti při přetečení zásobníku volání. Na zásobník volání, popsaný detailněji v kapitole 6.1 v teoretické části, je totiž ukládána většina proměnných, ale paměť RAM není neomezená, a proto při pokusu o přidání příliš mnoha dat na zásobník, například při použití rekurze (když určitá funkce volá sama sebe, například během výpočtu Fibonacciho posloupnosti), by mohl zásobník tzv. přetéct – začít ukládat data do míst, kam by neměl – a poškodit data jinde v RAM, tak, že je jednoduše přepíše. To by mohlo mít za následek nedefinované chování[13].

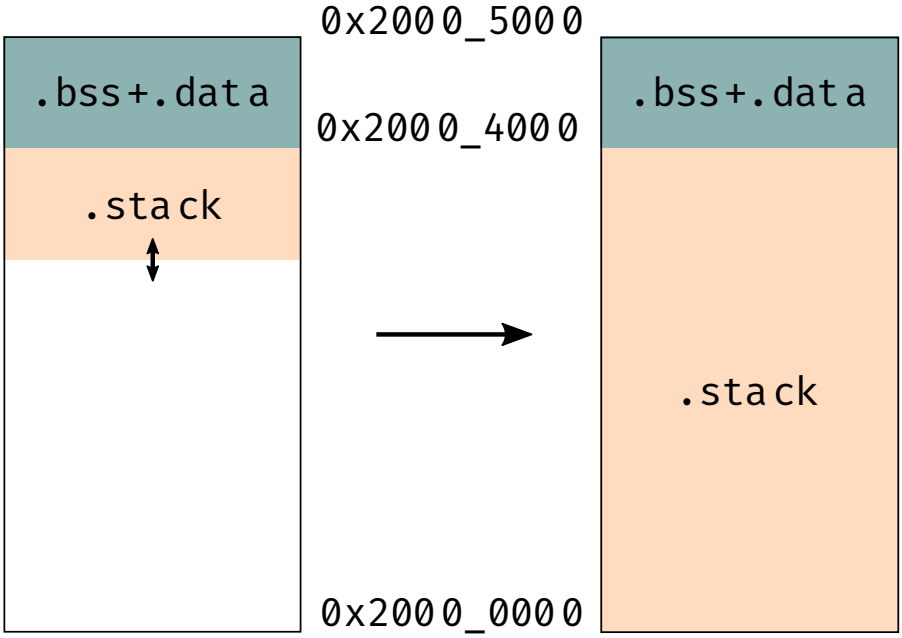
`flip-link` prohodí umístění statických proměnných a zásobníku volání v paměti a tím zabrání možnému poškození paměti tak, že kdyby zásobník přetekl, místo přetečení do oblasti RAM pro statické

[13] To je jednoduše stav, o kterém kompilátor předpokládá, že nikdy nenastane, např. proměnná typu `bool` o hodnotě jiné než 0 a 1. Nikdy k němu nesmí dojít.

proměnné opustí adresní rozsah celé paměti RAM. V adresním prostoru čipu RP2040 nad rozsahem RAM pak leží registry periferie umožňující komunikaci s flash pamětí, tyto registry však leží v rozsahu 0x1800_0000 až 0x1800_00FB, a adresní rozsah RAM začíná 0x2000_0000, tudíž celý rozsah 0x1800_00FC až 0x1FFF_FFFF (téměř 128 MiB) je nedefinovaný a pokus o přístup k němu vyvolá hardwarovou chybu, která obvykle vynutí zamrznutí systému, což je při přetečení zásobníku požadovaný účinek.



Obrázek 60: Poškození paměti při běžném přetečení zásobníku. Zdroj: (5)

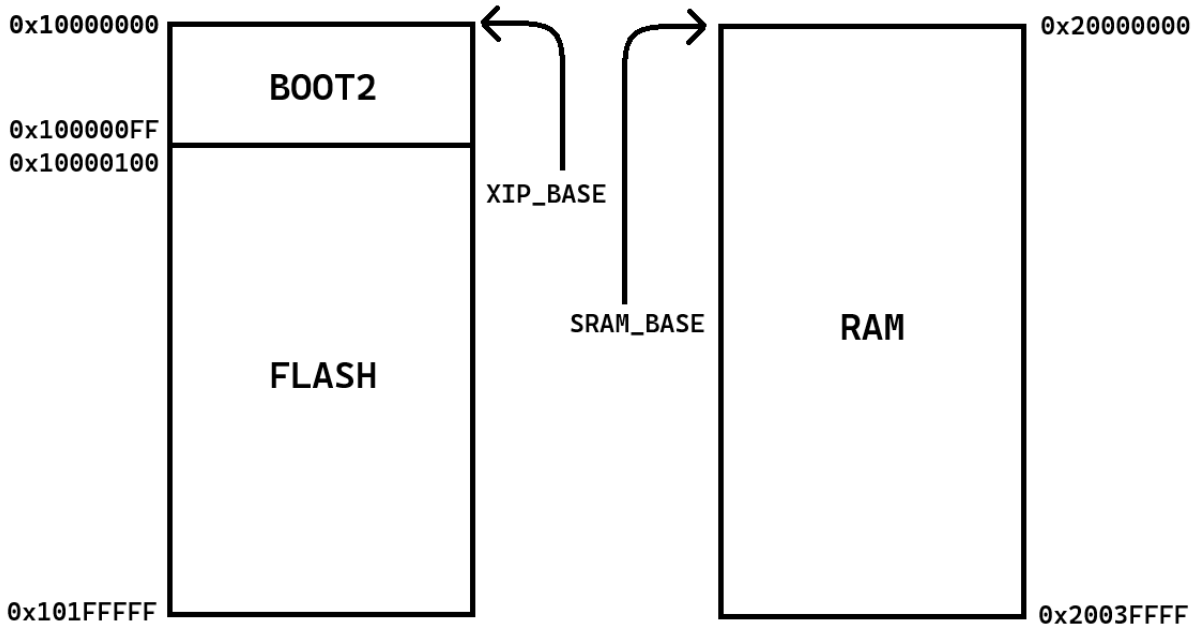


Obrázek 61: Rozložení linkerem flip-link, které místo toho vyvolá hardwarovou chybu. Zdroj: (5)

Linker je dále konfigurován skrze soubor `memory.x` v kořenovém adresáři, kde jsou pouze zadány adresy flash a RAM v adresním rozsahu a jejich velikosti. Zároveň je zajištěno, že data v sekci `.boot2` ve výsledném spustitelném souboru budou umístěna do prvních 256 bajtů ve flash. Řečenými daty je právě zavaděč druhé fáze popsaný v teoretické části v kapitole 3. Jak je zajištěno, aby v této sekci opravdu byl zavaděč boot2, je popsáno níže.

Soubor `build.rs` v kořenovém adresáři obsahuje tzv. kompilační skript („build script“). Sám je zkompilován a spuštěn ještě před začátkem kompilace našeho projektu, a jeho výstup dále konfiguruje Cargo. Na rozdíl od jednoduchých TOML souborů je to však stále spustitelný soubor, který v tomto případě zkopíruje obsah souboru `memory.x` do umístění, kde ho linker dokáže najít, a jeho umístění pak předá linkeru, aby daný soubor použil.

Vizualizace rozložení paměti s šipkami na adresy, které popisuje datasheet, je na obrázku 62:



Obrázek 62: Rozložení paměti, se značkami adres popsaných v datasheetu. Zdroj: vlastní

2.4 Ostatní soubory

Soubor `.gitignore` pouze říká použitému systému sledování verzí, Git, aby při tzv. „commitu“ – uložení aktuálního stavu jako nového bodu v historii repozitáře – ignoroval soubory a adresáře vypsané v tomto souboru. Záznamy začínající na vykřičník naopak zakazují daný soubor ignorovat, pokud by k tomu jinak došlo kvůli předchozímu pravidlu.

V adresáře `.vscode/` jsou i další, méně důležité konfigurační soubory pro použitý editor VSCode, které nijak neovlivňují samotný výsledný projekt, zejm. konfigurace dalšího zásuvného modulu `rust-analyzer`.

V souborech `LICENSE-APACHE` a `LICENSE-MIT` se nachází kopie licencí, pod kterými je projekt licencovaný. Toto umožňuje autorovi se dobrovolně vzdát autorských práv a (případně pod určitými podmínkami) povolit ostatním programátorům používat autorovu práci k různým účelům. Licence, pod kterými je licencovaný tento projekt, jsou Apache License 2.0 a MIT licence, přičemž si uživatel projektu smí vybrat, kterou z licencí musí naplnit (ne obě naráz). Obě licence jsou jednoduché a permisivní, povolující komerční i nekomerční použití, úpravy nebo distribuci projektu, vzdávající se jakékoli záruky. Ačkoli není zcela nutné projekt takto licencovat, je to do jisté míry slušností v programátorské komunitě, pokud uvolňujeme zdrojový kód. Licence byly zvoleny jednoduše tak, že byl tento projekt licencován stejně jako celý programovací jazyk Rust.

Soubor `README.md` obsahuje základní popis projektu a instrukce k jeho kompilaci. Soubor `TODO.md` obsahuje různé osobní poznámky autora k sobě samému, a mohou být ignorované.

3 Knihovny

3.1 HAL

Protože neustále muset přistupovat přímo k registrům je složité a špatně čitelné, existují tzv. HAL („=hardware abstraction layer“ – vrstva pro abstrakci hardwaru), které abstrahují práci s registry do vysokoúrovňových konceptů jako funkce, objekty a traity.

Některé HAL ale nepřistupují k hardwaru, ale poskytují univerzální rozhraní skrze traity, které mohou tradiční hardwarové HAL implementovat, a které mohou ovladače a další generický kód využívat. Generické ovladače přistupují k hardwaru pouze skrze tyto abstrakce a tudíž je možné je používat na jakékoli platformě poskytující implementaci těchto abstrakcí.

Knihovna `cortex-m` poskytuje nízkoúrovňový přístup k registrům a periferiím uvnitř jader Cortex-M, které jsou pro všechny mikrořadiče založené na této architektuře jednotné.

Příbuzná knihovna `cortex-m-rt` poskytuje spouštěcí kód a běhové prostředí. Konkrétně ji využíváme pro definici vstupního bodu pro náš program skrze atribut poskytovaný níže popsanou HAL, který interně využívá právě tuto knihovnu a kvůli kterému je potřeba tuto knihovnu explicitně deklarovat jako závislost.

Asi nejdůležitější knihovnou je však `rp2040-hal`, balíček vysokoúrovňových ovladačů pro RP2040. Tuto knihovnu využíváme pro většinu interakce s hardwarem. Zároveň tato knihovna implementuje všechna aplikovatelná rozhraní z knihovny `embedded-hal`, která je defacto standardem v ekosystému Rustu na mikrořadičích pro výše zmíněnou abstrakci hardwaru a definici univerzálních rozhraní a na které závisí většina generických ovladačů periferií.

Knihovna `rp2040-boot2` je naopak velmi krátká – obsahuje pouze několik statických proměnných, které obsahují již zkompilovaný strojový kód zavaděče boot2, popsaného v kapitole 3 teoretické části. Tento kód pochází přímo od výrobce čipu, knihovnou je nám pouze zprostředkován.

3.2 defmt

`defmt` je (přeloženo do češtiny z oficiální dokumentace) *„vysoce efektivní platforma pro logování zaměřená na zařízení s omezenými zdroji, jako mikrořadiče“* (6). Její efektivita spočívá v tom, že formátování logovacích hlášek tzv. „odkládá“ pro příjemce (počítač ladící daný mikrořadič). To znamená, že při pokusu o logování hodnoty `3.14` se nepokusí mikrořadič začít formátovat dané číslo do textu sám, aby odeslal text, ale odešle pouze samotné číslo a ponechá na příjemci, aby číslo zpracoval do textu a ten pak zobrazil. Ani pevně zakódované textové řetězce neodesílá mikrořadič přímo, ale pouze odešle číslo daného řetězce, který pak příjemce vyhledá u sebe a zobrazí.

Protože však základní knihovna `defmt` definuje pouze protokol a logovací makra, je třeba použít ještě další knihovnu, která zaregistruje transportní vrstvu, která je zodpovědná za samotný přenos dat příjemci. Protože používáme probe-rs, je nejvhodnější použít knihovnu `defmt-rtt`, která komunikuje protokolem SEGGER RTT, který interně využívá zásobníky v paměti RAM.

Pokud by náš program zpanikařil, bude použita knihovna `panic-probe`, která přenese chybovou hlášku přes `defmt` a vyvolá hardwarovou chybu, která způsobí zamrznutí systému.

3.3 Grafika

Jako ovladač pro námi použitý displej používáme výstižně pojmenovanou knihovnu `ssd1306`, která je zodpovědná za komunikaci s displejem. Podporuje protokol SPI i námi použitý I^2C, a volitelně podporuje i asynchronní komunikaci (kterou však nepoužíváme, protože nás program asynchronní není).

Pro vysokoúrovňovou grafiku – vykreslování textu, geometrických obrazců, obrázků atd. – používáme dále knihovnu `embedded-graphics`. Tato knihovna má k sobě přibalené i fonty pro vykreslovaný text, tudíž ji používáme pro většinu celé grafiky. Pro vykreslování obrázků však využíváme i knihovnu `tinybmp`, která dokáže zpracovat obrázky ve formátu BMP do surového binárního formátu, který dokáže `embedded-graphics` vykreslit.

Naše grafická vrstva využívá několik vrstev abstrakce: `embedded-graphics` je abstraktní pro různé displeje, ovladač `ssd1306` implementuje její traity, a zároveň pro interní abstrakci mezi různými komunikačními protokoly využívá interně knihovnu `display-interface` a její sesterské knihovny, které zase využívají `embedded-hal` pro abstrakci mezi různými implementacemi protokolů na různých platformách.

3.4 Ostatní

Protože nevyužíváme dynamickou správu paměti, nemůžeme použít datové struktury s měnitelnou velikostí, které poskytuje standardní knihovna, jako třeba `Vec` (pole) nebo `String` (textový řetězec). Pokud (nebo spíše protože) však přesto potřebujeme podobnou funkcionalitu, používáme knihovnu `heapless`. Tato knihovna také používá pole alokované na zásobník jako úložiště, ale nad něj poskytuje rozhraní podobné strukturám ze standardní knihovny, a skrze interní sledování délky uložených dat emuluje i měnitelnou velikost. Jediným omezením je, že úložné pole musí stále mít pevnou velikost, tudíž mají dané datové typy pevně danou *kapacitu* (=maximální velikost), která musí být známá při kompilaci, opět, protože paměť není alokována dynamicky.

Právě strukturu emulující `Vec` používáme pro implementaci zásobníku – pro práci se zásobníkem pouze přidáváme a odebíráme položky z pole.

4 Aplikační kód

Pro jednoduší čitelnost dělíme náš kód do několika logických modulů, které se nacházejí v různých souborech, všechny v adresáře `src/`. Kořenový modul se nachází v souboru `main.rs`.

4.1 main.rs

4.1.1 Okolní deklarace

Hned na začátku máme dvě deklarace atributů: `#![no_std]` a `#![no_main]`. Atributy v jazyce Rust plní obecnou funkci metadat vložených do zdrojového kódu, které se buď týkají celé crate, ve které jsou definované (když obsahují vykřičník) nebo objektu definovaného bezprostředně po nich (bez vykřičníku). V tomto případě se atributy týkají celého projektu.

Atribut `no_std` říká kompilátoru, že nechceme používat standardní knihovnu, a aby místo toho byla využita minimální standardní knihovna `core`. Standardní knihovnu totiž není možné použít bez operačního systému. Druhý atribut, `no_main`, říká kompilátoru, aby nedefinoval vstupní bod v našem zdrojovém kódu, protože ho definuje nějaký jiný objekt, který do našeho spustitelného souboru nalinkujeme (výše zmíněná knihovna `cortex-m-rt`).

Dále, pomocí klíčového slova `use`, které redefinuje nějakou (jakoukoliv) cestu z kontextu, kde je použité, jako jediný název existující v kontextu, ve kterém bylo toto klíčové slovo použito, importujeme různé objekty z našich závislostí. Závislosti jsou nám sice přístupné vždy, i bez těchto importů, ale vždy k nim přistupovat skrze jejich plnou cestu by se rychle stalo nečitelným, proto je přejmenováváme do našeho kontextu. Ve výchozím stavu je importován pod stejným názvem, ale pomocí použití klíčového slova `as` je možné dát předmětu nebo modulu jakýkoli název. Pomocí složených závorek můžeme kompaktně

importovat více předmětů nebo modulů naráz. Klíčové slovo `self` pak označí modul mateřský vzhledem k závorkám, a pomocí znaku hvězdičky `*` lze importovat všechny předměty z daného modulu. Když importujeme jako `as _`, tak tím říkáme, že sice tento objekt nebudeme používat přímo, ale chceme ho mít z nějakého důvodu linkovaný do konečného spustitelného souboru, protože definuje něco, co potřebujeme mít dostupné.

Při deklaraci modulu, který nemá žádné tělo (nemá žádné složené závorky), jako děláme ke konci bloku importů, se kompilátor pokusí najít soubor se stejným názvem jako název deklarovaného modulu (a příponou `.rs`), jehož obsah pak použije jako obsah modulu.

Dále deklarujeme statické proměnné a konstanty. Nejdříve použijeme takovou proměnnou z výše popsané knihovny `rp2040_boot2`, která odpovídá flash čipu použitém na desce Pi Pico, a uložíme ji do statické proměnné `BOOT2`. Pomocí atributů pak označíme proměnnou, že je použita, a tedy že ji kompilátor nemá odstranit (atribut `used`), a aby ji kompilátor umístil ve výsledném zkompilovaném souboru do sekce `.boot2` (atribut `link_section = ".boot2"`), kde ji očekává linker, aby ji umístil na správné místo v paměti flash. Protože by však chybné užití atributu `link_section` mohlo vést k umístění dat do sekcí, kam nepatří, je potřeba tento atribut označit jako „nebezpečný“ tak, že ho ohraničíme pomocí `unsafe(...)`.

Dále deklarujeme konstanty pro použitou frekvenci sběrnice I^2C (1 MHz) a zpracováváme soubory obrázků pomocí `tinybmp` do formátu, který je možné použít pro `embedded-graphics`. Mohli bychom je zpracovat i za běhu programu, ale je efektivnější to udělat už během kompilace a výsledek uložit do konstanty, která je pak vložena všude, kde je její hodnota použita. Konstanty a statické proměnné nepodporují inferenci typu.

Aby byly naše logovací hlášky opatřeny časovou značkou, deklarujeme kompaktní funkci, která přečte čas z periferie časovače. Řečeným časovačem je pouze 64bitový registr, který se zvyšuje jednou každou mikrosekundu, ale protože je RP2040 32bitový, je potřeba ho číst z dvou registrů a ty pak sloučit. Tuto funkci pak poskytneme knihovně `defmt` jako zdroj času. Atribut `inline` pouze říká, aby byl obsah funkce vložen přímo do kódu, kde je funkce volána, a aby nebyla zavolána jako běžná funkce (= aby ji kompilátor tzv. inlinoval), což snižuje latenci při pokusu o čtení času, protože není třeba podstupovat všechny procedury při volání funkcí (jako uložení návratové adresy na zásobník volání, aby procesor po konci funkce věděl, kam se má vrátit vykonávat instrukce).

4.1.2 Inicializace

Nakonec je konečně hlavní funkce, označená atributem `hal::entry` (tedy atributem `entry` poskytovaným knihovnou rp2040_hal) jako vstupní bod našeho programu.

Na začátku funkce je jednoduše převzít kontrolu nade všemi periferiemi, získat na ně reference a inicializovat je. Důvod, proč kontrolu *přebíráme*, je, že HAL povoluje pouze jednomu místu v programu mít kontrolu nad periferiemi, aby zabránil souběhu, tudíž žádná jiná funkce nemůže mít kontrolu nad HAL, ale hlavní funkce jí je musí propůjčit (pomocí referencí). Způsob inicializace je poměrně neměnný, takže bylo možné pouze zkopírovat kód z příkladů kódu, které poskytuje HAL, a jen upravit čísla pinů, ke kterým jsme fyzicky připojeni.

Dále zabalíme inicializovanou instanci struktury reprezentující displej do struktury `RefCell`. Jak je popsáno v teoretické části v kapitole 6.2.5, v běžných případech povoluje borrow checker, aby existovala pouze jedna měnitelná reference naráz, ale někdy toto pravidlo není dostatečně flexibilní. Proto poskytuje standardní knihovna `RefCell`, která tyto pravidla kontroluje *za běhu* při zavolání metody `borrow_mut()`. Pokud by v momentě zavolání metody byly porušeny pravidla půjčování, panika je přiměřenou reakcí.

Pro vytvoření instancí zásobníku a textového pole (do kterého uživatel píše čísla, než je přidá na zásobník) používáme „builder pattern“, kdy používáme separátní struktury, které se naplní výchozími hodnotami, které pak můžeme měnit, a které nakonec tzv. postavíme do skutečné výstupní struktury. Pokud je povolená konstanta `BIG_TEXT`, je použit vlastní styl textu s větším fontem, jinak je použit výchozí, menší styl. V obou případech je však nakonec použita metoda `build()`, které pak předáváme referenci na výše popsanou `RefCell`. Po jejich vytvoření vykreslíme obě struktury na displej a oznámíme uživateli přes UART, že inicializace byla dokončena a že vstupujeme do hlavní smyčky programu.

4.1.3 Hlavní smyčka

Hlavní smyčka běží donekonečna, není možné ji opustit jinak než nějakým způsobem resetu mikrořadiče. Smyčky v jazyce Rust je možné navíc označit tzv. štítkem, díky kterému když bychom se pohybovali v nějaké vnořené smyčce, a chtěli bychom okamžitě přeskočit k další iteraci smyčky (podobně jako příkaz `GOTO` v primitivnějších programovacích jazycích), použijeme klíčové slovo `continue` a jméno štítku, `'main`. Během celé smyčky pracujeme s případnými chybami a vykreslujeme displej, je-li to potřeba.

Ze začátku ve smyčce přečteme přesně 1 bajt z UART, převedeme přijatou 8bitovou hodnotu na textový znak a vejdeme do velkého match výrazu, který na základě toho, o jaký znak se jedná, rozhodne, jakou akci vykonat.

Pokud se jedná o znaky odpovídající klávese `Enter`, bude zavolána funkce, která zpracuje text v textovém poli na číslo a dané číslo pak přidá na zásobník. V případě znaku odpovídajícího klávese `Backspace` bude odstraněn poslední znak v textovém poli. Znaky tečky nebo čárky jsou ekvivalentní a

vloží desetinnou čárku do textového pole, pokud tam již žádná není, a před ní nulu, pokud je textové pole prázdné.

Klávesa N otočí znaménko v textovém poli, protože znak spojovníku/mínusu je interpretován jako matematická operace, podobně jako klávesa `+/-` na jiných kalkulátorech. Pokud se tam znaménko mínusu již nachází, odstraní ho, jinak ho přidá. Číslice od nuly do devítky jsou jednoduše přidány do textového pole.

Znaky plus, spojovník (mínus), hvězdička (krát) a lomítko (děleno) jsou běžné znaky na numerických klávesnicích počítačů a zde jsou interpretovány jako operátory – symboly označující matematické operace. Pokud při jejich stisknutí není textové pole prázdné, bude zpracováno jako při stisknutí klávesy `Enter`. Pokud se pak na zásobníku nenachází dostatek čísel pro provedení operace (alespoň 2 čísla), je opět navrácena chyba. Nakonec je opět použit další match výraz, který podle konkrétního znaku vykoná odpovídající operaci. Díky systému traitů nám umožňuje Rust „přetížení operátorů“ („operator overloading“), kdy dokáže programátor implementovat metody, které mají být vykonány při použití daného operátoru s danou dvojicí datových typů. Přetížení by však vždy mělo odpovídat logickému významu operátorů, takže např. při implementaci traitu `Add` („sečíst“) pro dva textové řetězce jsou tyto dva řetězce spojené. V našem případě opět používáme datové typy reprezentující čísla, takže je logický význam prostý. Výsledek matematické operace je opět přidán na zásobník, pokud při operaci nedošlo k žádné chybě.

Bajt o hodnotě 0x12 (=18 v desítkové soustavě) odpovídá klávesové zkratce `Ctrl+R`, a vyvolá opětovné vykreslení zásobníku i textového pole na displej. Hodnota 0x14 (=20) odpovídá stisknutí `Ctrl+T`, a zavolá metodu, která přepne do příkazového módu (popsáno níže). Hodnota 0x1B (=27) pak označuje ASCII znak `ESC`, který označuje začátek ANSI escape kódů, které však v tomto projektu pro jednoduchost ignorujeme – pouze přečteme 10 bajtů přes UART, které zalogujeme a zahodíme.

4.1.4 disp_grave_error(), disp_error()

Pro zobrazování chyb na displeji používáme tuto dvojici funkcí, kterým předáme samotnou `RefCell` obsahující displej. Zatímco `disp_error()` pouze zobrazí v pravém horním rohu malý obrázek – ikonku E, indikující, že se stala nějaká chyba, ze které se dokázal program zotavit, funkce `disp_grave_error()` zobrazí chybový obrázek na celém displeji. Pokud je jí předána reference na strukturu `Delay`, umožňující čekat určitý počet milisekund, počká funkce 10 sekund a resetuje mikrořadič, jinak zpanikaří a zamrzne ho.

4.1.5 parse_textbox()

Funkce `parse_textbox()` slouží pouze k zvýšení čitelnosti kódu, namísto duplikace stejného kódu u kódu pro klávesu `Enter` a u operátorů používáme právě tuto funkci pro kompaktní koordinaci zásobníku, textového pole a asociovaných funkcí pro námi použitý datový typ čísel (popsaný v kapitole 4.4 níže), které převádí textový řetězec na číslo. Zároveň, při změně procesu, jakým je toto provedeno, se změna projeví na obou místech a nemůže se stát, že by programátor zapomněl jedno z míst změnit.

4.2 command_mode.rs

Pro ovládání kalkulačky mimo prosté matematické operace je implementován ještě tzv. příkazový mód, což je speciální mód, do kterého uživatel přepne klávesovou zkratkou `Ctrl+T` a který zpracovává textové příkazy namísto čísel a matematických operací.

Celý soubor obsahuje pouze importy a funkci `handle_commands()`. Tato funkce vyšle příkaz displeji pro převrácení barev displeje – černá se stane bílou, bílá černou – a spustí smyčku čtení UART. Tato smyčka opět čte UART bajt po bajtu; pokud se jedná o alfanumerický znak (A-Z, 0-9) nebo mezeru, přidá ho do textového pole, pokud se jedná o klávesu `Backspace`, poslední znak smaže, pokud se jedná o `Enter`, vystoupí ze smyčky, pokud se jedná o `Ctrl+C`, ukončí celou funkci a navrátí hodnotu indikující, že byla operace zrušena uživatelem. Ostatní znaky ignoruje. Za povšimnutí také stojí, že v tomto souboru (ani v jiných souborech mimo `main.rs`) nepotřebujeme úvodní deklarace `#![no_std]` a `#![no_main]`.

Po konci smyčky (klávesou `Enter`) získá funkce text z textového pole a vejde do dalšího velkého match výrazu, který zpracuje daný text jako příkazy. Některé příkazy mají i kratší aliasy, fungující zcela stejně. Opět, v celém průběhu funkce jsou zpracovávány chyby.

Příkaz `reset` prostě vyvolá restart celého mikrořadiče, aniž by přerušil ladicí připojení. Příkazy `breakpoint` (nebo `bkpt` či `b`) a `breakpoint alt` (nebo `bkpt alt` či `b alt`) jsou jednoduché metody, jak může programátor kdykoli dorazit na ladicí zarážku (popsáno v teoretické části v kapitole 4.3) a analyzovat obsahy proměnných. První příkaz vyžaduje, aby programátor přidal zarážku manuálně ve svém vývojovém prostředí, druhý vyvolá zarážku pomocí instrukce JSA zvané `bkpt`, kterou však ne každé IDE umí správně zpracovat, proto jsou nabízeny obě metody. Příkaz `boot usb` (nebo `usb boot` či `usb`) pak restartuje RP2040 a donutí ho vejít do USB zavaděče, podobně jako při resetu při držení tlačítka BOOTSEL. Příkaz `redraw` (nebo `r`, `f5`, `refresh` či `reload`) vynutí znovuvykreslení zásobníku, aniž by byl jeho obsah upraven. Klávesová zkratka `Ctrl+R` má stejný efekt, jak bylo popsáno výše.

Příkaz začínající na `brightness` (nebo `brt`) je zpracován jako nastavení jasu displeje; textový řetězec příkazu je rozdělen podle poslední mezery, validován pro správnost, a text za poslední mezerou je

zpracován jako číslo. Povolená čísla jsou pouze v rozpětí 1-5, a každé z nich odpovídá vzestupně k jedné z pěti přednastavených úrovní jasu poskytovaných ovladačem displeje.

Příkaz `clear` (nebo `cls` či `c`) vymaže celý obsah zásobníku. Příkaz `duplicate` (nebo `dup`) zduplikuje vrchní předmět na zásobníku, čímž umožňuje jednoduché (i když neergonomické) umocňování přirozenými čísly skrze opakovanou duplikaci a násobení, přestože kalkulačka nemá vlastní klávesu přímo pro mocnění. Příkaz začínající na `drop` opět přijímá jako volitelný parametr číslo oddělené mezerou a odstraní zadaný počet vrchních předmětů na zásobníku. Pokud žádný parametr není dodán, zahodí pouze jeden vrchní. Příkaz `swap` prohodí vrchní dva předměty na zásobníku. Prázdný příkaz je ignorován jako zrušená akce, neexistující příkaz je nahlášen jako chyba.

Po konci match výrazu je vráceno zpět obrácení barev displeje a funkce navrátí hodnotu úspěchu – variantu `Ok`.

4.3 custom_error.rs

V celém projektu je potřeba robustně pracovat s chybami. Není přípustné, aby celý program okamžitě zpanikařil při každé drobné chybě, například když uživatel zadá číslo, které je příliš velké pro použitý datový typ (`i64` – 64bitové celé číslo, schopné pojmout hodnoty mezi -2^{63} a $2^{63}-1$). Paradigma jazyka Rust – chyby jsou pouze dalšími hodnotami – nám umožňuje navracet chyby pomocí `Option` a `Result`, ale ne vždy stačí pouze absence hodnoty, ale chceme komunikovat, co za chybu se stalo, proto jsou ve variantách `Err` přenášena další data.

Ale abychom nemuseli věčně převádět mezi různými typy chyb, které ta či ona knihovna navrací, používáme náš vlastní „supertyp“ – enum, který dokáže obsáhnout všechny možné chyby, které v našem programu mohou nastat, a který poskytuje jakousi jednotnou platformu pro spravování chyb. Tento „supertyp“ byl (nepříliš kreativně) nazván `CustomError`, a jeho varianty dokáží komunikovat mnoho různých chyb, od přetečení zásobníku, po chybu v zpracovávání čísla, po matematické přetečení datového typu, po špatně zformátovaný vstup, po vlastní signály jako „zrušeno“ či „nemožné“. Některé varianty neobsahují žádné další informace, některé obsahují podtyp chyby nahlášený knihovnou.

V souboru se nachází kromě samotného enumu také importy, implementace traitu `From` pro mnoho jiných typů chyb z různých knihoven (což povoluje pro metody z těchto knihoven používat operátor otazníku) a dva pomocné enumy. Ty jsou potřebné proto, že některé knihovny navrací enumy, které neimplementují všechny traity, které ten náš vyžaduje, a proto musíme vytvořit jejich kopii, která má stejné varianty a na které můžeme sami derivovat všechno potřebné. V implementacích `From` pak transparentně převádíme mezi odpovídajícími hodnotami.

Nakonec ještě definujeme krátké aliasy pro všechny typy chyb, se kterými v souboru pracujeme, včetně té naší, pro kompaktnost a čitelnost programu. `CE` jakožto alias `CustomError` dále zveřejňujeme, aby jej mohly importovat i jiné soubory za stejným účelem. Není však vhodné importovat všechny varianty enumu pomocí `from … import *`, protože kdybychom nějakou odstranili, v match výrazech by se z varianty stala všezáchytná proměnná, proto je doporučován spíše krátký alias.

4.4 decfix.rs

Během tvorby tohoto projektu se jeden problém stával stále zřejmějším a zřejmějším; celočíselné dělení. Ve staticky typovaných jazycích, mezi které se řadí i Rust, je typ proměnné pevně daný. Při pokusu o dělení dvou celočíselných hodnot je tedy výsledkem jednoduše opět celočíselná hodnota. Jakákoli desetinná část výsledku je jednoduše zahozena. Od kalkulaček se však očekává přesnost, tudíž tato ztráta informace je nepřijatelná.

Jako očividné a jednoduché řešení se nabízí používat čísla s plovoucí desetinnou čárkou, avšak ani ty nejsou nekonečně přesné, protože jejich velikost je omezena. Větším problémem je však skutečnost, že zabudované datové typy `f32` a `f64` jsou čísla s *binární* plovoucí desetinnou čárkou, což znamená, že hodnoty, které dokáží reprezentovat, nejsou přesně zarovnané s desítkovou soustavou, ve které operují lidé, což vede k nepřesnostem, kdy je potřeba zaokrouhlit na nejbližší *binární* reprezentaci, která však nemusí odpovídat typickému zaokrouhlování v desítkové soustavě. Jedním ze známějších příkladů je $0.1 + 0.2 = 0.30000000000000004$. Standard IEEE 754-2008 sice definuje i čísla s *dekadickou* plovoucí desetinnou čárkou, ale ty nejsou v běžných programovacích jazycích implementovány, proto jsem se rozhodl vyřešit problém vlastnoručně a implementovat čísla s dekadickou pevnou desetinnou čárkou ve struktuře `DecimalFixed`.

Struktura sama o sobě obsahuje pouze dvě hodnoty: `value` a `exponent`. Skutečná, logická hodnota celé struktury je $(value) \times 10\text{^}(exponent)$, tedy pokud se `value` rovná 12301 a `exponent` -3, logická hodnota je 12,301.

Výchozí exponent je, pokud není stanoveno jinak, -9, což odpovídá přesnosti na devět desetinných míst. Tento exponent, spolu s použitým datovým typem `i64` pro `value`, znamená, že kalkulačka dokáže reprezentovat hodnoty v následujícím rozsahu:

$$-9\,223\,372\,036{,}854\,775\,808 \leq x \leq 9\,223\,372\,036{,}854\,775\,807$$

V této kalkulačce jsou nové instance `DecimalFixed` vytvářeny zejména asociovanou funkcí `parse_str`, která je volána funkcí `parse_textbox` z kořenového modulu a která zpracuje textový řetězec na nové číslo. Tato metoda rozdělí vstup na dvě části podle prvního nalezeného znaku tečky zleva, což navrátí iterátor. První prvek, text vlevo od tečky, je zpracován na číslo a „škálován“ – vynásoben mocninou

desíti, aby byla zachována logická hodnota. Druhý prvek, pokud existuje a není prázdným řetězcem (kdyby vstup končil znakem tečky), je upraven, aby měl správnou délku (počet desetinných míst, na jejichž přesnost číslo ukládáme). Pokud je řetězec příliš dlouhý, jeho konec je zahozen, pokud je příliš krátký, je „vycpán“ nulami připojenými na jeho konec. Nakonec je také převeden na číslo a přičten k celému číslu nebo od něj odečten, podle znaménka celého čísla. Tento součet, potažmo rozdíl, je navrácen v hodnotě `value`.

Pro ladění jsou poskytované ještě asociované funkce `new` a `new_prescaled`, které přijímají ne text, ale celá čísla. První funkce vstup škáluje podle dodaného exponentu, druhá přesouvá tuto zodpovědnost na volajícího (a tak umožňuje vytvářet desetinná čísla). Metody `is_negative` a `is_zero` jednoduše informují, zda je číslo záporné nebo nulové, protože struktura z absence nutnosti neimplementuje traity `Ord` ani `Eq`.

Matematické operace jsou poskytovány skrze traity `Add`, `Sub`, `Neg`, `Mul` a `Div` – sčítání, odečítání, negace, násobení a dělení. Tyto traity umožňují výše zmíněné „přetížení operátorů“.

Pro sčítání, pokud jsou exponenty sčítanců stejné, je možné sečíst hodnoty `value` přímo, bez jakýchkoli úprav, jinak je hodnota s nižším exponentem škálována, aby byly exponenty stejné. Odečítání je implementováno jednoduše negací menšitele a použitím implementace sčítání. Negace (číslo opačné) také nijak neupravuje exponent, ale musí pracovat s chybami, protože v dvojkovém doplňku (nejčastější způsob kódování celých čísel) je absolutní hodnota nejnižšího reprezentovatelného čísla o 1 vyšší než nejvyšší reprezentovatelné číslo.

Násobení a dělení je komplikovanější, protože tyto operace mají vliv na exponent, a nejsou implementovány pro operandy s různými exponenty. Při násobení dvou hodnot je exponent změněn podle následujícího vzorce:

$$(v1 \times 10\text{^}e1\,) \times (v2 \times 10\text{^}e2\,) = (v1 \times v2) \times 10\text{^}(e1 + e2)$$

To znamená, že po vynásobení je potřeba výsledek škálovat zpět, aby byl udržen správný exponent. Zároveň, protože při násobení mohou hodnoty růst velmi rychle, jsou mezivýsledky ukládány do datového typu `i128`, aby nedošlo k přetečení, i kdyby byla výsledná hodnota reprezentovatelná.

V případě dělení platí podobný vzorec:

$$(v1 \times 10\text{^}e1\,) \div (v2 \times 10\text{^}e2\,) = (v1 \div v2) \times 10\text{^}(e1 - e2)$$

V tomto případě ale není možné nejdříve provést dělení a až pak škálovat, protože by byla desetinná část ztracena, ale nejdříve je exponent dělence zdvojnásoben, aby byl výsledný exponent stejný jako výchozí. Stejně jako u násobení, i u dělení je využit `i128` pro ochranu před přetečením.

Pro zobrazení hodnot je třeba je formátovat – převést na text, a pro tento účel je implementován trait `Display`. Pokud je exponent nulový, je hodnota formátována přímo. Pokud je kladný, je k samotné hodnotě připojeno ještě tolik nul, jaká je hodnota exponentu, což je efektivnější, než mocnění desítky a násobení, pro stejný výsledek. Pokud je pak exponent záporný, je hodnota zformátována ne jako výstup, ale výsledný text je uložen do proměnné, rozdělen, ošetřen odstraněním nul na jeho konci, a nakonec předán jako výstup s přidáním desetinné tečky.

Kromě výše zmíněné struktury, bloků `impl` k ní přidružených a potřebných importů obsahuje soubor pouze konstanty určující výchozí exponent a velikost struktur `String` použitých při práci s textem.

4.5 stack.rs

Zásobník je také implementován skrze vlastní strukturu, která (pro většinu operací) poměrně tence zabaluje strukturu `Vec` z knihovny `heapless` a obsahuje další logiku pro vykreslování na displej.

Na začátku souboru je importováno mnoho objektů z grafických i jiných knihoven, a v konstantách je nastavena maximální velikost zásobníku, stejně jako velikost vyrovnávací paměti (podobně jako v `decfix.rs`) nebo počet pixelů odstraněných z každého řádku (aby se na displej vešlo více řádků naráz).

Jak je popsáno výše v kapitole 4.1.2, pro inicializaci zásobníku i textového pole používáme „builder pattern“. Ve zdrojovém kódu to znamená, že je třeba definovat pomocnou strukturu `CustomStackBuilder`, která obsahuje pole `disp_dimensions`, `character_style` a `primitives_style`, obsahující rozměry displeje, styl textu a styl primitivních grafických obrazců (konkrétně obdélník, kterým při vykreslování zásobníku vymažeme potřebnou oblast displeje). Rozměry jsou obsaženy v níže popsané struktuře `DisplayDimensions`, která byla importována ze souboru `textbox.rs`.

Při zavolání metody `new()` jsou tyto pole vyplněny výchozími hodnotami, ale je možné je upravit pomocí ostatních metod. Metoda `build()` nakonec vytvoří novou instanci výsledné struktury a zkopíruje do ní svá pole i strukturu `RefCell`, kterou získala jako parametr.

Struktura `CustomStack` obsahuje ještě navíc pole `data` obsahující `Vec`, strukturu z knihovny `heapless`, která emuluje stejně nazvanou strukturu ze standardní knihovny a ve které jsou všechna data uložena, a pole `display_refcell` obsahující neměnnou referenci na `RefCell`, která umožnuje přístup k displeji.

Pro přidání položky na zásobník se nám nabízí metody `push()` přijímající jedinou položku, `push_iterator()` přijímající jakýkoli iterátor, `push_exact_iterator()` přijímající iterátor, který

rovněž implementuje subtrait `ExactSizeIterator`, `push_array()` přijímající pole a `push_slice()` přijímající plátek. `push_slice()` je také omezen ve svém vlastním bloku `impl` tak, že lze použít pouze v případě, kdy datový typ uložený na zásobníku implementuje `Clone`, protože nepřijímá vlastnictví, ale pouze referenci. Ostatní metody přijímají přes hodnotu.

Pro odebrání položky ze zásobníku existují metody `pop()` a `multipop()`. Zatímco `pop()` jednoduše navrací volitelnou hodnotu (hodnotu zabalenou do `Option`) i vlastnictví nad ní, `multipop()` navrací iterátor, který při každé iteraci navrátí jednu hodnotu (včetně vlastnictví nad ní) a odstraní ji, až do specifikovaného maximálního počtu položek, a pokud jeho životnost skončí dříve, než je vyčerpán, zahodí všechny zbývající položky k odstranění.

Poměrně jednodušeji fungují metody `peek()` a `multipeek()`, poskytující možnost nahlédnout na vrchol zásobníku bez odstranění vrchní položky. Tyto metody však navracejí pouze referenci na položky nebo plátek posledních několika položek respektive.

Metoda `clear()` vymaže všechna data ze zásobníku, metoda `len()` se dotáže na délku zásobníku, a metoda `is_empty()` se dotáže, zda je zásobník prázdný – zda je jeho délka 0.

Metoda `draw()` nakonec vykreslí zásobník, potažmo jeho vrchol, na displej. Nejdříve však musí vykreslit černý obdélník s černou výplní, aby efektivně vymazala danou oblast displeje, aby tam nezůstaly dříve vykreslené pixely. Pokud je zásobník prázdný, přeskočí funkce okamžitě k jeho vykreslení na displej a ukončí se, jinak však ještě předtím vypočítá několik pomocných proměnných. Mimo jiné je použita i výše popsaná metoda `multipeek()` pro získání vrchních položek ze zásobníku. Ve smyčce `for` jsou nakonec jednotlivé položky převedeny na text, který je uložen do zásobníku a vykreslen na správné místo na displeji.

4.6 textbox.rs

Pro větší ergonomii při zadávání čísel je ještě používáno textové pole u spodního okraje displeje, který obsahuje zadávaný text – číslice, a jehož obsah je při stisknutí klávesy `Enter` převeden na číslo. Toto umožňuje např. při překlepu stisknout klávesu `Backspace` a upravit zadání, než je přidáno na zásobník, ale nepodporuje však klávesy šipek pro přesun kurzoru – kurzor je čistě vizuální.

Kromě importů podobně jako v jiných souborech využívá tento soubor kromě konstant paralelních k těm nalezeným v souboru stack.rs ještě další; navíc je použita konstanta pro ovládání odsazení textového pole od spodního okraje displeje, pro zapnutí/vypnutí zobrazení kurzoru a pro jeho velikost. Nakonec je kompilátor donucen vyhodnotit během kompilace funkci `_check_consts()`, aby zkontroloval, že pokud je kurzor zapnout, tak že jeho velikost nesmí být větší než odsazení textového pole.

V souboru je definována i výše zmíněná struktura `DisplayDimensions` a její výchozí hodnoty – 128x64 pixelů. Důvod, proč je výchozí hodnota definována dvakrát, je současné omezení jazyka Rust, kdy traitové metody nelze označit `const`, aby bylo možné je spustit během kompilace a jejich výsledek uložit do konstanty. Současné řešení tento problém obchází tak, že definuje metodu, která nenáleží žádnému traitu, a tudíž je možné ji označit jako `const` funkci.

Podobně jako zásobník využívá i textové pole builder pattern pro vytvoření svých instancí a `RefCell` pro přístup k displeji. Metoda `draw()` rovněž vykresluje na displej, ale protože se textové pole skládá pouze z, inu, textu, je vykreslováno mnohem jednodušeji. Podle konstanty je pak volitelně vykreslen i kurzor, který však, jak již bylo řečeno, slouží pouze k vizuálním účelům a ve skutečnosti není pohyblivý.

Pro přidávání textu slouží metody `append_str()`, `append_char()`, `insert_at()` a `insert_str_at()`. První dvě metody přidávají na konec buď textový řetězec (datový typ `str`) nebo znak (datový typ `char`) respektive, druhé dvě tyto dva typy vkládají na specifikované místo (index) v textovém poli – vkládaná data jsou umístěna tak, aby jejich první bajt byl `index` bajtů od začátku, přičemž (stejně jako u pole), index prvního bajtu je nula. Tento index musí ležet na rozhraní dvou znaků, pro případ, že by pole obsahovalo Unicode znaky kódované více než jedním bajtem.

Metody `contains()`, `contains_str()`, `starts_with()`, `starts_with_str()`, `ends_with()` a `ends_with_str()` kontrolují, zda je zadaný znak nebo řetězec obsažen v textovém poli, buď obecně, nebo na začátku, nebo na konci. Protože v době napsání tohoto dokumentu je ještě trait `Pattern`, zajišťující generické implementace ve standardní knihovně, nestabilní, je potřeba separátních metod pro `char` a `str`.

Metody `remove_at()`, `clear()` a `backspace()` pak odstraňují text: `remove_at()` odstraní a navrátí jeden znak na určitém indexu, `clear()` jednoduše vymaže všechen text, a `backspace()` odstraní zadaný počet znaků z konce.

Metoda `get_text()` navrátí kopii zásobníku, se vší měnitelností, zatímco `get_text_str()` navrátí pouze efektivnější referenci. Metoda `len()` navrátí délku textového pole v bajtech (ne ve znacích ani v grafémech, ačkoli, protože v našem případě obsahuje pouze ASCII znaky, počet bajtů je ekvivalentní k počtu znaků/grafémů) a metoda `is_empty()` zkontroluje, zda je textové pole prázdné.

4.7 Další soubory

Používané obrázky se nachází také v adresáři `src/`. Náš kód využívá pouze soubory BMP, ostatní soubory jsou výstupními soubory z editoru Pixilart, který byl použit pro tvorbu obrázků.

ZÁVĚR

Samotný nápad pro projekt jsem dostal víceméně náhodou, protože mi RPN kalkulačky přišly zajímavé, a tak jsem se rozhodl si vytvořit vlastní. První prototyp byl napsán v MicroPythonu, ale kvůli jeho nepraktičnosti a jednoduchosti jsem se rozhodl vyzkoušet nový programovací jazyk Rust, o kterém jsem tolik slyšel.

Ze začátku bylo poměrně náročné se přizpůsobit zcela novém stylu programování, protože jsem neměl vůbec žádné předešlé zkušenosti s nízkoúrovňovým programováním, ale postupně jsem si na něj začal více a více zvykat, a s pomocí mnoha online materiálů jsem se nakonec naučil v něm pracovat. Protože (Micro)Python ukládá své proměnné automaticky na heap, zatímco ve svém projektu jsem heap vůbec neměl, musel jsem se naučit konkrétně paradigma jazyku Rust, které, jak jsem později zjistil, mi poměrně dobře vyhovuje: sledovat vlastníky a životnosti každého objektu a přemýšlet o tom, jak dlouho zůstává ukazatel / reference platná, sice může být občas náročné, ale umožňuje mi si vybudovat dobrý mentální model o tom, kdy a jak lze k objektu přistupovat.

Projekt jsem začal pomocí template z projektu použitého HAL, a nedlouho poté, co jsem začal na projektu pracovat, jsem jeho zdrojový kód (a konfigurační soubory) začal ukládat do Git repozitáře (verzovacího softwaru, který mi umožňuje sledovat historii změn a efektivně je synchronizovat mezi různými zařízeními). V jeho historii lze vidět různé fáze tvorby projektu: v první fázi (na větvi `prototype-1`) probíhal vývoj nejprimitivnějších funkcí projektu jako přidávání a odstraňování ze zásobníku a jeho vykreslování, během druhé fáze (na větvi `prototype-2`) jsem přidal podporu desetinných čísel a robustnější zacházení s chybami, ve třetí fázi (na větvi `prototype-3`) jsem přidal příkazový mód popsaný v kapitole 4.3, a nakonec ve čtvrté fázi (na větvích `prototype-3` a `main`) jsem prováděl různé úpravy a zlepšení podle toho, jak mě napadaly. Původní úmysl byl, že poté, co jsem sloučil poslední prototypovou větev na kořenovou větev `main`, tak bude zdrojový kód „zmražen“ a nebudu ho dále upravovat, ale nakonec jsem stejně musel provést opravu několika závažných chyb a další drobné změny.

Během tvorby projektu jsem používal mnoho nástrojů, od našeptávače („autocomplete“) přes ladicí software až po nástroj GitHub Copilot využívající strojového učení. Copilot funguje jako „inteligentní našeptávač“ doplňující kód na základě polohy kurzoru a okolního kontextu, který pak programátor může stisknutím klávesy `Tab` přijmout, a jako běžný chatbot zodpovídající otázky. Přes to jsem jej však používal pouze v míře natolik omezené, že se stále mohu prohlásit za originálního autora kódu, a Copilot jsem používal pouze jako nástroj – nezadal jsem pouze umělé inteligenci, aby kód napsala za mě.

Výsledkem celého projektu je funkční, otestovaná kalkulačka přijímající vstup v RPN zápisu. Kalkulačka se nedopouští matematických chyb ve svém rozsahu přesnosti. Kromě matematických operací rovněž umožňuje pracovat se samotným zásobníkem – duplikovat, prohazovat a odstraňovat položky – bez jejich úpravy matematickými operacemi. Kód kalkulačky je robustní, zpracovává chyby bez okamžitého pádu a je dostatečně čitelný a udržovatelný, umožňující jednoduché budoucí úpravy. V místech, kde je to třeba, je dále kód opatřen komentáři a základní dokumentací metod a funkcí.

Zdroje

1. **Nichols, Carol aj.** The Rust Programming Language. [Online] 2025. https://doc.rust-lang.org/stable/book/title-page.html.

2. **Klabnik, Steve aj.** Rust by Example. [Online] 2025. https://doc.rust-lang.org/stable/rust-by-example/.

3. **The Rust library team.** The Rust Standard Library Documentation. [Online] 2025. https://doc.rust-lang.org/stable/std/index.html.

4. **rp-rs.** rp2040-project-template. *GitHub.* [Online] 2025. [Citace: 27. březen 2026.] https://github.com/rp-rs/rp2040-project-template.

5. **Ferrous Systems GmbH.** flip-link. *GitHub.* [Online] 2025. [Citace: 27. březen 2026.] https://github.com/knurling-rs/flip-link.

6. —. defmt book. [Online] 29. leden 2026. [Citace: 4. únor 2026.] https://defmt.ferrous-systems.com/.

7. **Raspberry Pi Ltd.** RP2040 Datasheet: A microcontroller by Raspberry Pi. *Raspberry Pi Datasheets.* [Online] 20. únor 2025. [Citace: 26. říjen 2025.] https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf.

8. **Solomon Systech.** SSD1306 Advance Information. [Online] duben 2008. [Citace: 15. listopad 2025.] https://cdn-shop.adafruit.com/datasheets/SSD1306.pdf.

9. **DSS Circuits.** Effects of Varying I2C Pull-Up Resistors. *DSS Circuits.* [Online] 18. prosinec 2010. [Citace: 3. září 2019.] https://web.archive.org/web/20190903234513/http://dsscircuits.com/articles/effects-of-varying-i2c-pull-up-resistors.

10. **Wilson, Huon.** Finding Closure in Rust. *Huon on the internet.* [Online] 8. květen 2015. [Citace: 20. prosinec 2025.] https://huonw.github.io/blog/2015/05/finding-closure-in-rust/.

11. **Raspberry Pi Ltd.** Raspberry Pi Pico Datasheet: An RP2040-based microcontroller board. *Raspberry Pi Datasheets.* [Online] 15. říjen 2024. [Citace: 31. leden 2026.] https://datasheets.raspberrypi.com/pico/pico-datasheet.pdf.

12. **The Rust Cargo team.** The Cargo Book. [Online] 2026. [Citace: 1. únor 2026.] https://doc.rust-lang.org/cargo/index.html.

13. **Wolf, Carol E.** The Shunting Yard Algorithm. *The Department of Mathematics and Computer Science.* [Online] Datum neznámé. [Citace: 20. únor 2026.] https://mathcenter.oxford.emory.edu/site/cs171/shuntingYardAlgorithm/.